



Binaural Rendering Toolbox (BRT)

Technical documentation

Diana Research Group - University of Malaga

This document - Copyright © 2024 Diana Research Group

Table of contents

1. Welcome to Binaural Rendering Toolbox	5
1.1 Quick Start Guide	5
1.2 Components	5
1.3 Credits	5
1.4 Copyright and License	6
1.5 Acknowledgements	6
2. BRT Library	7
2.1 BRT Library: General Overview	7
2.1.1 Introduction	7
2.1.2 Key Features	7
2.1.3 Purpose and Applications	7
2.1.4 Description	8
2.1.5 Usage	10
2.2 Source Models	13
2.2.1 Source Models	13
2.2.2 Omnidirectional Source Model	14
2.2.3 Directivity Source Model	16
2.2.4 Virtual Source Model	18
2.3 Listener Models	19
2.3.1 Listener Acoustic Models	19
2.3.2 HRTF Models	20
2.3.3 RIR models	25
2.4 Environment Models	30
2.4.1 Environment Models	30
2.4.2 Free Field Environment Model	31
2.4.3 SDN Environment Model	33
2.4.4 ISM Environment Model	35
2.5 Bilateral Filter Models	36
2.5.1 Bilateral Filters Models	36
2.5.2 Bilateral SOS Filter Model	37
2.5.3 Bilateral FIR Filter Model	40
2.6 Services Modules	42
2.6.1 Service Modules	42
2.6.2 Spherical FIR Table	45
2.6.3 Spherical Interpolated FIRT able	50

2.6.4	Spherical SOS Table	56
2.6.5	Ambisonic BRIR	59
2.6.6	Room Service Module	61
2.6.7	Old services	64
2.7	Readers	68
2.7.1	Readers	68
2.7.2	SOFA Reader	69
2.7.3	OBJ & MTL File Reader	71
2.8	Infrastructure Entities	73
2.8.1	Infrastructure Entities	73
2.8.2	Processing Modules	74
2.8.3	Common	77
3.	Applications	78
3.1	BRT Applications: General Overview	78
3.1.1	Introduction	78
3.1.2	Key Features	78
3.1.3	Applications	78
3.1.4	Usage	79
3.2	BeRTA Rendeder	80
3.2.1	BeRTA Renderer	80
3.2.2	Settings Menu	81
3.2.3	Resources Menu	88
3.2.4	Configuration File Setup	90
3.2.5	Calibration	92
3.2.6	Safety Limiter	96
3.3	OSC Commands	97
3.3.1	OSC commands defined in BRT	97
3.3.2	Control commands	100
3.3.3	Overall commands	108
3.3.4	Resources commands	112
3.3.5	Source commands	123
3.3.6	Listener commands	130
3.3.7	Environment commands	136
3.3.8	Bilateral Filter commands	140
3.4	BRT OSC API for Unity - Integration Package	141
3.4.1	Package Contents	141
3.4.2	Getting Started: Basic Unity Setup	142
3.4.3	License	143

3.5	BRT OSC API for Matlab	144
3.6	Annotated Receiver Audio SOFA Convention	0
3.6.1	Key Fields	0

1. Welcome to Binaural Rendering Toolbox

The Binaural Rendering Toolbox (BRT) is a set of software libraries, applications, and definitions aimed as a virtual laboratory for psychoacoustic experimentation. The BRT is developed in the framework of the [SONICOM project](#) and include the algorithms developed in the [3D Tune-In Toolkit](#) in a new open, extensible architecture.

1.1 Quick Start Guide

You can use BRT in three different ways depending on the level of integration which you need with your applications:

- If you are a C++ programmer building audio applications, you can use the [BRT Library](#), as a C++ library integrated in your own audio application, which handles audio inputs and outputs.
- If you are using any other platform (Max MSP, PureData, Matlab, Unity, Python, etc.) and want to use BRT, you can use [BeRTA Renderer](#) controlled via OSC commands. BeRTA Renderer is a standalone application which manages audio interfaces and is able to read audio files. It is controllable via OSC commands and is able to perform all functionalities available in the BRT Library.
- If you just want to explore the main BRT features, or test or showcase some custom audio scene using a specific configuration of BRT models using BeRTA Renderer, you can use [BeRTA GUI](#) as a graphical interface to control [BeRTA Renderer](#).

1.2 Components

- [BRT Library](#). Core component that implements the algorithms for binaural rendering, providing a flexible and extensible framework for simulating complex auditory environments in psychoacoustic research.
- [BRT Applications](#).
- [BeRTA Renderer](#). Audio renderer that integrates the BRT library.
- [BeRTA GUI](#). Graphical interface to contro BeRTA Renderer with OSC messages.
- [OSC commands](#). Full definition of commands to control BeRTA Renderer.
- [SOFA files](#). BRT extensively uses files in SOFA format ([AES69-2022](#)) to manage HRTFs, BRIRs, source directivities, Headphone compensation and other binaural filters, near field compensation filters and [annotated audio](#).

1.3 Credits

This software is being developed by a team coordinated by

- [Arcadio Reyes-Lecuona](#) (Diana Research Group, [University of Malaga](#)). Contact: areyes@uma.es
- [Lorenzo Picinali](#) (Audio Experience Design Team, [Imperial College London](#)). Contact: l.picinali@imperial.ac.uk

The current members of the core development team are (in alphabetical order):

- [Maria Cuevas-Rodriguez](#) ([University of Malaga](#))
- [Daniel Gonzalez-Toledo](#) ([University of Malaga](#))
- [Luis Molina-Tanco](#) ([University of Malaga](#))

Other contributors:

- [Marco Fontana](#) (Developer of SDN Environment Model at the [Laboratorio di Informatica Musicale](#), Università degli Studi di Milano)
- [Francisco Morales Benítez](#) (Former contributor at the [University of Malaga](#) mainly in BeRTA GUI and BeRTA renderer)

1.4 Copyright and License

This software is distributed under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

Copyright (c) for each module belongs to its respective authors, who may vary as there are contributions from different institutions in this software. This copyright information is specified in the headers of the corresponding files

This library includes pieces of code from the 3DTI AudioToolkit, shared under GPLv3 license and copyright (c) by University of Málaga (contact: areyes@uma.es) and Imperial College London (contact: l.picinali@imperial.ac.uk). See headers in the source code files.

1.5 Acknowledgements



This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement no.101017743

2. BRT Library

2.1 BRT Library: General Overview

The **Binaural Rendering Toolbox (BRT)** is an advanced C++ library designed to support psychoacoustic research and the development of immersive audio applications. As a core component of the BRT Toolbox, the library provides a modular, open-architecture framework that facilitates the creation of dynamic and reproducible virtual acoustic scenarios. This section introduces the main features, purpose, and usage of the BRT Library.

2.1.1 Introduction

The BRT Library was developed as part of the SONICOM project to address the increasing need for flexible, extensible, and reproducible tools in psychoacoustic research. It builds upon the experience gained from the 3D Tune-In Toolkit while introducing significant architectural improvements, including modularity and ease of extension. The library is intended for researchers and developers aiming to simulate complex binaural scenarios, such as real-time dynamic environments with multiple listeners and sound sources.

2.1.2 Key Features

1. Modularity and Extensibility:

- The library adopts a header-only structure, simplifying integration into various projects.
- It is organized into three layers:
- **Top Layer:** This layer comprises high-level modules, referred to as "models," which include listener, source, and environment models, as well as binaural filters. These modules are interconnected to create diverse acoustic scenarios. *Developers using the BRT Library* to build audio applications will primarily interact with this layer, as it provides a straightforward interface for simulating complex binaural environments.
- **Middle Layer:** This layer contains a collection of signal processing and service modules that serve as the foundation for the high-level models. Signal processing modules include components like convolvers and filters, while service modules manage data such as impulse responses. *Developers contributing new algorithms or models* to the library will need to understand this layer, as it provides the building blocks for extending the library's functionality.
- **Bottom Layer:** At the core of the library, this layer consists of foundational classes and templates that define the modular architecture. It implements mechanisms for interconnecting modules and ensures the flexibility and scalability of the library. This layer is primarily relevant for developers seeking to modify or enhance the library's architecture itself.

2. Dynamic Virtual Acoustics:

- Supports anechoic and reverberant environments.
- Includes **advanced reverberation models**, such as SDN and hybrid approaches combining ISM and convolution.
- Enables simulation of near-field effects and dynamic HRTF swapping.
- Supports 6DoF for both Listeners and sources
- Supports **simultaneous multi-listener rendering** with independent positions, HRTFs, and processing chains.

3. Applications and Portability:

- The library, written in standard C++, is designed for deployment on various platforms, including standalone executables, mobile, and wearable devices. Its modular and multilevel structure enables flexible configurations, making deployment adaptable to diverse systems.

2.1.3 Purpose and Applications

The BRT Library provides a flexible and extensible framework tailored to the needs of researchers and developers in psychoacoustics, auditory modeling, and immersive audio rendering. Its design supports a variety of use cases, including:

- **Custom Binaural Rendering Pipelines:** Configure and implement tailored audio rendering workflows for specific experimental or application needs.
- **Dynamic Auditory Modeling:** Test and validate auditory models with dynamic scenarios, supporting 6DoF of both listeners and sources.
- **Rapid Testing of Simulated Characteristics:** Use integrated applications for quick testing of BRIRs, HRTFs, and other auditory properties.
- **Full Process Chain Characterization:** Characterize complete models, including source, environment, and listener chains, as directional impulse responses or log binaural stimuli along with the position and orientation data used for synthesis.

Developer-Friendly Design

The library minimizes the complexity of implementing novel algorithms and adapting to unique research requirements. Its modular structure offers well-defined internal interfaces and reusable building blocks, enabling efficient customization. Developers can:

- Use **predefined modules** to simulate specific acoustic scenarios, such as free-field or room acoustics.
- Extend the library by creating custom **low-level processing** or **high-level modeling** modules to support new features or research needs.

Active Development and Future Capabilities

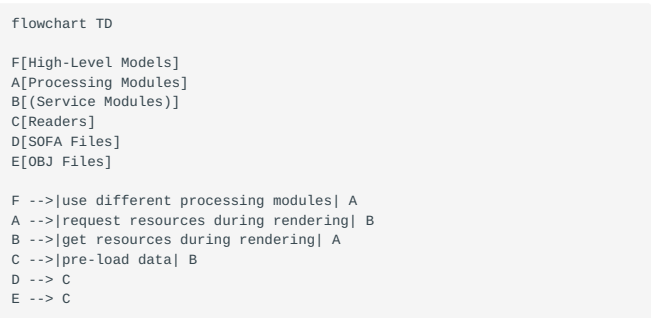
The BRT Library is actively maintained and continuously evolving. Current and planned developments focus on extending the spatial audio capabilities of the framework while maintaining its modular and reproducible design philosophy. Key directions include:

- **Native Ambisonics support**, enabling ambisonic sound fields as primary sources, including dynamic sound field rotation.
- Support for **ambisonic impulse responses** and the introduction of **Ambisonics-based reverberation models**.
- Development of **hearing loss and hearing aid simulations**, broadening the scope of auditory applications.
- **Directivity** of monaural sources and microphones using the new SOFA conventions.

2.1.4 Description

The library is organized into three layers, as previously discussed. From the perspective of a C++ programmer¹, using the library mainly involves working with high-level and service modules.

Architecture Overview



High-Level Modules

High-level modules are responsible for audio rendering. Each module models specific physical and/or psychoacoustic phenomena, depending on the use case. The

following types of models have been implemented, grouped into four categories:

- **Source Models:** Each monaural sound source to be rendered requires the instantiation of a source model. These models serve as the main entry points to the library for applications during rendering. At a minimum, applications must provide the audio samples of each source for every frame and, if applicable, update their position and orientation.
- [Omnidirectional Source Model](#)
- [Directional Source Model](#)
- **Listener Models:** Listener models generate binaural audio by receiving a single audio channel per source and producing two output channels (left and right ears). They can actually include Listener+Environment models using Binaural Room Impulse Responses (BRIR). They are categorized as follows:
- [HRTF Convolution Models:](#) These models simulate sound perception using Head-Related Transfer Functions (HRTF). Signals from sound sources are convolved with binaural impulse responses stored in HRTF Service modules. Whose data is usually read from a SOFA file. Currently two such models have been implemented: **Direct HRTF Convolution Model** and **Ambisonic BRIR Convolution Model**.
- [BRIR Convolution Models:](#) These models use Binaural Room Impulse Responses (BRIR) to simulate acoustic spaces, convolving source signals with room-specific responses stored in HRBRIR service modules. The RIR data will typically be read from a SOFA file and may contain impulse responses with the transmitter and receiver located at various locations in the room. Currently two models have been implemented: **Direct BRIR Convolution Model** and **Ambisonic BRIR Convolution Model**.
- **Environment Models:** These models simulate various acoustic environments:
- [Free Field:](#) Simulates propagation in a free-field environment, including propagation delay, attenuation, and filtering.
- [SDN:](#) Simulates room reverberation using the Scattering Delay Networks method.
- [ISM:](#) Simulates room reverberation using the Image Source Method.
- **Hybrid approaches:** See note below.

- **Bilateral Filter Models:** They perform filtering on binaural signals.
- **Bilateral-SOS Filter Model:** Generic module to perform binaural filtering based on second-order sections, enabling the simulation of devices such as ear protection devices. The SOS coefficients are loaded from SOFA file via a dedicated service module (see [SOS Coefficients Service Module](#)).
- **Bilateral FIR Filter Model:** Perform binaural filtering through convolution (in the frequency domain) with finite impulse responses, enabling applications such as **headphone compensation**, device equalization, or other binaural transfer function processing. The impulse responses are loaded from SOFA files via dedicated service modules (see [GeneralFIR](#)).
- *Non-linear filters: support advanced models such as **hearing loss simulation** (under development).*
- **Listener:** Each listener instantiated in the BRT library represents a “real” listener for which you want to render binaural audio. The application must keep its position/orientation updated and must at the end of each audio frame collect the output samples.

Note on hybrid reverberation approaches

The BRT Library does not implement hybrid reverberation as a single monolithic model.

Instead, hybrid rendering strategies can be achieved by combining multiple high-level modules.

A common hybrid configuration consists of:

- An **ISM Environment Model** to generate early reflections as virtual sources.
- A **BRIR-based Listener Model** to reproduce the late reverberant field through convolution.

This approach reflects the modular design philosophy of the library and allows independent control of early and late reflection processing.

Service Modules

Service modules store the essential data required for rendering. This data typically comes from SOFA files,

although other formats could be used. Key service modules include:

- **SphericalFIRTable:** Stores spherical FIR datasets indexed by direction for fast directional lookup and convolution.
- **SphericalInterpolatedFIRTable:** Stores spherical FIR datasets prepared for interpolation to provide smooth directional transitions.
- **SphericalSOSTable:** Stores spherical SOS filter coefficients indexed by direction for directional IIR filtering.
- **AmbisonicsBIR:** Stores the impulse responses of the virtual loudspeakers in the ambisonic domains, in order to achieve a process with simultaneous impulse responses convolution and ambisonic decoding.
- **Room:** Provides the geometric and acoustic room definition (walls, absorption, and convex enclosure) used by ISM and SDN environment simulations.

OLD SERVICE MODULES (OLD SERVICES PRESENT IN THE LIBRARY UP TO VERSION 2.5.0)

- **HRTF:** Stores head-related impulse responses indexed by azimuth and elevation.
- **BRIR:** Stores room-related impulse responses indexed by azimuth and elevation.
- **DirectivityTF:** Stores transfer functions of a sound source based on the position of the listener and the sources.
- **SOSCoefficients:** Stores coefficients for second-order sections of a filter, which can be fixed or vary based on distance, azimuth, and/or elevation.
- **GenericFIR:** Stores impulse responses indexed by azimuth and elevation.

2.1.5 Usage

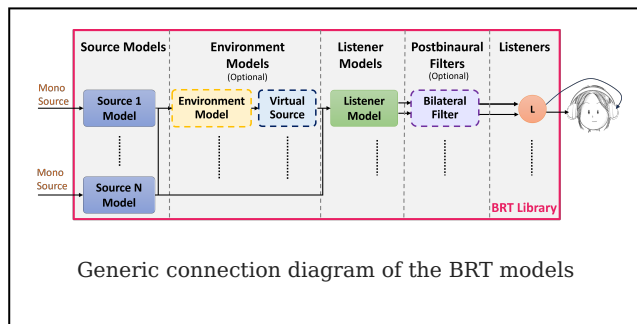
To render audio using the library, the required modules must be instantiated and interconnected based on the desired simulation (configuration). This is managed through the `brtManager` class [URL]. For detailed instructions, refer to the **Setup/Examples** section.

General Workflow

The diagram below outlines the modular interconnections in the BRT Library and the typical workflow for rendering audio:

1. **Source Models:** Instantiate a source model for each monaural sound source. The application is responsible for feeding these models with audio samples for each frame and updating their position and orientation as needed.
2. **Listener Models:** Connect source models to listener models to generate binaural audio. If an environment simulation is required, source models should first be connected to environment models.
3. **Environment Models (Optional):** These models simulate acoustic spaces and produce virtual sources, which are then connected to the listener models. Applications only need to connect the modules; the library handles virtual source generation.
4. **Bilateral Filters (Optional):** To simulate auditory devices, such as headphones, connect a bilateral filter between listener models and listener outputs.
5. **Listeners:** Instantiate one listener module for each real listener. The application must collect the output samples from each listener at the end of every audio frame.

This modular approach provides flexibility while ensuring that all components interact seamlessly. The application is tasked with managing input and output data flow, while the library takes care of the processing.



Examples

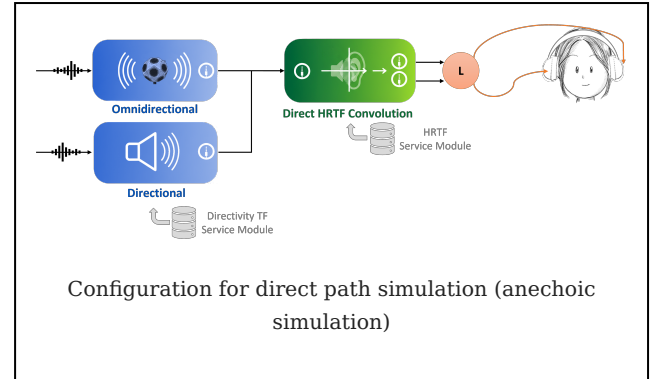
Below are examples of configurations that can be created, three examples of simple model connections and three more complex ones.

How can you do everything we are going to show you now?. If you are a C++ programmer and you are programming with the library, you would need to use the mechanisms provided by the library to instantiate the models and interconnect them. If you are using the BRT Renderer

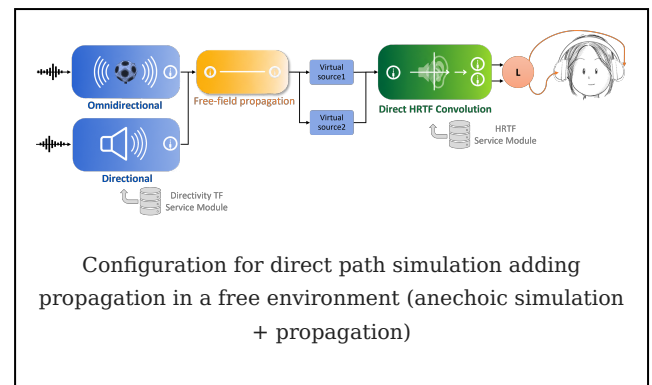
application, you will need to edit a configuration file according to your requirements.

BASIC EXAMPLES

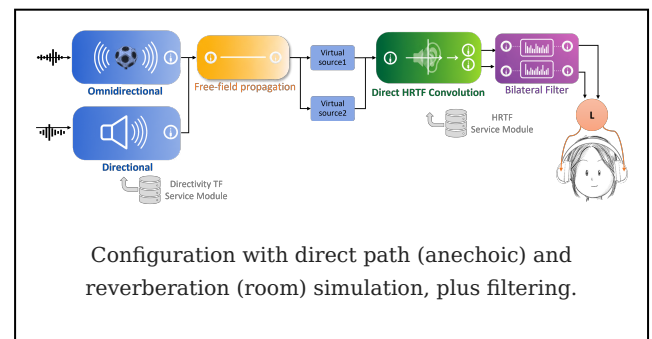
Example 1 - Basic Anechoic Simulation: Combine a listener model with source models for anechoic rendering.



Example 2 - Basic Anechoic Simulation + Propagation simulation: Add an freefield environment model to simulate propagation.



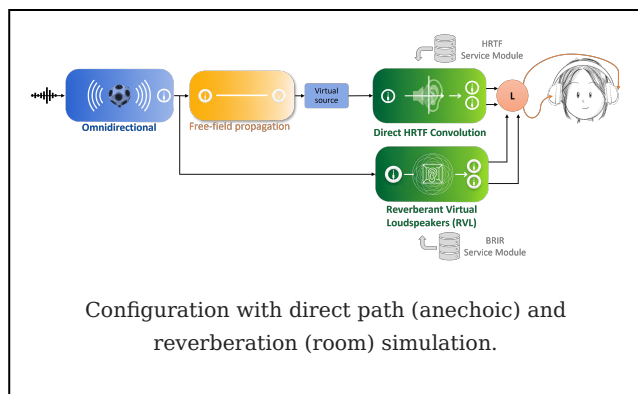
Example 3 - Basic Anechoic Simulation + Propagation simulation + custom filter: A bilateral filter could now be added at the end of the rendering chain. This would allow us to simulate a protection device or compensate for the frequency response of the headphones.



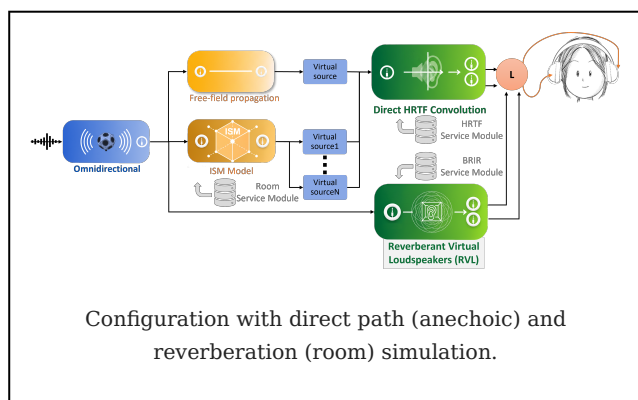
HYBRID SIMULATIONS

Example 4 - Hybrid simulation: Independent simulation for direct sound and room reverberation, using a hybrid approach. A simulation of the direct path using convolution

with HRTF and a simulation of the room using BRIR in the ambisonic domain, employing the Reverberant Virtual Loudspeakers method.

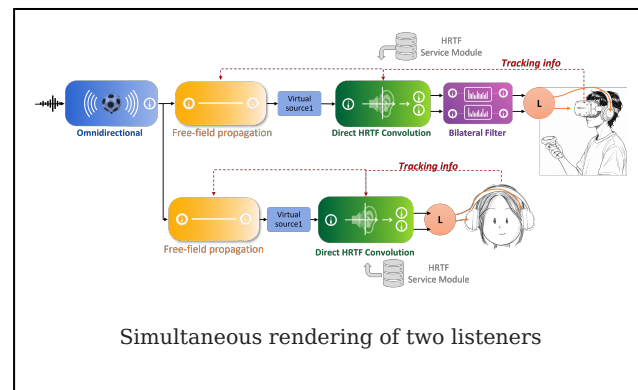


Example 5 - Hybrid Reverberation simulation (Direct path + ISM + BRIR Windowed): A hybrid model with independent simulation for direct sound and room reverberation. The direct path is simulated using convolution with an HRTF. A hybrid approach is used to simulate the room response, with ISM for early reflections and convolution with BRIR for the reverberant tail.



Example 6 - Simultaneous rendering of two listeners:

Simultaneous and independent rendering of two listeners. Each can have their own position in space, HRTF, etc. In addition, filtering is added to one of the listeners to simulate ear protective devices.



For further technical details and code examples, visit the official repository: [BRT Library GitHub](#).

1. If you prefer a higher-level approach, you can use our applications and control them via OSC commands. For more details, see [Applications](#). ←

2.2 Source Models

2.2.1 Source Models

Each monaural sound source to be rendered requires the instantiation of a source model. Each sound source must be connected to a source model that can transform the sound depending on the source position (location and orientation). These models therefore serve as the main library entry points for applications during rendering. At a minimum, applications must provide the audio samples of each source

for each frame and, if applicable, update their position and orientation.

There are currently two fundamental source models and one virtual source model:

- [Omnidirectional Model](#)
- [Directivity Model](#)
- [Virtual Sources Model](#)

2.2.2 Omnidirectional Source Model

The **Omnidirectional Source Model** serves as a straightforward interface for applications to interact with the BRT Library during the rendering process. This model is characterized by its simplicity, as it primarily passes the source's position and the audio samples provided by the application to the connected modules. These modules can include listener models and environment models. Applications must supply monaural audio samples for each frame and update the source's position and orientation when necessary.

Functional Overview

To use the omnidirectional source model, it must first be instantiated and connected to the desired modules, such as listener or environment models. Once connected, the application is responsible for managing the source's position and orientation, which can be updated dynamically as needed using the method `SetSourceTransform`. During each audio frame, the application must also provide the corresponding monaural audio samples, using the `SetBuffer` method.

Connections

Modules to which it connects:

- Environment models
- Listener models

C++ developer

- **File:** /include/SourceModels/SourceOmnidirectionalModel.hpp
- **Class name:** CSourceOmnidirectionalModel
- **Inheritance:** CSourceModelBase
- **Namespace:** BRTSourceModel

Class inheritance diagram

```
classDiagram
    direction TB
    class CSourceModelBase
    <<interface>> CSourceModelBase
    class CSourceOmnidirectionalModel
    class CSourceDirectivityModel
    class CVirtualSourceModel
    CSourceModelBase <|-- CSourceOmnidirectionalModel
    CSourceModelBase <|-- CSourceDirectivityModel
    CSourceOmnidirectionalModel <|-- CVirtualSourceModel
    class CBRTConnectivity
    class CAdvancedEntryPointManager
    class CEntryPointManager
    class CExitPointManager
    class CCommandEntryPointManager
    CEntryPointManager <|-- CAdvancedEntryPointManager
    CAdvancedEntryPointManager <|-- CBRTConnectivity
    CExitPointManager <|-- CBRTConnectivity
    CCommandEntryPointManager <|-- CBRTConnectivity
    CBRTConnectivity <|-- CSourceModelBase
```

How to instantiate

```
// Assuming that the ID of this environment model is contained in
_environmentID.
brtManager.BeginSetup();
std::shared_ptr<BRTSourceModel::CSourceOmnidirectionalModel>
soundSource = brtManager->CreateSoundSource<BRTSourceModel::CSourceOmnidirectionalModel>(soundSourceID, _environmentID);
brtManager.EndSetup();
if (soundSource == nullptr) {
    // error
}
```

How to connect

Connect it to a listener model.

```
// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTSourceModel::CSourceModelBase>
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel = brtManager->GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listenerModelID);
if (listenerModel != nullptr) {
    bool control = listenerModel->ConnectSoundSource(soundSource);
}
```

Connect it to a environment model.

```
// Assuming that the ID of this source model is contained in _sourceID
and
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTEnvironmentModel::CEnvironmentModelBase>
environmentModel = brtManager->GetEnvironmentModel<BRTEnvironmentModel::CEnvironmentModelBase>(_environmentModelID);
if (environmentModel != nullptr) {
    bool control = environmentModel->ConnectSoundSource(_sourceID);
}
```

Public methods

```
void SetBuffer(const CMonoBuffer<float>& _buffer)
CMonoBuffer<float> GetBuffer()

void SetSourceTransform(Common::CTransform _transform)
const Common::CTransform& GetSourceTransform() const

std::string GetID()

TSourceType GetSourceType()
```

2.2.3 Directivity Source Model

The **Directivity Source Model** provides an interface for applications to interact with the BRT Library while implementing frequency-dependent directivity for sound sources. This model takes into account the direction of the listener relative to the source and applies a corresponding filter to transform the audio signal as it would be radiated in that direction. Once processed, the model transmits the filtered audio samples to the connected modules, such as listener or environment models, while also providing real-time updates on the source's position and orientation. Applications must supply monaural audio samples for each frame and dynamically update the source's position and orientation as needed.

Functional Overview

After instantiating the Directivity Source Model and connecting it to the desired modules (listener and/or environment models), the application manages the source's position and orientation using the `SetSourceTransform` method. This allows for dynamic updates to the source's state. For each audio frame, the application must also provide the corresponding monaural audio samples via the `SetBuffer` method.

When the model receives the audio samples, it performs a convolution with the directivity data to account for the directional characteristics of the source. This data is supplied by the a [service module](#), which provides frequency responses based on the source's orientation and the relative position between the source and the listener. The convolutions are executed in the frequency domain using the [uniformly partitioned convolution](#) algorithm, ensuring efficient and accurate processing.

Architecture

The internal block diagram of this class is as follows:

```

flowchart TD
    A[Audio Samples]
    subgraph B[CSourceDirectivityModel]
        subgraph D[CDirectivityConvolver]
            E[CUniformPartitionedConvolution]
        end
    end
    end
    A -->|input| B
    G[(Service Module)]
    G -->|Directivity IR| D
    F[Audio Samples]
    E -->|output| F
  
```

Configuration Options

This model allows configuration by calling its methods or by BRT internal commands:

- **Directivity (on/off):** Omnidirectional sound source when off.
- **DirectivityTF to be used:** The DirectivityTF service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.

Connections

Modules to which it connects:

- Environment models
- Listener models

C++ developer

- **File:** /include/SourceModels/SourceDirectivityModel.hpp
- **Class name:** CSourceDirectivityModel
- **Inheritance:** CSourceModelBase
- **Namespace:** BRTSourceModel

Class inheritance diagram

```

classDiagram
    direction TB
    class CSourceModelBase
    <<interface>> CSourceModelBase
    class CSourceOmnidirectionalModel
    class CSourceDirectivityModel
    class CVirtualSourceModel

    CSourceModelBase <|-- CSourceOmnidirectionalModel
    CSourceModelBase <|-- CSourceDirectivityModel
    CSourceOmnidirectionalModel <|-- CVirtualSourceModel

    class CBRTConnectivity
    class CAdvancedEntryPointManager
    class CEntryPointManager
    class CExitPointManager
    class CCommandEntryPointManager

    CEntryPointManager <|-- CAdvancedEntryPointManager
    CAdvancedEntryPointManager <|-- CBRTConnectivity
    CExitPointManager <|-- CBRTConnectivity
    CCommandEntryPointManager <|-- CBRTConnectivity
    CBRTConnectivity <|-- CSourceModelBase

```

How to instantiate

```

// Assuming that the ID of this environment model is contained in
_environmentID.
brtManager.BeginSetup();
std::shared_ptr<BRTSourceModel::CSourceDirectivityModel> soundSource
= brtManager-
>CreateSoundSource<BRTSourceModel::CSourceDirectivityModel>(soundSourceID
brtManager.EndSetup();
if (soundSource == nullptr) {
    // error
}

```

How to connect

Connect it to a listener model.

```

// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTSourceModel::CSourceModelBase>
std::shared_ptr<BRTLListenerModel::CLListenerModelBase> listenerModel =
brtManager-
>GetListenerModel<BRTLListenerModel::CLListenerModelBase>(_listenerModelID)
if (listenerModel != nullptr) {
    bool control = listenerModel->ConnectSoundSource(soundSource);
}

```

Connect it to a environment model.

```

// Assuming that the ID of this source model is contained in _sourceID
and
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTEnvironmentModel::CEnvironmentModelBase>
environmentModel = brtManager-
>GetEnvironmentModel<BRTEnvironmentModel::CEnvironmentModelBase>(_environm
if (environmentModel != nullptr) {
    bool control = environmentModel->ConnectSoundSource(_sourceID);
}

```

Public methods

```

void SetBuffer(const CMonoBuffer<float>& _buffer)
CMonoBuffer<float> GetBuffer()

```

```

void SetSourceTransform(Common::CTransform _transform)
const Common::CTransform& GetSourceTransform() const

std::string GetID()

TSourceType GetSourceType()

bool SetDirectivityTF(std::shared_ptr< BRTServices::CDirectivityTF >
_sourceDirectivityTF) override
std::shared_ptr<BRTServices::CDirectivityTF> GetDirectivityTF()
override
void RemoveDirectivityTF() override

void SetDirectivityEnable(bool _enabled) override

```

2.2.4 Virtual Source Model

The **Virtual Source Model** is an internal component of the BRT Library, used primarily by certain environment models to simulate complex acoustic scenarios. For each original source, the model generates multiple virtual sources that may differ in position, orientation, and audio samples.

These virtual sources act as independent omnidirectional sources, enabling accurate representation of reflections, reverberation, and other spatial effects.

This model is not intended for direct use by applications but instead functions as a foundational tool for the library's environment models to perform advanced acoustic simulations.

2.3 Listener Models

2.3.1 Listener Acoustic Models

Listener models are high-level modules of the BRT library that handle audio rendering. More specifically these modules try to simulate the binaural sound perceived by the subjects. These models can simulate only the direct sound, or only the reverberation, or all of them together. Currently, four models have been implemented, grouped into two categories:

Listener Models based on HRTFs

The BRT library includes listening models that use head impulse responses (HRTF) to simulate the sound perceived by a user in a realistic way. These models work by convolving the signals from the sound sources with the binaural head impulse response (HRTF). This HRTF will typically be read from a SOFA file. Currently two such models have been implemented:

- [Based on direct convolution with HRTF](#): Simulates the direct path using convolution with HRTFs.
- [Based on convolution with HRTF in the Ambisonics domains](#): Simulates the direct path using convolution with HRTFs in the ambisonic domain.

Listener & Environment Models based on BRIRs

The BRT library includes environment models that leverage room impulse responses (RIRs) to simulate realistic acoustic spaces. These models function by convolving the signals from sound sources with the binaural room impulse response (BRIR) of the room. The RIR will be read from a SOFA file and can contain impulse responses with both emitter and receiver located in several locations in the room. Two models are currently implemented:

- [Based on direct convolution with BRIR](#): Simulates both the direct path and reverberation through convolution with a BRIR.
- [Based on convolution with BRIR in the Ambisonics domain](#): Simulates the direct path and reverberation using convolution with a BRIR in the ambisonic domain.

Others

- *Listener Auditory Model of Hearing Loss: (Under development).*
- *Listener Model of Hearing Aid: (Under development).*

2.3.2 HRTF Models

Listener Models Based on HRTFs

The BRT Library includes listener models that utilize **Head-Related Transfer Functions (HRTFs)** to realistically simulate the sound perceived by a listener. An HRTF is a mathematical representation of how an individual's head, ears, and torso affect the sound arriving from a specific direction before reaching the eardrums. This representation captures how the human auditory system localizes sound in a 3D space, including cues such as interaural time differences (ITD) and interaural level differences (ILD).

By convolving the signals from the sound sources with the binaural HRTFs, these models replicate the way sound interacts with the anatomy of the listener, offering a highly realistic auditory experience. This convolution process enables the simulation of the **direct sound path**, meaning the sound traveling directly from the source to the listener without reflections or reverberation.

Data Source for HRTFs

These HRTF rendering models require HRTF data to be preloaded and pre-processed in a service module. More specifically, this is done in the SphericalFIRTable or the SphericalInterpolatedFIRTable, depending on whether or not the data was interpolated upon loading. For more details, see the documentation on the [service modules](#).

The HRTFs used by these models are typically read from files formatted in the [SOFA \(Spatially Oriented Format for Acoustics\)](#). This standard is widely adopted in the audio research community for storing spatially oriented acoustic data, including head-related impulse responses.

The BRT Library provides an integrated HRTF loader, capable of reading SOFA files and extracting the necessary data for simulation. For more details, see the documentation on the [Readers](#).

Implemented Listener Models

Currently, the following listener models based on HRTFs have been implemented:

- [Direct HRTF Convolution Model](#): Simulates the direct sound path using convolution with HRTFs.
- [HRTF Convolution model in the Ambisonics Domain](#): Simulates the direct sound path using convolution with HRTFs in the ambisonic domain.

These models are designed to provide flexibility and precision for a variety of psychoacoustic experiments and immersive audio applications.

Listener Model based on HRTF direct convolution

OVERVIEW

The **Listener Direct HRTF Convolution Model** module of the library allows the rendering of spatial audio from multiple sound sources. It simulates the direct path between source and listener by taking advantage of *direct convolution* with head impulse responses (HRTF). This module independently processes each of the input sources, taking into account their positions and the position and orientation of the listener.

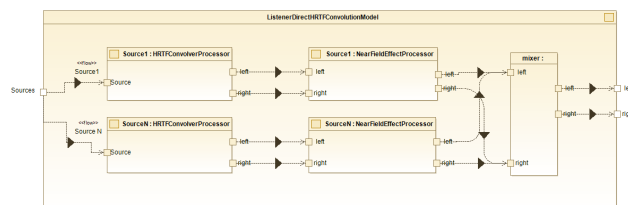
Subsequently, this model performs a *near-field correction*, independent for each of the sources. This correction is made by filtering the signals corresponding to each ear. The coefficients of these filters are dependent on the distance between source and listener and the interaural azimuth.

Finally, the model mixes all channels, left and right separately, to give a single output per ear.

The convolutions are performed in the frequency domain using the uniformly partitioned convolution algorithm. Filtering is performed in the time domain. The HRTF service module is responsible for providing the impulse responses in each frame. While the SOS filter module is in charge of providing the filters coefficients. Both classes must have been previously configured, read more in the [services modules section](#).

ARCHITECTURE

The internal block diagram of this class is as follows:



Listener Direct HRTF Convolution Model Internal Block diagram.

CONFIGURATION OPTIONS

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off):** Silent when off.
- **Gain (float):** Extra gain to be applied to the model output.
- **Spatialization (on/off):** Transparent when off.
- **Interpolation (on/off):** When switched on, HRIRs and delays are calculated at the exact position (relative source-listener position). For this purpose, barycentric interpolation is performed, starting from the three closest points. When it is switched off, the HRIR and delay with the closest position are chosen.
- **Near Field Compensation (on/off):** The near field correction is applied when on.
- **ITD Simulation (on/off):** When activated, a separate delay is added to each ear to simulate the interaural time difference¹. This delay is provided by the HRTF service module and may be provided in the SOFA structure or will be calculated from the head size, for more information see service modules [section](#). When off, it does not simulate interaural time difference.
- **Parallax Correction (on/off):** When it is switched on, a cross-ear parallax correction is applied. This correction is based on calculating the projection of the vector from the ear to the source on the HRTF sphere (i.e. the sphere on the surface of which the HRTF was measured), giving a more accurate rendering, especially for near-field and far-field sound sources. When deactivated, the calculation is based on the centre of the listener's head.
- **HRTF to be used:** The HRTF service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.
- **Nearfield filter (SOS filter) to be used:** The SOS filter service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.

CONNECTIONS

Modules to which it supports connections:

- Source models
- Environment models

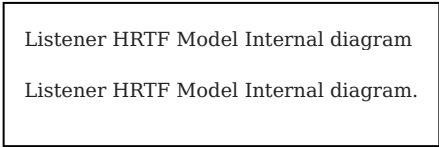
Modules to which it connects:

- Listener
- Billateral Filter

C++ developer

- **File:** /include/ListenerModels/ListenerDirectHRTFConvolution.hpp
- **Class name:** CListenerDirectHRTFConvolutionModel
- **Inheritance:** CListenerModelBase
- **Namespace:** BRTLlistenerModel
- **Classes that instance:**
 - BRTProcessing::CHRTFConvolverProcessor
 - BRTProcessing::CNearFieldEffectProcessor

CLASS INHERITANCE DIAGRAM



HOW TO INSTANTIATE

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
brtManager.BeginSetup();
std::shared_ptr<BRTLlistenerModel::CListenerDirectHRTFConvolutionModel> lis
=
brtManager.CreateListenerModel<BRTLlistenerModel::CListenerDirectHRTFConvo
brtManager.EndSetup();
if (listenerModel == nullptr) {
    // ERROR
}
```

HOW TO CONNECT

Connect it to a listener.

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTLBase::CListener> listener =
brtManager.GetListener(_listenerID);
if (listener != nullptr) {
    brtManager.BeginSetup();
    bool control = listener->ConnectListenerModel(_listenerModelID);
    brtManager.EndSetup();
}
```

Connect an environment model to it.

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTLlistenerModel::CListenerModelBase> listenerModel =
brtManager.GetListenerModel<BRTLlistenerModel::CListenerModelBase>(_listen
if (listenerModel != nullptr) {
    brtManager.BeginSetup();
    bool control = listenerModel-
>ConnectEnvironmentModel(_environmentModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTLsourceModel::CSourceModelBase>
std::shared_ptr<BRTLlistenerModel::CListenerModelBase> listenerModel =
brtManager-
>GetListenerModel<BRTLlistenerModel::CListenerModelBase>(_listenerModelID)
if (listenerModel != nullptr) {
    bool control = listenerModel->ConnectSoundSource(soundSource);
}
```

PUBLIC METHODS

```
void EnableModel() override
void DisableModel() override
```

```

void EnableSpatialization() override
void DisableSpatialization() override
bool IsSpatializationEnabled() override

void EnableInterpolation() override
void DisableInterpolation() override
bool IsInterpolationEnabled() override

void EnableNearFieldEffect() override
void DisableNearFieldEffect() override
bool IsNearFieldEffectEnabled() override

void EnableITDSimulation() override
void DisableITDSimulation() override
bool IsITDSimulationEnabled() override

void EnableParallaxCorrection() override
void DisableParallaxCorrection() override
bool IsParallaxCorrectionEnabled() override

bool SetHRTF(std::shared_ptr<BRTServices::CHRTF > _listenerHRTF)
override
std::shared_ptr<BRTServices::CHRTF> GetHRTF() const override
void RemoveHRTF() override

bool
SetNearFieldCompensationFilters(std::shared_ptr<BRTServices::CSOSFilters>
_listenerILD) override
std::shared_ptr<BRTServices::CSOSFilters>
GetNearFieldCompensationFilters() const override
void RemoveNearFieldCompensationFilters() override

bool
ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool ConnectSoundSource(const std::string & _sourceID) override
bool
DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool DisconnectSoundSource(const std::string & _sourceID) override

bool ConnectEnvironmentModel(const std::string & _environmentModelID)
override
bool DisconnectEnvironmentModel(const std::string &
_environmentModelID) override

void ResetProcessorBuffers()
void UpdateCommand() override

```

1. To perform the convolution task correctly, avoiding comb filters, it is necessary that the delays of the impulse responses have been removed and store them in a separate parameter. ←

Listener Model based on HRTF convolution in the Ambisonics domain

The **Listener Ambisonic Virtual Loudspeakers Model** module enables spatial audio rendering from multiple sound sources by simulating the direct path between the source and the listener through convolution in the ambisonic domain. This is based on the impulse responses stored in an HRTF and the selected ambisonic order (currently implemented up to order 3). The process involves two main stages: a bilateral ambisonic encoding and an ambisonic convolution/decoding.

In the first stage, a bilateral ambisonic encoding is performed for each input sound source. This encoding generates (N) ambisonic channels per ear, where (N) depends on the selected ambisonic order. The encoding process begins by introducing independent delays for each ear to simulate interaural time differences (ITD). Subsequently, a near-field correction is applied through independent filtering of the signals for each ear, with filter coefficients determined by the distance between the source and the listener as well as the interaural azimuth. Finally, ambisonic encoding is carried out separately for each ear, resulting in (N) ambisonic channels for each ear per sound source.

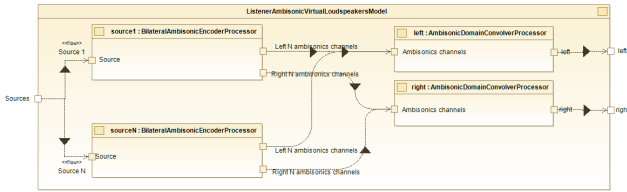
In the second stage, two tasks are performed simultaneously: convolution with impulse responses and ambisonic decoding (for more details, see the [AmbisonicBIR](#) section). There are two convolution/decoding blocks, one for each ear, allowing independent processing for the left and right channels. Each block begins by separately mixing the ambisonic channels generated during the encoding stage. It then performs convolution in the frequency domain for each channel using precomputed impulse responses stored in the [AmbisonicBIR](#) service module. These impulse responses represent the ambisonic mixture of the virtual loudspeaker responses, enabling both convolution and ambisonic decoding to be executed simultaneously. Finally, the output is mixed and transformed back into the time domain, resulting in the final signal for the corresponding ear.

This modular approach ensures efficient and precise spatial audio rendering. The linearity of the operations is leveraged to reduce computational overhead by minimizing the number of convolutions required. The result is a highly realistic simulation of the direct sound path in the ambisonic domain, offering support for scalable ambisonic orders up to the third order.

For further details on the functionality of the [Bilateral Ambisonic Encoder](#) and the [Ambisonic Domain Convolver](#), refer to their respective sections in the documentation.

ARCHITECTURE

The internal block diagram of this class is as follows:



Listener Ambisonic Virtual Loudspeakers Model - Internal diagram

CONFIGURATION OPTIONS

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off):** Silent when off.
- **Near Field Compensation (on/off):** The near field correction is applied when on.
- **ITD Simulation (on/off):** When activated, a separate delay is added to each ear to simulate the interaural time difference¹. This delay is provided by the HRTF service module and may be provided in the SOFA structure or will be calculated from the head size. When off, it does not simulate interaural time difference.
- **Parallax Correction (on/off):** When it is switched on, a cross-ear parallax correction is applied. This correction is based on calculating the projection of the vector from the ear to the source on the HRTF sphere (i.e. the sphere on the surface of which the HRTF was measured), giving a more accurate rendering, especially for near-field and far-field sound sources. When deactivated, the calculation is based on the centre of the listener's head.
- **HRTF to be used:** The HRTF service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.
- **Nearfield filter (SOS filter) to be used:** The SOS filter service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.
- **Ambisonic Order:** The order of the ambisonic coding to be used. Currently only orders between 1 (default) and 3 are valid.
- **Ambisonic Normalization:** The ambisonic normalization to be used. The available options are: N3D (default), SN3D, maxN

CONNECTIONS

Modules to which it supports connections:

- Source models
- Environment models

Modules to which it connects:

- Listener
- Bilateral Filter

C++ developer

- **File:**
\\include\\ListenerModels\\ListenerAmbisonicVirtualLoudspeakersModel.hpp
- **Class name:**
CListenerAmbisonicVirtualLoudspeakersModel
- **Inheritance:** CListenerModelBase
- **Namespace:** BRTLListenerModel
- **Classes that instance:**
- BRTProcessing::CAmbisonicDomainConvolverProcessor
- BRTProcessing::CBilateralAmbisonicEncoderProcessor

CLASS INHERITANCE DIAGRAM

Listener Ambisonic Virtual Loudspeakers class diagram

Listener Ambisonic Virtual Loudspeakers class diagram

HOW TO INSTANTIATE

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
brtManager.BeginSetup();
std::shared_ptr<BRTLListenerModel::CListenerAmbisonicVirtualLoudspeakersModel>
= brtManager.CreateListenerModel<BRTLListenerModel::CListenerAmbisonicVirtualLoudspeakersModel>(_listenerModelID);
brtManager.EndSetup();
if (listenerModel == nullptr) {
    // ERROR
}
```

HOW TO CONNECT

Connect it to a listener.

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTBase::CListener> listener =
brtManager.GetListener(_listenerID);
if (listener != nullptr) {
    brtManager.BeginSetup();
    bool control = listener->ConnectListenerModel(_listenerModelID);
    brtManager.EndSetup();
}
```

Connect an environment model to it.

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel =
brtManager.GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listenerModelID);
if (listenerModel != nullptr) {
    brtManager.BeginSetup();
    bool control = listenerModel->ConnectEnvironmentModel(_environmentModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTSourceModel::CSourceModelBase>
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel =
brtManager.GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listenerModelID);
if (listenerModel != nullptr) {
```

```
bool control = listenerModel->ConnectSoundSource(soundSource);
}
```

PUBLIC METHODS

```
void EnableModel() override
void DisableModel() override

void EnableNearFieldEffect() override
void DisableNearFieldEffect() override
bool IsNearFieldEffectEnabled() override

void EnableITDSimulation() override
void DisableITDSimulation() override
bool IsITDSimulationEnabled() override

void EnableParallaxCorrection() override
void DisableParallaxCorrection() override
bool IsParallaxCorrectionEnabled() override

bool SetAmbisonicOrder(int _ambisonicOrder) override
int GetAmbisonicOrder() override

bool SetAmbisonicNormalization(Common::TAmbisonicNormalization
_ambisonicNormalization) override
bool SetAmbisonicNormalization(std::string _ambisonicNormalization)
override
Common::TAmbisonicNormalization GetAmbisonicNormalization() override

bool SetHRTF(std::shared_ptr< BRTServices::CHRTF > _listenerHRTF)
override
std::shared_ptr<BRTServices::CHRTF> GetHRTF() const override
void RemoveHRTF() override

bool
SetNearFieldCompensationFilters(std::shared_ptr<BRTServices::CSOSFilters>
_listenerILD) override
std::shared_ptr<BRTServices::CSOSFilters>
GetNearFieldCompensationFilters() const override
void RemoveNearFieldCompensationFilters() override

bool
ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool ConnectSoundSource(const std::string & _sourceID) override
bool
DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool DisconnectSoundSource(const std::string & _sourceID) override

bool ConnectEnvironmentModel(const std::string & _environmentModelID)
override
bool DisconnectEnvironmentModel(const std::string &
_environmentModelID) override

void ResetProcessorBuffers()
void UpdateCommand() override
```

1. To perform the convolution task correctly, avoiding comb filters, it is necessary that the delays of the impulse responses have been removed. ←

2.3.3 RIR models

Listener & Environment Models Based on BRIRs

The BRT Library includes environment models that use **Room Impulse Responses (RIRs)** to simulate realistic acoustic spaces. These models function by convolving the signals from sound sources with the **Binaural Room Impulse Response (BRIR)** of a specific room. A BRIR is an extension of the RIR that captures not only the acoustic characteristics of the room but also the binaural filtering effects caused by the listener's anatomy. This allows for a highly immersive audio experience, as the BRIR models how sound interacts with both the environment and the listener's physiology.

The BRIR typically includes both the direct sound path and the reverberation components of the sound field. It is recorded using microphones placed in or near the listener's ears while a sound source emits test signals from different positions in the room. If the BRIR does not include information about the direct sound path, the rendering performed by the library will simulate only the acoustic characteristics of the environment, such as reflections and reverberation. This ensures flexibility in adapting to different BRIR datasets while maintaining accurate spatial rendering.

Data Source for BRIRs

The BRIR rendering models require BRIRsF data to be preloaded and preprocessed in the BRIR service module. For more details, see the documentation on the [SphericalFIRTable service module](#).

The BRIRs used by these models are typically stored in files formatted according to the [SOFA \(Spatially Oriented Format for Acoustics\)](#). This format is widely adopted in acoustic research for storing spatially oriented impulse responses. The BRT Library includes a dedicated module for reading and managing these files. This module is capable of loading BRIR data from SOFA files and extracting the necessary data for simulation. For more details, see the documentation on the [Readers](#).

Implemented Models

Two BRIR-based environment models are currently available in the BRT Library:

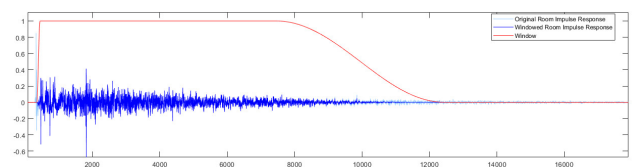
- **Direct BRIR Convolution Model:** In this model, each sound source signal is convolved independently with the corresponding BRIR. This approach ensures the spatial resolution defined in the BRIR dataset, as each source has its own dedicated convolution process.
- **Ambisonic Reverberant Virtual Loudspeakers (RVL) Model:** This model encodes all sound sources into an ambisonic format of order 1, 2, or 3 before performing convolution in the ambisonic domain. By convolving the ambisonic signals with the BRIR, this model achieves more efficient processing when rendering multiple sources, as it requires fewer BRIR datasets for the directions of arrival.

Refining the Impulse Response

To optimize the impulse response, the BRT Library allows for windowing techniques that refine the BRIR. A rectangular window with smoothed edges can be applied using raised cosine fade-in and fade-out transitions, with a configurable slope. This process supports:

- **Silencing Initial Impulse Components:** Useful for removing the direct sound path, which could be simulated separately using a [Listener Model](#), or eliminating early reflections, which might be simulated using the Image Source Method in a hybrid approach.
- **Truncating Late Impulse Components:** Reduces computational cost when working with long BRIRs and allows combining the BRIR convolution with simpler models for late reverberation tails, such as Feedback Delay Networks (FDN).

This flexibility provides greater control over computational efficiency and ensures compatibility with hybrid modeling approaches.



Windowing a Room Impulse Response

Listener & Environment Model based on BRIR direct convolution

The **Listener Direct BRIR Convolution Model** module enables spatial audio rendering from multiple sound sources. It uses direct convolution with the binaural room impulse responses (BRIR¹) to simulate both direct sound and reverberation, providing a complete representation of the acoustic interaction between the source and the listener. If the impulse responses do not include information about the direct path, the simulation is limited to the reverberation of the environment².

Each sound source is processed independently, considering its relative position with respect to the listener. In the final step, the model combines all channels, left and right separately, to produce a single output per ear.

Convolutions are performed in the frequency domain using the uniform partitioned convolution algorithm. These impulse responses are provided by the **BRIR service module** for each audio frame. To ensure proper functioning, the service module must be configured beforehand. More information can be found in the [service section](#).

Distance Attenuation

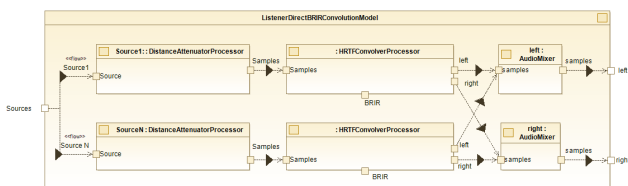
While not entirely accurate, this model also includes a distance simulation, which is disabled by default. This is a simulation of the distance attenuation due to propagation in space. It is based on the **inverse square law**, which states that the intensity of sound decreases proportionally to the square of the distance to the source. This phenomenon, termed **distance-based attenuation**, is a critical factor in our perception of sound intensity.

The input signal is **attenuated based on the distance** between the source and the listener, replicating the natural reduction in sound intensity over distance. This means that a fixed amount of attenuation is applied to the signal each time the distance is doubled from a reference distance. By default, this attenuation has a value of -3 dB and the reference distance is 1 metre. Thus, the applied attenuation is calculated using the following expression:

$$\text{attenuation} = 10^{\left\{ \left(\frac{\text{Distance Attenuation Factor}}{-3 \text{ dB}} \right) * \log_{10} \left(\frac{\text{ReferenceDistance}}{\text{distance}} \right) \right\}}$$

ARCHITECTURE

The internal block diagram of this class is as follows:



Listener Direct BRIR Convolution Model Internal diagram.

CONFIGURATION OPTIONS

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off)**: Silent when off.
- **Spatialization (on/off)**: Transparent when off.
- **Interpolation (on/off)**: When switched on, BRIRs are calculated at the exact position (relative source-listener position). For this purpose, barycentric interpolation is performed, starting from the three closest points. When it is switched off, the HRIR with the closest position are chosen.
- **BRIR to be used**: The BRIR service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.
- **Distance Attenuation (on/off)**: No attenuation is applied when disabled.
- **Distance Attenuation Factor**: Sets a new distance attenuation factor, default is -3dB.

CONNECTIONS

Modules to which it supports connections:

- Source models

Modules to which it connects:

- Listener
- Bilateral Filter

C++ developer

- **File:** /include/ListenerModels/ListenerAmbisonicVirtualLoudspeakersModel.hpp
- **Class name:** CListenerAmbisonicVirtualLoudspeakersModel
- **Inheritance:** CListenerModelBase
- **Namespace:** BRTLListenerModel
- **Classes that instance:**
- BRTProcessing::CHRTFConvolverProcessor

CLASS INHERITANCE DIAGRAM

Listener Direct BRIR Convolution Model Internal diagram

Listener Direct BRIR Convolution Model Internal diagram.

HOW TO INSTANTIATE

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
brtManager.BeginSetup();
std::shared_ptr<BRTLListenerModel::CListenerAmbisonicVirtualLoudspeakersModel>
= brtManager.CreateListenerModel<BRTLListenerModel::CListenerAmbisonicVirtualLoudspeakersModel>(_listenerModelID);
brtManager.EndSetup();
if (listenerModel == nullptr) {
    // ERROR
}
```

HOW TO CONNECT

Connect it to a listener.

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTBase::CListener> listener = brtManager.GetListener(_listenerID);
if (listener != nullptr) {
    brtManager.BeginSetup();
    bool control = listener->ConnectListenerModel(_listenerModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTSourceModel::CSourceModelBase>
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel = brtManager->GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listenerModelID);
if (listenerModel != nullptr) {
    bool control = listenerModel->ConnectSoundSource(soundSource);
}
```

PUBLIC METHODS

```
void EnableModel() override
void DisableModel() override

void EnableSpatialization() override
void DisableSpatialization() override
bool IsSpatializationEnabled() override

void EnableInterpolation() override
void DisableInterpolation() override
bool IsInterpolationEnabled() override

void EnableDistanceAttenuation() override
void DisableDistanceAttenuation() override
bool IsDistanceAttenuationEnabled() override

bool SetDistanceAttenuationFactor(float _distanceAttenuationFactorDB)
```

```
override
float GetDistanceAttenuationFactor() override

void ResetProcessorBuffers()

bool SetHRBRIR(std::shared_ptr<BRTServices::CHRBRIR> _listenerBRIR) override
std::shared_ptr<BRTServices::CHRBRIR> GetHRBRIR() const override
void RemoveHRBRIR() override

bool ConnectSoundSource(const std::string & _sourceID) override
bool ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase> _source) override
bool DisconnectSoundSource(const std::string & _sourceID) override
bool DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase> _source) override

void UpdateCommand() override
```

1. A BRIR captures the acoustic characteristics of a room from the perspective of a specific listener, as it is recorded using microphones placed in the listener's ears. This is why we refer to this module as both a listener and environment model. ←
2. This is the typical way we use the library. ←

Listener Model based on BRIR convolution in the Ambisonics domain

The **Listener Ambisonic Reverberant Virtual Loudspeakers (RVL) Model** enables spatial audio rendering from multiple sound sources. It performs convolution in the ambisonic domain using binaural room impulse responses (BRIR¹) and the selected ambisonic order (currently up to third order). This process simulates both direct sound and reverberation, providing a complete representation of the acoustic interaction between the source and the listener. If the impulse responses do not include information about the direct path, the simulation is limited to the reverberation of the environment². The process consists of two main stages: ambisonic encoding and ambisonic convolution/decoding.

In the first stage, a ambisonic encoding is performed for each input sound source. This encoding generates (N) ambisonic channels per ear, where (N) depends on the selected ambisonic order, and per sound source.

In the second stage, two tasks are performed simultaneously: convolution with impulse responses and ambisonic decoding (for more details, see the [AmbisonicBIR](#) section). There are two convolution/decoding blocks, one for each ear, allowing independent processing for the left and right channels. Each block begins by separately mixing the ambisonic channels generated during the encoding stage. It then performs convolution in the frequency domain for each channel using precomputed impulse responses stored in the [AmbisonicBIR](#) service module. These impulse responses represent the ambisonic mixture of the virtual loudspeaker responses, enabling both convolution and ambisonic decoding to be executed simultaneously. Finally, the output is mixed and transformed back into the time domain, resulting in the final signal for the corresponding ear.

This modular approach ensures efficient and precise spatial audio rendering. The linearity of the operations is leveraged to reduce computational overhead by minimizing the number of convolutions required. The result is a highly realistic simulation of the direct sound path in the ambisonic domain, offering support for scalable ambisonic orders up to the third order.

For further details on the functionality of the [Bilateral Ambisonic Encoder](#) and the [Ambisonic Domain Convolver](#), refer to their respective sections in the documentation.

Distance Attenuation

While not entirely accurate, this model also includes a distance simulation, which is disabled by default. This is a simulation of the distance attenuation due to propagation in space. It is based on the **inverse square law**, which states that the intensity of sound decreases proportionally to the

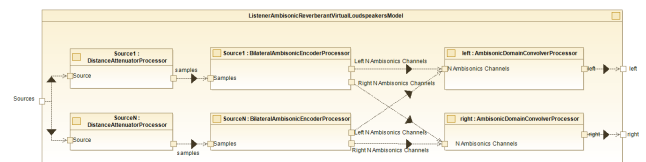
square of the distance to the source. This phenomenon, termed **distance-based attenuation**, is a critical factor in our perception of sound intensity.

The input signal is **attenuated based on the distance** between the source and the listener, replicating the natural reduction in sound intensity over distance. This means that a fixed amount of attenuation is applied to the signal each time the distance is doubled from a reference distance. By default, this attenuation has a value of -3 dB and the reference distance is 1 metre. Thus, the applied attenuation is calculated using the following expression:

$$\text{attenuation} = 10^{\left\{ \left(\text{Distance Attenuation Factor} / -3 \text{ dB} \right) * \log_{10}(\text{ReferenceDistance} / \text{distance}) \right\}}$$

ARCHITECTURE

The internal block diagram of this class is as follows:



Listener Ambisonic Reverberant Virtual Loudspeakers (RVL) Model - Internal diagram.

CONFIGURATION OPTIONS

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off):** Silent when off.
- **BRIR to be used:** The BRIR service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.
- **Ambisonic Order:** The order of the ambisonic coding to be used. Currently only orders between 1 (default) and 3 are valid.
- **Ambisonic Normalization:** The ambisonic normalization to be used. The available options are: N3D (default), SN3D, maxN
- **Distance Attenuation (on/off):** No attenuation is applied when disabled.
- **Distance Attenuation Factor:** Sets a new distance attenuation factor, default is -3dB.

CONNECTIONS

Modules to which it supports connections:

- Source models

Modules to which it connects:

- Listener
- Bilateral Filter

C++ developer

- File: /include/ListenerModels/ListenerAmbisonicReverberantVirtualLoudspeakersModel.hpp
- Class name: CListenerAmbisonicReverberantVirtualLoudspeakersModel
- Inheritance: CListenerModelBase
- Namespace: BRTLListenerModel
- Classes that instance:
- BRTProcessing::CAmbisonicDomainConvolverProcessor
- BRTProcessing::CBilateralAmbisonicEncoderProcessor

CLASS INHERITANCE DIAGRAM

Listener Ambisonic Reverberant Virtual Loudspeakers (RVL) Model class diagram

Listener Ambisonic Reverberant Virtual Loudspeakers (RVL) Model - class diagram.

HOW TO INSTANTIATE

```
// Assuming that the ID of this listener model is contained in
_listenerModelID.
brtManager.BeginSetup();
std::shared_ptr<BRTLListenerModel::CListenerAmbisonicReverberantVirtualLou
=
brtManager.CreateListenerModel<BRTLListenerModel::CListenerAmbisonicReverb
brtManager.EndSetup();
if (listenerModel == nullptr) {
    // ERROR
}
```

HOW TO CONNECT

Connect it to a listener.

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTBase::CListener> listener =
brtManager.GetListener(_listenerID);
if (listener != nullptr) {
    brtManager.BeginSetup();
    bool control = listener->ConnectListenerModel(_listenerModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the soundSource could be a ID(string) or a
std::shared_ptr<BRTSourceModel::CSourceModelBase>;
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel =
brtManager->GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listenerModelID)
if (listenerModel != nullptr) {
    bool control = listenerModel->ConnectSoundSource(soundSource);
}
```

PUBLIC METHODS

```
void EnableModel() override
void DisableModel() override

bool SetAmbisonicOrder(int _ambisonicOrder) override
int GetAmbisonicOrder() override

bool SetAmbisonicNormalization(Common::TAmbisonicNormalization
_ambisonicNormalization) override
bool SetAmbisonicNormalization(std::string _ambisonicNormalization)
override
Common::TAmbisonicNormalization GetAmbisonicNormalization() override
```

```
void EnableDistanceAttenuation() override
void DisableDistanceAttenuation() override
bool IsDistanceAttenuationEnabled() override

bool SetDistanceAttenuationFactor(float _distanceAttenuationFactorDB)
override
float GetDistanceAttenuationFactor() override

bool SetHRBRIR(std::shared_ptr<BRTServices::CHRBRIR> _listenerBRIR)
override
std::shared_ptr<BRTServices::CHRBRIR> GetHRBRIR() const override
void RemoveHRBRIR() override

bool
ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool ConnectSoundSource(const std::string & _sourceID) override
bool
DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool DisconnectSoundSource(const std::string & _sourceID) override

void ResetProcessorBuffers()
void UpdateCommand() override
```

1. A BRIR captures the acoustic characteristics of a room from the perspective of a specific listener, as it is recorded using microphones placed in the listener's ears. This is why we refer to this model as both a listener and environment model. ←
2. This is the standard way we use the library. ←

2.4 Environment Models

2.4.1 Environment Models

Environmental simulations play a crucial role in achieving realistic binaural audio rendering by replicating how sound interacts with its surroundings. These simulations model the acoustic behavior of spaces, including reflections, reverberation, and the effects of air propagation, to recreate an immersive auditory experience that closely resembles real-world scenarios. By combining environmental models with listener and source models, the BRT Library ensures accurate spatial audio reproduction.

What we call environment models in the BRT Library are models that simulate the propagation of sound in a given acoustic environment using geometric algorithms based on

virtual sources. Currently, the BRT Library provides two environmental models:

- **Free Field Environment Model:** Simulates direct sound propagation in open spaces, accounting for propagation delay, distance-based attenuation, and filtering effects caused by the medium.
- **SDN Environment Model:** Implements room acoustics simulation using Scattering Delay Networks¹, a computationally efficient algorithm for modeling reflections and reverberation in enclosed spaces.
- **ISM Environment Model:** Simulates room early reflections using the Image Source Method in complex room shapes.
- **Hybrid approaches:** The BRT Library does not implement hybrid reverberation as a single monolithic model. Instead, hybrid rendering strategies can be achieved by combining multiple high-level modules. A common hybrid configuration consists of:
 - An **ISM Environment Model** to generate early reflections as virtual sources.
 - A **BRIR-based Listener Model** to reproduce the late reverberant field through convolution.

This approach reflects the modular design philosophy of the library and allows independent control of early and late reflection processing.

And we are working to incorporate the following:

- **Amibisonic Reverberation** : (Under Development)

1. de Sena, E., Hacıhabiboglu, H., & Cvetkovic, Z. (2011, February 2). Scattering Delay Network: An Interactive Reverberator for Computer Games. 41st International Conference: Audio for Games. ←

2.4.2 Free Field Environment Model

The **Free Field Environment Model** simulates sound propagation in a free-field environment, focusing on the direct sound path. Specifically, it models the propagation delay, distance-based attenuation, and filtering effects caused by the medium.

In a free-field environment, sound propagates without any interaction with obstacles or boundaries, such as walls or objects, allowing it to travel unimpeded from the source to the listener. This type of propagation is characterized by the **inverse-square law**, which states that the sound intensity decreases proportionally to the square of the distance from the source. This phenomenon, known as **distance-based attenuation**, plays a crucial role in how we perceive the loudness of sounds in open spaces.

Additionally, sound waves traveling through a medium, such as air, are subject to **frequency-dependent filtering**. Higher frequencies are more susceptible to attenuation due to energy absorption by the medium, which results in a natural filtering effect as the distance increases. This filtering can significantly impact the timbre of sounds heard at greater distances.

The model also accounts for **propagation delay**, which represents the time it takes for sound to travel from the source to the listener. This delay is determined by the speed of sound in the medium, which is approximately 343 m/s in air under standard atmospheric conditions. By accurately simulating these factors, the Free Field Environment Model provides a realistic representation of sound propagation in open environments.

Functional overview

The **Free Field Environment Model** simulates sound propagation in open spaces. It processes monaural input signals connected from source models and generates virtual monaural output sources that are connected to listener models. For each source connected to the model, a corresponding virtual output source is created to represent the processed signal. The user only needs to specify the listener model to which this environment model is connected, after which sources can be freely added or removed as needed.

SIGNAL PROCESSING CHAIN

The model’s signal processing involves three main stages. First, the input signal is **attenuated based on the distance** between the source and the listener, replicating the natural reduction in sound intensity over distance. This means that a fixed amount of attenuation is applied to the signal each time the distance is doubled from a reference distance. By default, this attenuation has a value of -6.0206

dB and the reference distance is 1 metre. Thus, the applied attenuation is calculated using the following expression:

$$\text{attenuation} = 10^{\left\{ \left(\text{Distance Attenuation Factor} / -6.0206 \text{ dB} \right) * \log_{10} \left(\text{ReferenceDistance} / \text{distance} \right) \right\}}$$

Although *not yet implemented*, a **filtering** stage will eventually simulate the frequency-dependent attenuation effects caused by propagation through air

Finally, the signal passes through a **delay line** that accurately models the propagation delay corresponding to the distance between the source and listener. This sequence ensures that the output signals reflect realistic free-field sound propagation.

DYNAMIC ADAPTATION

The delay line is dynamically updated to account for changes in the relative positions of the source and the listener, maintaining accurate modeling of propagation effects in real-time. This dynamic behavior makes the Free Field Environment Model a vital component for simulating direct sound paths in spatial audio systems.

Configuration Options

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off):** Silent when off.
- **Gain (float):** Extra gain to be applied to the model output.
- **Distance Attenuation (on/off):** No attenuation is applied when disabled.
- **Propagation Delay (on/off):** No delay is applied when disabled.
- **Distance Attenuation Factor:** Sets a new distance attenuation factor, default is -6.0206dB.

Connections

Modules to which it supports connections:

- Source models

Modules to which it connects:

- Listener models

C++ developer

- **File:** /include/EnvironmentModels/FreeFieldEnvironmentModel.hpp
- **Class name:** CFreeFieldEnvironmentModel
- **Inheritance:** CEnvironmentModelBase
- **Namespace:** BRTEnvironmentModel
- **Classes that instance:**
- BRTEnvironmentModel::CFreeFieldEnvironmentProcessor

Class inheritance diagram

Free field Model Internal diagram

Free field Model Internal diagram.

How to instantiate

```
// Assuming that the ID of this environment model is contained in
_environmentID.
brtManager.BeginSetup();
std::shared_ptr<BRTEnvironmentModel::CFreeFieldEnvironmentModel>
environmentModel =
brtManager.CreateEnvironment<BRTEnvironmentModel::CFreeFieldEnvironmentMo
brtManager.EndSetup();
if (environmentModel == nullptr) {
    // error
}
```

How to connect

Connect it to a listener model.

```
// Assuming that the ID of this environment model is contained in
_environmentModelID and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTLListenerModel::CLListenerModelBase> listenerModel =
brtManager.GetListenerModel<BRTLListenerModel::CLListenerModelBase>(_listen
if (listenerModel != nullptr) {
    brtManager.BeginSetup();
    bool control = listenerModel-
>ConnectEnvironmentModel(_environmentModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the ID of this source model is contained in _sourceID
and
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTEnvironmentModel::CEnvironmentModelBase>
environmentModel = brtManager-
>GetEnvironmentModel<BRTEnvironmentModel::CEnvironmentModelBase>(_environm
if (environmentModel != nullptr) {
    bool control = environmentModel->ConnectSoundSource(_sourceID);
}
```

Public methods

```
void EnableModel()
void DisableModel()

void SetGain(float _gain)
float GetGain()

void EnableDistanceAttenuation()
void DisableDistanceAttenuation()
bool IsDistanceAttenuationEnabled()
```

```
void EnablePropagationDelay()
void DisablePropagationDelay()
bool IsPropagationDelayEnabled()

bool SetDistanceAttenuationFactor(float _distanceAttenuationFactorDB)
float GetDistanceAttenuationFactor()

bool
ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source)
bool ConnectSoundSource(const std::string & _sourceID)

bool
DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source)
bool DisconnectSoundSource(const std::string & _sourceID)

void ResetProcessorBuffers()

void UpdateCommand()
```

2.4.3 SDN Environment Model

The **SDN Environment Model** implements an acoustic room simulation using **Scattering Delay Networks (SDNs)**. This approach models the acoustics of a room by employing acoustic reverberators, as described by Enso De Sena et al. in the paper [Efficient Synthesis of Room Acoustics via Scattering Delay Networks](#).

Scattering Delay Networks simulate the acoustics of an enclosure using a network of delay lines connected by scattering junctions. The parameters of the model are derived from the physical properties of the simulated room, enabling a realistic approximation of the room's acoustic behavior. SDNs accurately model first-order reflections and make progressively coarser approximations for higher-order reflections. The algorithm supports unequal and frequency-dependent wall absorption, directional sound sources, and microphones, making it versatile for various room configurations.

Functional overview

The **SDN Environment Model** simulates the acoustics of a shoebox-shaped room by processing input signals from connected source models and generating virtual output sources linked to the corresponding listener models. For each connected source, the model creates seven virtual output sources: six representing the reflections from the walls, ceiling, and floor, and one representing the direct sound path. The user only needs to specify the listener model to which the SDN Environment Model is connected, allowing sources to be freely added or removed as required.

For each source, the model constructs scattering nodes and waveguides corresponding to the six reflective surfaces of the room. These virtual sources are positioned based on the geometry of the room and the relative positions of the source and the listener, with the delay and attenuation for each source calculated accordingly. The additional virtual source representing the direct sound path ensures the simulation includes both the reflections and the direct propagation.

To configure the simulation accurately, users must define the room's dimensions and the absorption coefficients for each wall. The model assumes the room's center is located at the origin of the coordinate system (0, 0, 0), ensuring a consistent spatial framework for the simulation. The shoebox-shaped room is described by three parameters:

- **Length:** The dimension of the room along the X-axis.
- **Width:** The dimension of the room along the Y-axis.
- **Height:** The dimension of the room along the Z-axis.

Furthermore, the definition of absorption coefficients is required for each wall, which can be defined as either frequency-independent or frequency-dependent (9 bands).

Architecture

The internal block diagram of this class is as follows:



SDN environment Model Internal diagram.

Configuration Options

This model allows configuration by calling its methods or by BRT internal commands:

- **Model (on/off):** Silent when off.
- **Gain (float):** Extra gain to be applied to the model output.
- **Direct path (on/off):** Direct path silent when off.
- **Reverb path (on/off):** Reverb path silent when off.
- **Setup ShoeBox Room (dimensions):** Set the room geometry. Only shoebox rooms (six walls) are allowed for the SDN model
- **Setup Room Wall Absortion (absortion):** Set walls absorption coefficients. Absorptions are set for nine octave bands: 62.5Hz, 125Hz, 250Hz, 500Hz, 1KHz, 2KHz, 4KHz, 8KHz and 16KHZ

Connections

Modules to which it supports connections:

- Source models

Modules to which it connects:

- Listener models

C++ developer

- **File:** /include/EnvironmentModels/SDNEnvironmentModel.hpp
- **Class name:** CSDNEnvironmentModel
- **Inheritance:** CEnviromentModelBase
- **Namespace:** BRTEnvironmentModel
- **Classes that instance:**
- BRTEnvironmentModel::CSDNEnvironmentProcessor
- Common::CRoom

Class inheritance diagram

Free field Model Internal diagram

Free field Model Internal diagram.

How to instantiate

```
// Assuming that the ID of this environment model is contained in
_environmentID.
brtManager.BeginSetup();
std::shared_ptr<BRTEnvironmentModel::CSDNEnvironmentModel>
environmentModel =
brtManager.CreateEnvironment<BRTEnvironmentModel::CSDNEnvironmentModel>(_
brtManager.EndSetup();
if (environmentModel == nullptr) {
    // error
}
```

How to connect

Connect it to a listener model.

```
// Assuming that the ID of this environment model is contained in
_environmentModelID and
// that the ID of this listener model is contained in
_listenerModelID.
std::shared_ptr<BRTLListenerModel::CListenerModelBase> listenerModel =
brtManager.GetListenerModel<BRTLListenerModel::CListenerModelBase>(_listen
if (listenerModel != nullptr) {
    brtManager.BeginSetup();
    bool control = listenerModel-
>ConnectEnvironmentModel(_environmentModelID);
    brtManager.EndSetup();
}
```

Connect a source model to it.

```
// Assuming that the ID of this source model is contained in _sourceID
and
// that the ID of this environment is contained in
_environmentModelID.
std::shared_ptr<BRTEnvironmentModel::CEnviromentModelBase>
environmentModel = brtManager-
>GetEnvironmentModel<BRTEnvironmentModel::CEnviromentModelBase>(_environm
if (environmentModel != nullptr) {
    bool control = environmentModel->ConnectSoundSource(_sourceID);
}
```

Public methods

```
void EnableModel() override
void DisableModel() override

void SetGain(float _gain) override
float GetGain()

bool SetupShoeBoxRoom(float length, float width, float height)
```

```
Common::CRoom GetRoom()

bool SetRoomWallAbsortion(int wallIndex, float absortion)
bool SetRoomAllWallsAbsortion(float _absortion)
bool SetRoomWallAbsortion(int wallIndex, std::vector<float>
absortionPerBand)
bool SetRoomAllWallsAbsortion(std::vector<float> absortionPerBand)

void EnableDirectPath() override
void DisableDirectPath() override
bool IsDirectPathEnabled() override

void EnableReverbPath() override
void DisableReverbPath() override
bool IsReverbPathEnabled() override

bool
ConnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool ConnectSoundSource(const std::string & _sourceID) override

bool
DisconnectSoundSource(std::shared_ptr<BRTSourceModel::CSourceModelBase>
_source) override
bool DisconnectSoundSource(const std::string & _sourceID) override

void ResetProcessorBuffers()

void UpdateCommand() override
```

2.4.4 ISM Environment Model

⚠️ (Section under construction) ⚠️

2.5 Bilateral Filter Models

2.5.1 Bilateral Filters Models

The objective of this model is to provide capabilities for filtering binaural audio signals. In other words, the filters can be adjusted separately for each channel, and these are processed independently. The implementation of this filtering is not spatially oriented, which means that the filter coefficients do not depend on the position of the source or the listener. Filtering is performed either using IIR filters based on second-order sections or by convolution with impulse responses.

The BRT library currently offers two models of bilateral filters:

- **SOS Bilateral Filter Model:** Generic module to perform binaural filtering based on second-order sections, enabling the simulation of devices such as ear protection devices. The SOS coefficients are loaded from SOFA file via a dedicated service module (see [SOS Coeficients Service Module](#)).
- **FIR Bilateral Filter Model:** Perform binaural filtering through convolution (in the frequency domain) with finite impulse responses, enabling applications such as **headphone compensation**, device equalization, or other binaural transfer function processing. The impulse responses are loaded from SOFA files via dedicated service modules (see [GeneralFIR](#)).

2.5.2 Bilateral SOS Filter Model

Available from BRT v2.5.0

Last update: 9 Jan 2026

Overview

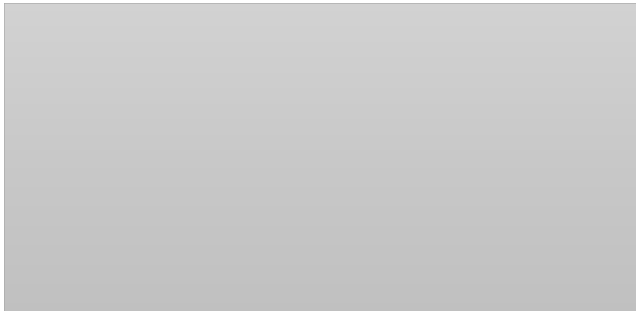
The **Bilateral SOS Filter Model** (class `BRTBilateralFilter::CSOSBilateralFilterModel`, defined in `SOSBilateralFilterModel.hpp`) is a binaural filtering model that applies an **independent chain of second-order sections (SOS / biquads, IIR)** to each ear.

This approach is well suited for simulating devices or effects that are naturally modeled using IIR filters, such as **hearing protection devices** or other ear-dependent equalization systems.

The SOS coefficients are provided through the dedicated service module `SOSCoefficients`, typically loaded from a **SOFA** file.

Architecture

The internal block diagram of this class is as follows:



Bilateral SOS Filter Model Internal Block diagram.

INHERITANCE

- `CSOSBilateralFilterModel` derives from `CBilateralFilterModelBase`, which itself inherits from `BRTBase::CModelBase`.
- The model is integrated into the processing graph through the `BRTBase::CBRTManager`.

INTERNAL COMPONENTS

The main internal elements of the model are:

- `BRTServices::CSOSCoefficients`: Service that stores and provides SOS coefficient sets.
- `BRTFilters::CSOSFilter`: IIR filter implementation that processes the SOS chain for each ear.
- `Common::CAudioMixer`: (base class) Used to mix multiple incoming audio streams per ear before filtering.

Configuration Options

The model allows confcan be enabled and disabled dynamically:

- **Enable/Disable Model**: If disabled, audio samples pass through unchanged.
- **Gain (float)**: Extra gain to be applied to the model output
- **SOS Filter Coefficients to be used**: The SOS coefficients service module to be used for rendering. The system supports dynamic, hot-swapping of the service module being used.

Connections

Modules to which it supports connections:

- Listener models

Modules to which it connects:

- Listener

For C++ developers

C++ developer

- **File:** /include/BilateralFilterModels/SOSBilateralFilterModel.hpp
- **Class name:** CSOSBilateralFilterModel
- **Inheritance:** CBilateralFilterModelBase
- **Namespace:** BRTBilateralFilter
- **Classes that instance:**
- BRTFilters::CSOSFilter

Class inheritance diagram

SOS Bilateral Filter Model Internal diagram

SOS Bilateral Filter Model Internal diagram.

How to instantiate

```
brtManager.BeginSetup();
std::shared_ptr<BRTBilateralFilter::CSOSBilateralFilterModel>
binauralFilter =
brtManager.CreateBinauralFilter<BRTBilateralFilter::CSOSBilateralFilterMo
brtManager.EndSetup();
if (binauralFilter == nullptr) {
    // ERROR
}
```

How to connect

Connect it to a listener.

```
std::shared_ptr<BRTBase::CListener> listener =
brtManager.GetListener(_listenerID);
if (listener != nullptr) {
    brtManager.BeginSetup();
    bool control = listener->ConnectBinauralFilter(_binauralFilterID);
    brtManager.EndSetup();
}
```

Connect an listener model to it.

```
std::shared_ptr<BRTBilateralFilter::CBilateralFilterModelBase>
binauralFilter =
brtManager.GetBinauralFilter<BRTBilateralFilter::CBilateralFilterModelBas
if (binauralFilter != nullptr) {
    brtManager.BeginSetup();
    bool control = binauralFilter->
ConnectListenerModel(_listenerModelID);
    brtManager.EndSetup();
}
```

Public methods

```
void EnableModel()
void DisableModel()
bool IsModelEnabled()

void SetGain(float _gain)
float GetGain()

bool
SetSOSFilterCoefficients(std::shared_ptr<BRTServices::CSOSCoefficients>
_listenerILD)
std::shared_ptr<BRTServices::CSOSCoefficients> GetSOSFilter() const
void RemoveSOSFilter()

bool ConnectListenerModel(const std::string & _listenerModelID,
Common::T_ear _ear = Common::T_ear::BOTH)
bool
ConnectListenerModel(std::shared_ptr<BRTListenerModel::CListenerModelBase
```

```

_listenerModel, Common::T_ear_ear = Common::T_ear::BOTH)

bool DisconnectListenerModel(const std::string & _listenerModelID,
Common::T_ear_ear = Common::T_ear::BOTH)
bool
DisconnectListenerModel(std::shared_ptr<BRTLListenerModel::CListenerModelB
_listenerModel, Common::T_ear_ear = Common::T_ear::BOTH)

T_BilateralFilterType GetBilateralFilterType() const

bool IsConnectedToListener()
bool IsConnectedToListenerModel()
void ConnectionToListenerEstablished(const std::string & _listenerID)

std::vector<std::string> GetInputs() { return inputConnections; }
std::vector<std::string> GetOutputs() { return outputConnections; }

```

2.5.3 Bilateral FIR Filter Model

Available from BRT v2.5.0

Last update: 9 Jan 2026

Overview

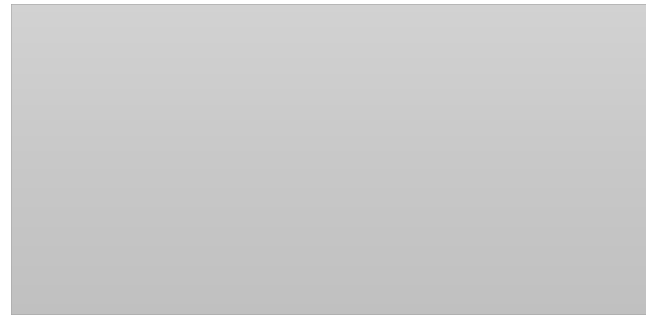
The **Bilateral FIR Filter Model** (class `BRTBilateralFilter::CFIRBilateralFilterModel`, defined in `FIRBilateralFilterModel.hpp`) implements **binaural filtering through frequency-domain convolution** using **finite impulse responses (FIR)**. Each ear is processed independently by convolving the input signal with a fixed impulse response defined in the frequency domain.

This approach enables applications such as **headphone compensation, device equalization**, or other forms of binaural transfer function processing.

Impulse responses are provided by the dedicated service module `GeneralFIR`, typically loaded from **SOFA** files.

Architecture

The internal block diagram of this class is as follows:



Bilateral SOS Filter Model Internal Block diagram.

```
### Inheritance
- `CFIRBilateralFilterModel` derives from `CBilateralFilterModelBase`,
  which itself inherits from `BRTBase::CModelBase`.
- The model is integrated into the processing graph through the
  `BRTBase::CBRTManager`.

### Internal components

The main internal elements of the model are:

- `BRTServices::CGeneralFIR` : Service that stores and provides the
  impulse responses (IRs) in the frequency domain.
- `BRTFilters::CFIRFilter` (Frequency-domain FIR filter processor):
  Performs FFT-based convolution independently for the left and right
  ears.
- `Common::CAudioMixer` (base class): Mixes multiple incoming audio
  streams per ear before convolution is applied.

---

# Configuration Options
The model allows confcan be enabled and disabled dynamically:

- **Enable/Disable Model** : If disabled, audio samples pass through
  unchanged.
- **Gain (float)**: Extra gain to be applied to the model output
- **Impulse Responses to be used** : The General FIR service module to
  be used for rendering. The system supports dynamic, hot-swapping of the
  service module being used.

---

## Connections
```

```
Modules to which it supports connections:

- Listener models

Modules to which it connects:

- Listener

---

## For C++ developers

<details>
<summary>For C++ developer</summary>
<ul>
<li><strong>File</strong>: /include/BilateralFilterModels/
  FIRBilateralFilterModel.hpp</li>
<li><strong>Class name</strong>: CFIRBilateralFilterModel</li>
<li><strong>Inheritance</strong>: CBilateralFilterModelBase</li>
<li><strong>Namespace</strong>: BRTBilateralFilter</li>
<li><strong>Classes that instance</strong>:
  <ul>
    <li>BRTFilters::CFIRFilter</li>
  </ul>
</li>
</ul>
</li>
</ul>

<h2>Class inheritance diagram</h2>
<div style="border: 1px solid #000; padding: 10px; display: inline-block;">
  
  <p style="text-align: center;">FIR Bilateral Filter Model Internal
  diagram.</p>
</div>
<br>

<h2>How to instantiate</h2>

```cpp
brtManager.BeginSetup();
std::shared_ptr<BRTBilateralFilter::CFIRBilateralFilterModel>
binauralFilter =
brtManager.CreateBinauralFilter<BRTBilateralFilter::CFIRBilateralFilter
Model>(_binauralFilterID);
brtManager.EndSetup();
if (binauralFilter == nullptr) {
 // ERROR
}
```

### How to connect

Connect it to a listener.

```
std::shared_ptr<BRTBase::CListener> listener =
brtManager.GetListener(_listenerID);
if (listener != nullptr) {
 brtManager.BeginSetup();
 bool control = listener->ConnectBinauralFilter(_binauralFilterID);
 brtManager.EndSetup();
}
```

Connect an listener model to it.

```
std::shared_ptr<BRTBilateralFilter::CBilateralFilterModelBase>
binauralFilter =
brtManager.GetBinauralFilter<BRTBilateralFilter::CBilateralFilterModelBase>
if (binauralFilter != nullptr) {
 brtManager.BeginSetup();
 bool control = binauralFilter->
 >ConnectListenerModel(_listenerModelID);
 brtManager.EndSetup();
}
```

### Public methods

```
void EnableModel()
void DisableModel()
bool IsModelEnabled()

void SetGain(float _gain)
float GetGain()

bool SetFIRTable(std::shared_ptr<BRTServices::CGeneralFIR> _firTable)
```

```

std::shared_ptr<BRTServices::CGeneralFIR> GetFIRTable() const
void RemoveFIRTable()

bool ConnectListenerModel(const std::string & _listenerModelID,
Common::T_ear _ear = Common::T_ear::BOTH)
bool
ConnectListenerModel(std::shared_ptr<BRTListenerModel::CListenerModelBase>
_listenerModel, Common::T_ear _ear = Common::T_ear::BOTH)

bool DisconnectListenerModel(const std::string & _listenerModelID,
Common::T_ear _ear = Common::T_ear::BOTH)
bool

```

```

DisconnectListenerModel(std::shared_ptr<BRTListenerModel::CListenerModelBase>
_listenerModel, Common::T_ear _ear = Common::T_ear::BOTH)

T_BilateralFilterType GetBilateralFilterType() const

bool IsConnectedToListener()
bool IsConnectedToListenerModel()
void ConnectionToListenerEstablished(const std::string & _listenerID)

std::vector<std::string> GetInputs() { return inputConnections; }
std::vector<std::string> GetOutputs() { return outputConnections; }

```

## 2.6 Services Modules

### 2.6.1 Service Modules

Service modules are auxiliary components responsible for managing and providing the resources required by the **Processing Models** to execute **binaural audio rendering algorithms** efficiently.



#### About the service modules

We have made a major change to the service modules starting with version 3.0.0, in order to resolve limitations we had and also simplify the code. You can find the documentation for the previous service modules, present in the library up to version 2.5.0, at this [link](#).

These resources include several key elements used during spatial audio processing:

- **Head-Related Transfer Functions (HRTFs)**, which simulate the direct sound path perceived by the listener model.
- **Binaural Room Impulse Responses (BRIRs)**, used to reproduce the reverberation characteristics of acoustic environments.
- **Source directivity data**, which represent the directional radiation patterns of sound sources.
- **Second-order filter section (SOS) coefficients**, which allow accurate modeling of, for example, near-field effects.
- **FIR filter coefficients**, which can be used for headphone equalization or hearing-protection simulations.
- **Impulse responses of virtual loudspeakers in the Ambisonics domain**, enabling simultaneous convolution and ambisonic decoding.
- **Geometric and acoustic descriptions of rooms**, used to simulate acoustic environments.



#### Spatial organization

Most resources managed by the service modules are **spatially indexed by azimuth, elevation, and distance**.

Non-spatial resources are also supported, such as FIR coefficients for headphone compensation filters.

Together, these resources provide the foundation required to generate **realistic and immersive binaural audio experiences**.

#### Architecture Overview

The interaction between the main architectural components of the BRT framework is illustrated below.

flowchart TD

```
F[Models]
A[Processing Modules]
B[(Service Modules)]
C[Readers]
D[SOFA Files]
E[OBJ Files]

F -->|use different processing modules| A
A -->|request resources during rendering| B
B -->|get resources during rendering| A
C -->|pre-load data| B
D --> C
E --> C
```

- **Processing Models** implement the binaural rendering algorithms.
- **Service Modules** store and organize the spatial audio resources used by those algorithms.
- **Readers** load the resources from external data formats.



#### Supported formats

The BRT library includes readers capable of loading spatial audio resources from **SOFA** and **OBJ** formats. For more details see the **READERS** section.

### Available Service Modules

The following service modules are available for storing and managing spatial audio resources.

Module	Description
<b>SphericalFIRTable</b>	Stores FIR-based spatial data such as <b>HRTFs, BRIRs, source directivity information, and general FIR filter coefficients.</b>
<b>SphericalInterpolatedFIRTable</b>	Stores <b>HRTF and source directivity data</b> and generates a <b>regular spherical grid</b> through interpolation, providing a predictable spatial structure.
<b>SphericalSOSTable</b>	Stores <b>second-order filter section (SOS) coefficients</b> , which may be constant or vary depending on <b>distance, azimuth, and/or elevation.</b>
<b>AmbisonicBRIR</b>	Stores the <b>impulse responses of virtual loudspeakers in the ambisonic domain</b> , enabling a pipeline where <b>impulse response convolution and ambisonic decoding occur simultaneously.</b>
<b>Room</b>	Stores the <b>geometric and acoustic description of a room</b> , including its <b>vertices, walls, and frequency-band absorption</b>

Module	Description
	<b>coefficients</b> for each surface. An addition to the MTL format has been defined in order to store this acoustic information.

## 2.6.2 Spherical FIR Table

### Overview

*SphericalFIRTable* is a **Service Module** that stores collections of **Finite Impulse Response (FIR) filters organized in spherical coordinates**. It is designed to manage spatially indexed filter datasets such as HRTFs, BRIRs, and directional source responses.

Each filter entry is associated with a **direction and distance relative to the listener**, defined by **azimuth, elevation, and distance**. The module provides a structured container that enables Processing Models to efficiently retrieve spatial impulse responses during binaural rendering.

In addition to storing FIR data, the module supports **temporal windowing of impulse responses**, allowing the beginning or end of the IR to be modified. This capability is particularly useful for adapting datasets to different rendering strategies.

Within the BRT ecosystem, *SphericalFIRTable* acts as a **central repository for spatial FIR datasets shared across rendering algorithms**.

### Role in the Architecture

In the BRT architecture, *SphericalFIRTable* serves as a **data provider for Processing Models**. Rendering algorithms query the table to retrieve FIR filters corresponding to the current spatial configuration of sources and listeners. The module itself does not read external files. Instead, **Readers** import datasets from formats such as **SOFA or other measurement formats** and populate the table structure.

This separation ensures that **data loading, storage, and signal processing remain independent components**, improving modularity and extensibility.

### Data Organization

The data structure is organized hierarchically to support **multiple measurement contexts and spatial resolutions**.

At the highest level, FIR datasets are grouped by a **reference position in 3D space**. Each reference position typically corresponds to a **measurement location of the sound source or listener**, which is particularly useful for BRIR datasets where impulse responses are measured at different positions in a room.

Within each reference position, the data is further divided into **distance buckets**, each representing a **sphere of**

**measurements at a specific radial distance**. This structure allows datasets such as HRTFs measured at multiple radii to coexist in the same table. During rendering, the system selects the **distance sphere closest to the requested source distance**.

Inside each distance bucket, FIR filters are organized by **azimuth and elevation**. A **KD-tree spatial index** is used to perform efficient nearest-neighbour searches, allowing the system to retrieve the **closest available measurement direction** when an exact match is not present.

The stored impulse responses can also be **temporally windowed**. Samples at the beginning of the IR may be replaced with zeros to delay the response, while the tail of the impulse response can be **truncated** to reduce its duration. This allows the dataset to be adapted to different rendering requirements without modifying the original measurements.

This hierarchical structure enables flexible storage of datasets with **arbitrary spatial sampling distributions**, without requiring a regular directional grid.

### Internal FIR Representation

Before being stored in the module, the HRIRs are transformed into a representation optimized for real-time convolution. Each HRIR is partitioned into fragments matching the input buffer size in order to support **Uniformly Partitioned Overlap-Save (UPOLS) convolution**. An FFT is then applied to every partition and the resulting spectra are stored in memory by the Service Module. As a result, the binaural rendering stage performs convolution directly in the **frequency domain**, significantly improving computational efficiency.

#### DATA HIERARCHY

```
graph LR
 A[Reference Position] --> B[Distance Sphere]
 B --> C[Direction: Azimuth / Elevation]
 C --> D[Partitioned FR of the left ear]
 C --> E[Partitioned FR of the right ear]
```

### Supported Data Types

*SphericalFIRTable* can store several types of spatial FIR datasets, including:

- **HRTFs (Head-Related Transfer Functions)** — impulse responses describing the filtering effect of the listener's anatomy for different directions.
- **BRIRs (Binaural Room Impulse Responses)** — spatial impulse responses that include head-related filtering and room acoustics.
- **Source Directivity Data** — directional radiation patterns of sound sources represented as FIR filters.
- **Generic Spatial FIR Filters** — any FIR dataset indexed or not by spatial direction.

All datasets are represented as impulse responses associated with spherical coordinates and optionally multiple distances or reference positions.

### Typical Use Cases

A common use case is **binaural rendering of virtual sound sources**, where the Processing Model queries the table using the source direction relative to the listener. The system retrieves the closest available FIR measurement and applies it during convolution.

For **multi-distance HRTF datasets**, the renderer selects the sphere whose radius best matches the current source distance. This enables more accurate rendering when datasets include near-field measurements.

For **room acoustics datasets such as BRIRs**, the system selects the reference position closest to the current source position in the environment. In these cases, the impulse responses can also be **windowed or truncated** to remove unwanted portions of the response, enabling efficient implementation of **hybrid rendering approaches** that combine early reflections and late reverberation models.

### Related Service Modules

#### SphericalInterpolatedFIRTable

*Spherical Interpolated FIR Table* extends the functionality of *SphericalFIRTable* by performing interpolation between measurement directions. This allows Processing Models to obtain FIR filters for arbitrary directions even when the dataset is sparsely sampled. In contrast, *SphericalFIRTable* performs nearest-neighbour retrieval and always returns the closest available measurement.

#### SphericalSOSTable

SphericalSOSTable stores spatially organized IIR filters represented as second-order sections (SOS) instead of FIR

impulse responses. While both modules use spherical indexing, *SphericalFIRTable* is intended for impulse-response-based datasets, whereas *SphericalSOSTable* stores parametric filter representations.

### Configuration Options

This service allows you to configure the following parameters:

- **Enable/Disable Woodworth ITD:** Toggles the application of the Woodworth ITD formula for HRTF processing.
- **Set/Get Head Radius:** Configures or retrieves the radius of the listener's head model.
- **Set parameters for the windowing IR process:** Defined the windowing parameters.

### Summary

*SphericalFIRTable* is a core Service Module that stores spatially organized FIR filter datasets used by binaural rendering algorithms.

Its hierarchical structure supports multiple reference positions, multiple measurement distances, and arbitrary directional sampling. Efficient spatial queries are implemented through nearest-neighbour searches using KD-tree structures.

This design allows BRT to manage complex spatial datasets such as multi-distance HRTFs and room-based BRIR measurements in a flexible and scalable way.

## For C++ developer

## C++ developer

- **File:** /include/ServiceModules/SphericalFIRTable.hpp
- **Class name:** CSphericalFIRTable
- **Inheritance:** CServicesBase
- **Namespace:** BRTServices

## Class inheritance diagram

```
classDiagram
 direction TB
 class CServicesBase
 <<interface>> CServicesBase
 class CSphericalFIRTable
 class CSphericalInterpolatedFIRTable
 class CSphericalSOSTable
 class CAmbisonicBIR
 CServicesBase <|-- CSphericalFIRTable
 CServicesBase <|-- CSphericalInterpolatedFIRTable
 CServicesBase <|-- CSphericalSOSTable
 CServicesBase <|-- CAmbisonicBIR
```

## How to instantiate and load

```
// Assuming SOFA_FILEPATH contains the SOFA filename including the
path
std::shared_ptr<BRTServices::CSphericalFIRTable> hrtf =
std::make_shared<BRTServices::CSphericalFIRTable>();
bool hrtfSofaLoaded = LoadSofaFile(SOFA_FILEPATH, hrtf);
if (!hrtfSofaLoaded) {
 // ERROR
}
```

## How to connect it to a listener

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the HRTF is already successfully loaded into hrtf.
std::shared_ptr<BRTBase::CListener> listener = brtManager->
GetListener(listenerID);
listener->SetHRTF(hrtf);
```

Public Methods of `CSphericalFIRTable`

Category	Method	Description
Constructor	<code>CSphericalFIRTable()</code>	Constructs a spherical FIR table.
ITD Customization	<code>EnableWoodworthITD()</code>	Enables Woodworth ITD processing.
	<code>DisableWoodworthITD()</code>	Disables Woodworth ITD processing.
	<code>IsWoodworthITDEnabled() const</code>	Returns whether Woodworth ITD processing is enabled.
FIR Partition Info	<code>GetNumberOfSubfiltersFR() const</code>	Returns the number of frequency subfilters.
	<code>GetSubfilterLengthFR() const</code>	Returns the length of a frequency subfilter.
Cranial Geometry	<code>SetHeadRadius(float)</code>	Sets the head radius.
	<code>GetHeadRadius() const</code>	Returns the head radius.
	<code>RestoreHeadRadius()</code>	Restores the head radius to its default value.
	<code>SetEarPosition(T_ear, CVector3)</code>	Sets the ear position.
	<code>GetEarLocalPosition(T_ear) const</code>	Returns the local position of the ear.
	<code>SetCranialGeometryAsDefault()</code>	Restores the cranial geometry to its default values.

Category	Method	Description	Category	Method	Description
Measurement Metadata	<code>GetDistanceOfMeasurement(...)</code>	Returns distance associated with closest measurement for the spatial parameters.		<code>GetFR_2Ears()</code>	Returns current parameters with closest measurement for the spatial parameters.
			Delay Retrieval	<code>GetFR_Delay(...)</code>	Returns the impulse response delay for the current parameters.
IR Windowing	<code>SetWindowingParameters(...)</code>	Configures the windowing parameters for the impulse response.	Metadata	<code>GetReferencePositions() const</code>	Returns the reference positions for the current parameters.
	<code>GetWindowingParameters(...)</code>	Returns the current windowing parameters.			
FIR Table Setup	<code>BeginSetup(...)</code>	Initializes the table configuration before inserting impulse responses.			
	<code>AddIR(...)</code>	Adds a new impulse response measurement to the table.			
	<code>EndSetup()</code>	Finalizes the table structure after all impulse responses have been added.			
FIR Retrieval	<code>GetFR_SpatiallyOriented(...)</code>	Returns the frequency-domain FIR partitions for one ear at a given spatial direction.			
	<code>GetFR_SpatiallyOriented_2Ears(...)</code>	Returns the frequency-domain FIR partitions for both ears at a given spatial direction.			

## 2.6.3 Spherical Interpolated FIRT able

### Overview

*SphericalInterpolatedFIRTable* is a Service Module designed to store and manage **finite impulse response (FIR) data organized in spherical coordinates**. It extends the concept of spatial FIR storage by generating a **regular spherical sampling grid through interpolation**. This regularized representation provides predictable spatial indexing and facilitates efficient access during rendering. The module is particularly suited for datasets such as HRTFs or source directivity measurements that originate from irregular measurement distributions.

### Role in the Architecture

Within the BRT architecture, Service Modules provide structured access to resources used by rendering algorithms. *SphericalInterpolatedFIRTable* stores spatial FIR datasets that are typically loaded by **Readers** from external formats such as SOFA files. **Processing Models** then query this module during runtime to retrieve impulse responses corresponding to specific spatial directions. This separation allows algorithms to remain independent from file formats and raw dataset organization.

### Data Organization

The data is organized as a collection of **distance-dependent spherical tables**. Each distance defines a *distance bucket*, representing a measurement sphere centered on the listener. Inside each bucket, impulse responses are indexed by **azimuth and elevation**, forming a regular directional grid. This structure allows the system to manage datasets that contain measurements at multiple listener-source distances.

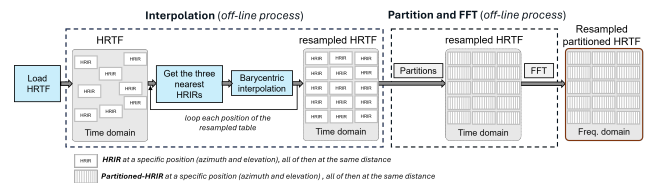
For HRTF datasets, each distance bucket corresponds to a **measurement sphere around the listener**. During rendering, the system typically selects the **closest available sphere** to the requested source distance. This approach preserves the physical meaning of distance-dependent measurements while keeping runtime access efficient.

Within each sphere, the directional grid is generated through **barycentric interpolation of the original measurements**. This produces a consistent spatial sampling even when the source dataset is irregular. At runtime, Processing Models can either retrieve the **nearest grid direction** or perform a **fast barycentric interpolation** between neighboring directions for smoother spatial transitions.

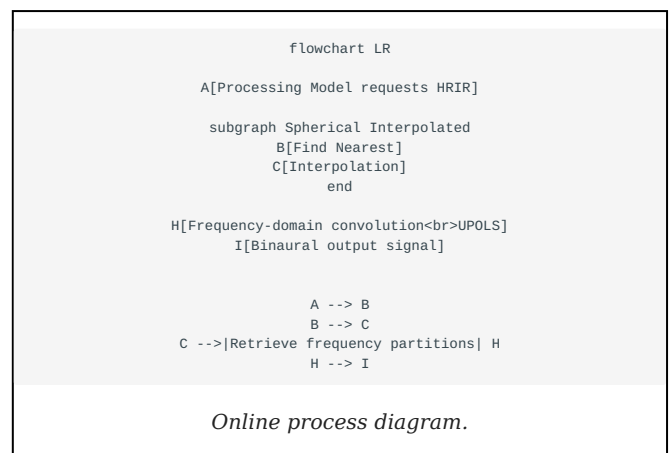
### Internal FIR Representation

Before being stored in the module, the HRIRs are transformed into a representation optimized for real-time convolution. Each HRIR is partitioned into fragments matching the input buffer size in order to support **Uniformly Partitioned Overlap-Save (UPOLS) convolution**. An FFT is then applied to every partition and the resulting spectra are stored in memory by the Service Module. As a result, the binaural rendering stage performs convolution directly in the **frequency domain**, significantly improving computational efficiency.

The following diagram summarizes the **preprocessing and runtime stages** involved in the internal representation of HRIR data.



Offline interpolation process diagram.



### DATA STORE HIERARCHY

```

graph LR
 B[Distance Sphere]
 B --> C[Direction: Azimuth / Elevation]
 C --> D[Partitioned FR of the left ear]
 C --> E[Partitioned FR of the right ear]

```

### Interpolation Strategy

For applications requiring **smooth and continuous 3DoF binaural rendering**, BRT provides interpolation capabilities within this Service Module. This is particularly useful in dynamic virtual environments when the available HRTF measurements do not form a regular or dense directional grid. Many datasets contain sparse measurements or large regions without data, which can lead to discontinuities when rendering directly from the measured points.

To address this limitation, *SphericalInterpolatedFIRTable* estimates HRIRs for the **exact direction and distance of the sound source**. During an **offline preprocessing stage**, the algorithm identifies the three closest measured directions surrounding each target grid point. A **barycentric interpolation** is then performed between the corresponding HRIRs to estimate the impulse response at that location.

The result of this preprocessing step is a **regular spherical grid of FIR responses**, which is stored internally by the module. The density of this grid is controlled by a configurable parameter called spatial resolution<sup>1</sup>, being the intended angular separation between two adjacent points at the equator or two adjacent points in the median plane (azimuth=0). Lower spatial resolutions produce denser grids and more accurate spatial reconstruction, at the cost of increased memory usage and preprocessing time. This regularization allows Processing Models to perform efficient spatial lookups during rendering. Additional real-time interpolation between neighboring grid points can also be applied when smoother directional transitions are required. You can find more details on how this grid is generated in this [document](#).

Interpolating HRIRs with different **interaural time differences (ITDs)** can introduce audible artifacts and degrade rendering quality. To avoid this issue, the HRTF Service Module handles **ITDs independently from the FIR interpolation and convolution processes**. This separation prevents temporal inconsistencies when combining impulse responses from different measurement directions. For this reason, user-imported HRIR datasets should provide **ITD information stored separately from the impulse responses**.

After the spatial interpolation step, the appropriate ITD can be applied to the rendered signal. These ITDs may be estimated by interpolating the ITDs associated with the three nearest HRIR measurements, or in another extra feature, they could be synthesized from geometric information such as **interaural azimuth and listener head circumference**.

#### Supported Data Types

The module can store several types of spatial FIR datasets used in binaural rendering:

- **HRTFs (Head-Related Transfer Functions)** – impulse responses describing how sound from a spatial direction is filtered by the listener's anatomy.
- **Source directivity datasets** – directional impulse responses describing how a sound source radiates energy in space.

These datasets share the common property of representing **direction-dependent acoustic filtering**.

#### Typical Use Cases

A typical use case is **binaural rendering with interpolated HRTFs**, where the rendering engine queries impulse responses for arbitrary source directions. The regular grid generated by this module simplifies spatial interpolation and improves runtime predictability. Another use case is modeling **directional sound sources**, where the source radiation pattern is represented as FIR responses over a spherical domain. The module can also support research workflows where measured datasets must be resampled to a consistent spatial resolution.

#### Related Service Modules

##### SphericalFIRTable

Stores FIR responses indexed by spherical coordinates but **preserves the original measurement distribution**. It does not generate a regular grid and therefore relies on the spatial structure provided by the dataset.

##### SphericalSOSTable

Stores spatial filters represented as **second-order section (SOS) filter banks** instead of FIR impulse responses. This representation is typically used when parametric or IIR filter models are preferred over impulse responses.

#### Configuration Options

This service allows you to configure the following parameters:

- **Set Grid Spatial Resolution:** Set the spatial resolution that allows the grid to be defined.
- **Enable/Disable Woodworth ITD:** Toggles the application of the Woodworth ITD formula for HRTF processing.
- **Set/Get Head Radius:** Configures or retrieves the radius of the listener's head model.
- **Set parameters for the windowing IR process:** Defined the windowing parameters.

#### Summary

*SphericalInterpolatedFIRTable* provides a structured container for **spatial FIR datasets resampled onto a regular spherical grid**. By separating spatial resource management from rendering algorithms, it supports modular and flexible binaural processing pipelines. The module enables predictable spatial lookup and simplifies interpolation during runtime. Within the BRT architecture,

it acts as a bridge between **dataset Readers and binaural Processing Models**, supporting reproducible research in spatial audio rendering.

**For C++ developer**

C++ developer

- **File:** /include/ServiceModules/SphericalInterpolatedFIRTable.hpp
- **Class name:** CSphericalInterpolatedFIRTable
- **Inheritance:** CServicesBase
- **Namespace:** BRTServices

Class inheritance diagram

```
classDiagram
direction TB

class CServicesBase
<<interface>> CServicesBase
class CSphericalFIRTable
class CSphericalInterpolatedFIRTable
class CSphericalSOSTable
class CSphericalSOSSTable
class CAmibisonicBIR

CServicesBase <|-- CSphericalFIRTable
CServicesBase <|-- CSphericalInterpolatedFIRTable
CServicesBase <|-- CSphericalSOSTable
CServicesBase <|-- CAmibisonicBIR
```

How to instantiate and load

```
// Assuming SOFA_FILEPATH contains the SOFA filename including the
path
std::shared_ptr<BRTServices::CSphericalInterpolatedFIRTable> hrtf =
std::make_shared<BRTServices::CSphericalInterpolatedFIRTable>();
bool hrtfSofaLoaded = LoadSofaFile(SOFA_FILEPATH, hrtf);
if (!hrtfSofaLoaded) {
 // ERROR
}
```

How to connect it to a listener

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the HRTF is already lsuccessfully loaded into hrtf.
std::shared_ptr<BRTBase::CListener> listener = brtManager->
GetListener(listenerID);
listener->SetHRTF(hrtf);
```

Public Methods of CSphericalInterpolatedFIRTable

Category	Method	Description	Category	Method	Description
Constructor	<code>CSphericalInterpolatedFIRTable()</code>	Create spherical interpolated FIR table	Measurement Metadata	<code>double</code> <code>GetDistanceOfMeasurement(const Common::CTransform &amp; _referenceLocation, const double &amp; _azimuth, const double &amp; _elevation, const double &amp; _distance) const override</code>	Return distance of measurement
ITD Customization	<code>void EnableWoodworthITD() override</code>	Enable Woodworth ITD interpolation	IR Windowing	<code>void SetWindowingParameters(float _fadeInBegin, float _riseTime, float _fadeOutCutoff, float _fallTime) override</code>	Configure IR windowing parameters
	<code>void DisableWoodworthITD() override</code>	Disable Woodworth ITD interpolation		<code>void GetWindowingParameters(float &amp; _fadeInWindowThreshold, float &amp; _fadeInWindowRiseTime, float &amp; _fadeOutWindowThreshold, float &amp; _fadeOutWindowRiseTime) const override</code>	Retrieve IR windowing parameters
	<code>bool IsWoodworthITDEnabled() const override</code>	Return whether Woodworth ITD interpolation is currently enabled	Interpolation Grid	<code>void SetGridSamplingStep(int _samplingStep) override</code>	Set grid sampling step
FIR Partition Info	<code>const int32_t</code> <code>GetNumberOfSubfiltersFR() const override</code>	Return number of frequency domain parameters		<code>int GetGridSamplingStep() const override</code>	Return grid sampling step
	<code>const int32_t</code> <code>GetSubfilterLengthFR()</code> <code>const override</code>	Return length of parameters subfilter	FIR Table Setup	<code>bool BeginSetup(const int32_t &amp; _HRIRLength, const BRTServices::TEXTRAPOLATION_METHOD &amp; _extrapolationMethod) override</code>	Begin FIR table setup
Cranial Geometry	<code>void SetHeadRadius(float _headRadius) override</code>	Set head radius		<code>void AddIR(const Common::CVector3 &amp; _referencePosition, const double &amp; _azimuth, const double &amp; _elevation, const double &amp; _distance, TIRStruct &amp;&amp; _newIR) override</code>	Add IR to table
	<code>float GetHeadRadius() const override</code>	Return current head radius		<code>bool EndSetup() override</code>	End FIR table setup
	<code>void RestoreHeadRadius() override</code>	Reset head radius to default			
	<code>void SetEarPosition(Common::T_ear _ear, Common::CVector3 _earPosition) override</code>	Set ear position			
	<code>Common::CVector3</code> <code>GetEarLocalPosition(Common::T_ear _ear) const override</code>	Return local position of the ear			
	<code>void SetCranialGeometryAsDefault() override</code>	Reset cranial geometry			

Category	Method	Description
FIR Retrieval	<pre>const TFRPartitions GetFR_SpatiallyOriented(const float &amp; _azimuth, const float &amp; _elevation, const float &amp; _distance, const Common::CTransform &amp; _referenceLocation, const Common::T_ear &amp; ear, bool _runTimeInterpolation) const override</pre>	1. Returns the frequency-domain FIR partitions for one ear at a given spatial direction (with optional runtime interpolation).  ons that are irrelevant here, it is also <i>SamplingStep</i> in some older parts of the C++ its. ←
	<pre>const Common::CEarPair&lt;TFRPartitions&gt; GetFR_SpatiallyOriented_2Ears(const float &amp; _azimuth, const float &amp; _elevation, const float &amp; _distance, const Common::CTransform &amp; _referenceLocation, bool _runTimeInterpolation) const override</pre>	Returns the frequency-domain FIR partitions for both ears (with optional runtime interpolation).
Delay Retrieval	<pre>const Common::CEarPair&lt;uint64_t&gt; GetFR_Delay(const float &amp; _azimuthCenter, const float &amp; _elevationCenter, const float &amp; _distance, const Common::CTransform &amp; _referenceLocation, bool _runTimeInterpolation) const override</pre>	Returns the interaural delays associated with the selected FIR responses (with optional runtime interpolation).

## 2.6.4 Spherical SOS Table

### Overview

*SphericalSOSTable* is a Service Module designed to store **second-order section (SOS) filter coefficients** within the BRT architecture. These coefficients represent parametric filter structures that may either remain constant or vary spatially. The module is particularly useful for representing direction-dependent filters such as **near-field compensation filters** or other parametric acoustic corrections. By organizing SOS filters in a spatial structure, the module enables efficient retrieval of the filter corresponding to a given listener-source direction.

### Role in the Architecture

Within the BRT architecture, *SphericalSOSTable* acts as a **resource container** accessed by Processing Models during runtime. Processing Models query the module to obtain the appropriate SOS filter coefficients based on the current spatial configuration. The data stored in the module is typically loaded by **Readers**, which parse external formats and convert them into the internal spatial organization used by BRT. This separation allows resource management to remain independent from the signal processing algorithms.

### Data Organization

The data is organized as a **collection of spherical tables, one for each distance**. Each distance defines a *distance bucket*, representing a measurement sphere centered at the listener. Within each bucket, filters are indexed by **azimuth and elevation**, forming a regular directional grid. This structure allows the module to manage datasets containing measurements taken at multiple source-listener distances. During rendering, the system selects the **closest available sphere** to the requested source distance, preserving the physical meaning of distance-dependent measurements while maintaining efficient runtime access.

Inside each distance bucket, the IIR filters are organized according to their **azimuth and elevation coordinates**. A **KD-tree spatial structure** is used to perform efficient nearest-neighbor searches within the table. This allows the system to retrieve the closest available measurement direction whenever an exact directional match is not present.

Although this is the primary design, the structure is flexible enough to support other usage patterns. The module can also store **direction-independent filters** by inserting coefficients with azimuth, elevation, and distance set to zero. In this case, a single filter per ear can be stored and retrieved without any spatial dependency.

Another example of this flexibility can be seen in our implementation of the **near-field effect filters**. In this instance, the coefficients are indexed by interaural azimuth, with elevation fixed at zero. This approach exploits the symmetry of the acoustic response with respect to the interaural axis. Further details about this configuration can be found in the *Near-Field* section.

### DATA STORE HIERARCHY

```
graph LR
 B[Distance Sphere]
 B --> C[Direction: Azimuth / Elevation]
 C --> D[IIR coefficients of the left ear]
 C --> E[IIR coefficients of the right ear]
```

### Supported Data Types

The module stores **parametric filter representations** expressed as cascades of second-order sections. Each entry typically contains the numerator and denominator coefficients of the SOS structure. This representation is commonly used for efficient and numerically stable implementations of IIR filters. Both **single filters** and **spatially varying collections of filters** can be represented within the same framework.

### Typical Use Cases

A couple of common use cases might be: storing **near-field compensation filters** that vary depending on the distance between the sound source and the listener. The processing models responsible for near-field rendering can query the module to obtain the appropriate SOS filter for the current source direction. Another use of this module is to store filters for simulating direction-independent filters, such as protective headphones or a very simple equalisation device.

### Related Service Modules

#### SphericalFIRTable

*SphericalFIRTable* stores **impulse responses (FIR filters)** organized spatially, typically representing HRTFs or BRIRs. In contrast, *SphericalSOSTable* stores **parametric IIR filters expressed as SOS coefficients**, which are generally more compact and suitable for certain acoustic corrections.

#### SphericalInterpolatedFIRTable

*SphericalInterpolatedFIRTable* performs interpolation between spatial measurements, *SphericalSOSTable* typically retrieves the filter associated with the closest available spatial position. These modules therefore address different filter representations and rendering needs.

**Summary**

*SphericalSOSTable* provides a structured way to store and access **spatially organized parametric filters** within BRT. By representing filters as SOS coefficients and organizing them over spherical coordinates, the module supports efficient retrieval of direction-dependent filtering resources. It complements FIR-based resource modules and enables Processing Models to integrate parametric acoustic corrections into binaural rendering pipelines.

**For C++ developer**

C++ developer

- **File:** /include/ServiceModules/SphericalSOSTable.hpp
- **Class name:** CSphericalSOSTable
- **Inheritance:** CServicesBase
- **Namespace:** BRTServices

Class inheritance diagram

```
classDiagram
direction TB

class CServicesBase
<<interface>> CServicesBase
class CSphericalFIRTable
class CSphericalInterpolatedFIRTable
class CSphericalSOSTable
class CAmbisonicBIR

CServicesBase <|-- CSphericalFIRTable
CServicesBase <|-- CSphericalInterpolatedFIRTable
CServicesBase <|-- CSphericalSOSTable
CServicesBase <|-- CAmbisonicBIR
```

How to instantiate and load

```
// Assuming SOFA_FILEPATH contains the SOFA filename including the
path
std::shared_ptr<BRTServices::CSphericalSOSTable> sosFilter =
std::make_shared<BRTServices::CSphericalSOSTable>();
bool sofaLoaded = LoadSofaFile(SOFA_FILEPATH, sosFilter);
if (!sofaLoaded) {
 // ERROR
}
```

How to connect it to a listener model

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the HRTF is already lsuccessfully loaded into hrtf.
std::shared_ptr<BRTBase::CListener> listener = brtManager-
>GetListener(listenerID);
listener->SetNearFieldCompensationFilters(hrtf);
```

How to connect it to a bilateral filter by the listener ID

```
std::shared_ptr<BRTBase::CListener> listener = brtManager-
>GetListener(listenerID);
listener->SetBilateralSOSFilter(filter);
```

How to connect it directly to a bilateral filter

```
std::shared_ptr<BBRTBilateralFilter::CBilateralFilterModelBase>
filter = brtManager->GetBilateralFilter(bilateralFilterID);
filter->SetSOSFilterCoefficients(filter);
```

Public Methods of CSphericalSOSTable

Category	Method
Constructor	CSphericalSOSTable()
Ear Geometry	void SetEarPosition(Common::T_ear _ear, Common::CVector3 _earPosition)
SOS Table Setup	bool BeginSetup()  void AddCoefficients(float _azimuth, float _elevation, float _distance, Common::CEarPair<CMonoBuffer<float>> && newCoefs)  bool EndSetup()
SOS Retrieval	const std::vector<float> GetSOSCoefficients_SpatiallyOriented(float _azimuth, float _elevation, float _distance, Common::T_ear ear) const  const Common::CEarPair<CMonoBuffer<float>> GetSOSCoefficients_2Ears() const

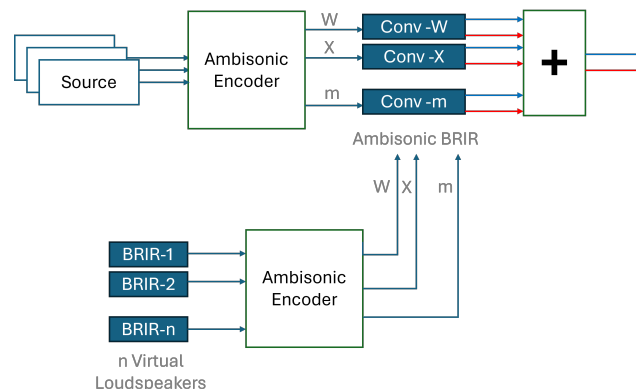
## 2.6.5 Ambisonic BRIR

The Ambisonic BRIR Convolution Module of the Binaural Rendering Toolbox enables the spatial audio rendering of multiple sound sources in an ambisonic environment. By leveraging the convolution with Binaural Room Impulse Responses (BRIR) in the ambisonic domain, this module processes the input sources considering their positions and the listener's position and orientation. It supports ambisonic encoding of orders 1, 2, or 3, and uses to decode virtual loudspeakers located at the vertices of regular polyhedra (octahedron for first-order, icosahedron for second-order, and dodecahedron for third-order).

In order to optimize the process, the module convolves the ambisonic signal directly with the ambisonic mix of the virtual loudspeakers' impulse responses, subsequently mixing all right and left channels separately, as it's shown in the diagram shown below. [Uniformly partitioned convolution](#) is used to compute the convolution in the frequency domain.

The diagram below illustrates the key components of the system and how they work together. From handling input parameters to generating and processing ambisonic signals, the system employs ambisonics and convolution techniques to accurately simulate 3D audio environments. The components and their respective functionalities are detailed below.

- **Input Handling:** The module accepts a set of audio sources along with their positions in space, as well as the position and orientation of the listener.
- **Ambisonic Encoding:** The relative positions of each source with respect to the listener are used to encode the sources into an ambisonic format of the specified order (1st, 2nd, or 3rd).
- **Virtual Loudspeaker:** Impulse responses are got from the BRIR file for those positions of the virtual loudspeakers, arranged at the vertices of a regular polyhedron corresponding to the ambisonic order.
- **Convolution:** Rather than convolving each virtual loudspeaker signal with the HRIR individually, the module convolves the ambisonic signal directly with the ambisonic mix of the BRIRs for the virtual loudspeakers.
- **Channel Mixing:** The convoluted signals are mixed into separate left and right channels, resulting in the final binaural audio output.



Ambisonic BRIR model diagram.

### Functional Overview

To properly load and configure a file using this class, the process must begin with the **Begin Setup** method, which initializes the system with a specified ambisonic order and normalization scheme. This step sets up the necessary parameters for subsequent operations. Once all required configurations and impulse responses have been added, the setup process is finalized using the **End Setup** method. This ensures that the class is ready for use, as indicated by the **IsReady** method, which verifies if the Ambisonic BRIR is loaded and operational. During the setup, additional methods can be used to enhance the configuration. For instance, **Add Impulse Response** allows adding a new impulse response associated with a specific channel and listener position. Finally, the system provides methods for retrieving configuration details, such as **Get Channel Partitioned IR**, which fetches the partitioned impulse response for a given channel and ear, taking into account the listener's current position and orientation.

Configuration Options

The methods provided by this service are as follows.

- **Begin Setup:** Initializes the setup process with the specified parameters.
- **End Setup:** Completes the setup process and confirms the system is ready.
- **Reset:** Resets the system to its initial state.
- **Is Ready:** Checks if the system is ready and the BRIR is loaded.
- **Add Impulse Response:** Adds a new impulse response associated with a specific channel and listener position.
- **Get Channel Partitioned IR:** Retrieves the partitioned impulse response for a channel and a specific ear, considering the listener's position and orientation.
- **Add Impulse Responses From HRIR:** Adds multiple impulse responses from a shared HRIR resource.

BRIR Format

A BRIR (Binaural Room Impulse Response) must be loaded. [Readers](#) can be used to load BRIR SOFA files. Depending on

the Ambisonic order, the direction of the virtual loudspeakers are searched in the BRIR file. If some of them is not found, an interpolation among the three closest directions is performed. If different listener positions are provided, a different set of virtual loudspeakers will be created for every listener position. When rendering, the one with the closest position to the listener is selected to convolve. To determine if two positions are the same, a resolution of 1 meter is used.

Depending on how the BRIR is captured, several cases may be considered

- One fixed listener with several source positions around it
- One listener position with different orientations and one fixed source
- Different listener positions with sources around

 C++ developer

Section under construction

## 2.6.6 Room Service Module

The Room Service Module provides the geometric and acoustic description of an enclosed space used by the BRT's simulation engines. It offers a unified representation of room boundaries, wall materials, and spatial relationships required by the **Image-Source Method (ISM)** and the **Scattering Delay Network (SDN)** rendering algorithms.

This module consists of two main classes:

- `CRoom` — container of all walls and high-level room operations.
- `CWall` — geometric and acoustic description of a single wall element.

Both classes form the core data structure used by the acoustic simulators to evaluate reflections, image sources, occlusion, and room-material interactions.

### 1. Purpose and Scope

The Room module describes **the physical enclosure** in which acoustic propagation is simulated. Its responsibilities include:

- Defining the **geometric boundaries** of a room using planar polygonal walls.
- Storing **per-wall acoustic absorption** data across 9 octave bands.
- Ensuring the **convexity** and **validity** of the room geometry.
- Providing a standardized interface used internally by:
- **ISM (Image-Source Method)** for reflection and image-source generation.
- **SDN (Scattering Delay Network)** for simulating late reverberation and diffusion.
- Computing geometric relationships such as:
- Point containment (whether a source or image lies inside the room).
- Image-room generation for reflective simulation steps.

This module is **not** responsible for rendering, DSP, or file loading. OBJ/MTL importing is described in a separate section (see [OBJ Reader](#) documentation).

## 2. Room Geometry Requirements

### 2.1 CONVEXITY REQUIREMENT (MANDATORY)

The BRT assumes that all rooms are **strictly convex**. This is a **strong requirement** of the underlying acoustics algorithms:

- Convexity ensures correct visibility and ordering of reflections in ISM.
- Image-room generation relies on consistent wall normals and enclosure shape.
- Non-convex geometry may lead to invalid or missing reflections.

Users must ensure that custom geometries — whether built manually or imported via OBJ — meet this condition.

### 2.2 WALL REPRESENTATION

Each wall is represented as a **planar, convex polygon** (typically a quadrilateral). Walls are defined by a list of 3D vertices stored in counter-clockwise order as seen from inside the room.

#### Automatically enforced planarity

If a vertex slightly deviates from the plane defined by the first three vertices, it is automatically projected onto the plane, guaranteeing that all wall polygons remain strictly planar.

#### Wall normals

Wall normals are computed automatically using the corner order. By convention and requirement:

- **The normal must point towards the interior of the room.**
- This direction determines the inward-facing reflection plane used by the ISM engine.

Users creating custom rooms must ensure the correct ordering of corners to preserve this convention.

## 3. CWall: Acoustic and Geometric Description of a Wall

A `CWall` models one reflective boundary of the room. It provides:

### 3.1 GEOMETRY

- Convex polygon defined by an ordered list of vertices.
- Automatically enforced planarity.
- Access to geometric properties such as normal vector, center, projection, intersections, and distances.

### 3.2 ACOUSTIC ABSORPTION

Each wall stores **nine absorption coefficients**, representing **absorbed energy over incident energy** at the following octave-band center frequencies: 62.5 Hz, 125 Hz, 250 Hz, 500 Hz, 1 kHz, 2 kHz, 4 kHz, 8 kHz, 16 kHz.

Even when setting a single frequency-independent absorption value, it is internally propagated to all 9 bands.

### 3.3 ACTIVE / INACTIVE WALLS

Walls can be **active** (reflective) or **inactive** (transparent). Typical uses include:

- Disabling floor or ceiling reflections for controlled experiments.
- Reducing the number of reflections and virtual sources in specific tests.
- Creating “partially open” spaces while maintaining convexity.

Inactive walls behave as if they do not exist during simulation.

## 4. CRoom: Full Room Container and High-Level Operations

The `CRoom` class stores and manages the complete acoustic enclosure. It provides two main construction pathways and several utilities required by the simulators.

### 4.1 ROOM CONSTRUCTION OPTIONS

#### A. Shoebox Rooms

`SetupShoeBox()` creates a rectangular room defined by: length (X axis), width (Y axis), height (Z axis).

This generates six walls with:

- Proper geometry and orientations.
- Correct inward-facing normals.
- Editable absorption coefficients.
- Optional activation/deactivation of individual walls.

Shoebox walls **cannot be reshaped**, but their absorption and active status can be modified.

#### B. Arbitrary Geometry Rooms

`SetupRoomGeometry()` accepts a `TRoomGeometry` structure containing:

- A list of **corner vertices**
- A list of **walls**, each defined by a list of vertex indices

This method is the natural endpoint of the OBJ loader, and also allows users to construct custom convex rooms programmatically.

Arbitrary geometry and shoebox rooms are fully equivalent once created.

### 4.2 IMAGE ROOM GENERATION

`getImageRooms()` generates a set of **image rooms**, one for each active wall. These image rooms are used by the **Image-Source Method** inside the BRT to:

- Generate virtual (reflected) sources
- Compute high-order reflections
- Propagate room geometry recursively

This operation is internal to the acoustic engine, but the method is documented for completeness.

### 4.3 POINT-IN-ROOM EVALUATION

`CheckPointInsideRoom()` determines whether a given point lies inside the room and measures the distance to the nearest wall. This is essential internally to:

- Validate positions of real sources and their image sources
- Handle propagation constraints
- Avoid reflections that occur outside the convex enclosure

This function is exposed for advanced users, although it is primarily used by the BRT internally.

## 5. Limitations and Notes

- The room must remain **convex** at all times.
- Walls must be **planar**; slight deviations are corrected automatically.
- Extremely fine or irregular meshes may reduce numerical robustness, although no issues are currently known.
- Vertices defining each wall should remain reasonable in size and spacing to ensure accurate simulation.

## 6. Relationship with OBJ Loader

Rooms can be constructed manually or imported from an OBJ/MTL file (see *OBJ Loader* section). The OBJ loader simply creates the internal `TRoomGeometry` structure and passes it to `CRoom`. See *OBJ Reader* section for more information.

## 7. Summary

The Room Service Module provides a robust, acoustically meaningful representation of an enclosed environment for binaural rendering. It:

- Defines the boundaries of the room via polygonal walls
- Stores multi-band acoustic absorption
- Guarantees geometric constraints required for ISM and SDN
- Generates image rooms for recursive reflection modeling
- Validates source positions during propagation

Together, `CRoom` and `CWall` form the geometric and acoustic foundation of all room-based simulation within the BRT.

2.6.7 Old services

Old Services

Note

Old services present in the library up to version 2.5.0.

Service modules are auxiliary components designed to manage and supply the essential resources required for process models to perform binaural audio synthesis algorithms effectively. These resources include various key elements, such as the Head-Related Transfer Function (HRTF), which is critical for simulating the direct sound path perceived by the listener model; the Binaural Room Impulse Response (BRIR), which enables accurate simulation of room reverberation effects; and directivity data, which is vital for capturing the directional characteristics of sound sources. Additionally, second-order filters are employed to accurately model near-field effects, ensuring precise spatial audio reproduction even in close-proximity scenarios. Together, these resources form the foundation for creating immersive and realistic binaural audio experiences.

Currently, five service modules are implemented:

- **HRTF**: Stores head-related impulse responses indexed by azimuth and elevation.
- **BRIR**: Stores room-related impulse responses indexed by azimuth and elevation.
- **Directivity TF**: Stores transfer functions of a sound source based on the position of the listener and the sources.
- **SOS Coefficients**: Stores coefficients for second-order sections of a filter, which can be fixed or vary based on distance, azimuth, and/or elevation.
- **Ambisonics BIR**: Stores the impulse responses of the virtual loudspeakers in the ambisonic domains, in order to achieve a process with simultaneous impulse responses convolution and ambisonic decoding.
- **General FIR**: Stores impulse responses indexed by azimuth and elevation.
- **Room**: It stores the vertices and walls of a room. It also stores the absorption coefficients per band for each of the walls.

HRTF Service Module

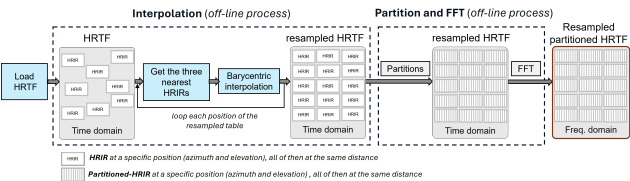
The HRTF service module is responsible for handling the reading, storage, and processing of HRTFs. These functions are essential for enabling the listener model to render spatial audio ([read more](#)). This module optimizes the accessibility and organization of the HRTF data, ensuring efficient processing and integration within the overall rendering pipeline.

An HRTF consists of a collection of HRIRs, each associated with specific measurement directions, including azimuth (left-right) and elevation (up-down) and distance. The BRT is designed to handle arbitrary distributions of HRIR measurement directions, meaning it does not rely on a regular or complete directional grid and imposes no minimum density requirements. The HRTF Service Module is in charge of estimating HRIRs for the exact direction and distance of the source if these are not explicitly included in the HRTF data. For this estimation, the algorithm selects the three nearest points at which the HRTF was measured, and performs a barycentric interpolation among the HRIRs corresponding to these three locations. In this way, this module stores the HRTF in the form of a regular grid, enabling more efficient use of resources during binaural rendering. For more details about this grid, click [here](#).

Interpolating between HRIRs with varying ITDs can lead to issues, such as audible artifacts and reduced rendering quality. To address this, the HRTF Service Module handles ITDs independently from the interpolation and convolution processes. To do so, user-imported HRIRs should be provided with ITDs stored separately. The ITDs to be added after interpolation can be either estimated by interpolating among those corresponding to the three closest HRIRs, or synthesised using data about the location of the sound source (specifically the interaural azimuth) and the head circumference of the listener.

Each HRIR is partitioned in chunks to match the input buffer length. This is done in order to use the [UPOLS convolution](#). In addition, an FFT is applied to each of the HRIR partitions and stored in memory by the Service Module, since the convolution is done in the frequency domain.

The following diagram illustrates the processing performed on the HRTF table before it is stored. This processing is performed offline, ensuring that, in real-time, we have a regular table that is faster to access.



HRTF offline process.

FUNCTIONAL OVERVIEW

The methods in the **HRTF class** are designed to facilitate the loading and management of HRTFs within the renderer. This service allows loading multiple HRTFs as needed, enabling flexible spatial audio configurations. To load an HRTF file, a separate **Reader** class is required to parse the file and add HRIRs to the HRTF class one by one. The process begins by calling the **Begin Setup** method to initialize the configuration. HRIRs are then added incrementally using the **Add HRIR** method. Once all the data has been loaded, the **End Setup** method finalizes the setup, generating a complete HRTF table for use. Additionally, the class provides various configuration options, such as setting the sampling rate, adjusting the angular grid resolution, enabling the Woodworth ITD model, and defining head and ear properties, as detailed in the methods below. These features ensure precise and efficient integration of HRTFs into the rendering pipeline.

CONFIGURATION OPTIONS

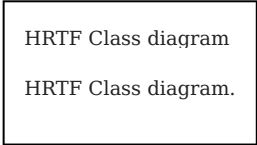
The methods provided by this service are as follows.

- **Begin Setup:** Prepares the HRTF object for initializing and starts the configuration.
- **End Setup:** Finalizes the setup process and store in the configured HRTF data.
- **Set Sampling Rate:** Specifies the sampling rate for the HRTF data
- **Add HRIR:** Adds a Head-Related Impulse Response (HRIR) to the HRTF dataset for a specific position.
- **Set/Get file name:** Assigns or retrieves the file name of the HRTF data to be used.
- **Set/Get sampling step for the grid:** Defines or retrieves the angular step size used to sample the HRTF grid.
- **Enable/Disable Woodworth ITD:** Toggles the application of the Woodworth ITD formula for HRTF processing.
- **Get HRIR Partitioned:** Retrieves the interpolated and partitioned stored HRTF for an specific direction.
- **Get HRIR Delay:** Calculates and returns the delay associated with the HRIR for a specific position.
- **Set/Get Head Radius:** Configures or retrieves the radius of the listener's head model.
- **Set ear position:** Sets the position of the ears relative to the head model.

C++ developer

- **File:** /include/LServiceModules/HRTF.hpp
- **Class name:** CHRTF
- **Inheritance:** CServicesBase
- **Namespace:** BRTServices

CLASS INHERITANCE DIAGRAM



HOW TO INSTANTIATE AND LOAD

```
// Assuming SOFA_FILEPATH contains the SOFA filename including the path
std::shared_ptr<BRTServices::CHRTF> hrtf =
std::make_shared<BRTServices::CHRTF>();
bool hrtfSofaLoaded = AppUtils::LoadSofaFile(SOFA_FILEPATH,
hrtf);
if (!hrtfSofaLoaded) {
// ERROR
}
```

HOW TO CONNECT IT TO A LISTENER

```
// Assuming that the ID of this listener is contained in _listenerID
and
// that the HRTF is already lsuccessfully loaded into hrtf.
std::shared_ptr<BRTBase::CListener> listener = brtManager-
>GetListener(listenerID);
listener->SetHRTF(hrtf);
```

PUBLIC METHODS

```
int32_t GetHRIRLength() const
void SetGridSamplingStep(int _samplingStep)
int GetGridSamplingStep()

bool BeginSetup(int32_t _HRIRLength,
BRTServices::TEXTRAPOLATION_METHOD _extrapolationMethod) override
void AddHRTFTable(T_HRTFTable&& newTable)
void AddHRIR(double _azimuth, double _elevation, double _distance,
Common::CVector3 listenerPosition, THRIRStruct&& newHRIR) override
bool EndSetup() override

void EnableWoodworthITD()
void DisableWoodworthITD()
bool IsWoodworthITDEnabled()

const std::vector<CMonoBuffer<float>>
GetHRIRPartitioned(Common::T_ear ear, float _azimuth, float
_elevation, bool runTimeInterpolation, const Common::CTransform& /*
_listenerLocation*/) const override
THRIRPartitionedStruct GetHRIRDelay(Common::T_ear ear, float
_azimuthCenter, float _elevationCenter, bool runTimeInterpolation,
Common::CTransform& _listenerLocation)
const int32_t GetHRIRNumberofSubfilters() const
const int32_t GetHRIRSubfilterLength() const override
bool IsHRTFLoaded()
float GetHRTFDistanceOfMeasurement() override

void SetTitle(std::string _title) override
void SetDatabaseName(std::string _databaseName) override
void SetListenerShortName(std::string _listenerShortName) override

void SetFilename(std::string _fileName) override
std::string GetFilename() override

void SetHeadRadius(float _headRadius) override
float GetHeadRadius() override

void SetEarPosition(Common::T_ear _ear, Common::CVector3
_earPosition) override
void SetCranialGeometryAsDefault() override
Common::CVector3 GetEarLocalPosition(Common::T_ear _ear)

void SetSamplingRate(int _samplingRate)
```

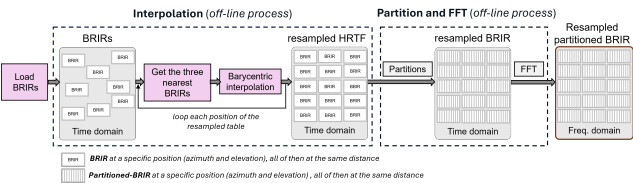
BRIR Service Module

The BRIR service module is responsible for managing the reading, storage, and processing of BRIRs. These functions are crucial for enabling the system to simulate room reverberation and provide accurate spatial audio rendering ([read more](#)). This module ensures that the BRIR data is well-organized and easily accessible, facilitating efficient processing and seamless integration within the overall audio rendering pipeline.

An BRIR consists of a collection of BRIRs, each associated with specific measurement directions, including azimuth (left-right) and elevation (up-down) and distance. The BRT is designed to handle arbitrary distributions of BRIR measurement directions, meaning it does not rely on a regular or complete directional grid and imposes no minimum density requirements. The HR-BRIR Service Module is in charge of estimating BRIRs for the exact direction and distance of the source if these are not explicitly included in the HRTF data. For this estimation, the algorithm selects the three nearest points at which the BRIR was measured, and performs a barycentric interpolation among the BRIRs corresponding to these three locations. In this way, this module stores the BRIR in the form of a regular grid, enabling more efficient use of resources during binaural rendering. For more details about this grid, click [here](#).

Each BRIR is partitioned in chunks to match the input buffer length. This is done in order to use the [UPOLS convolution](#). In addition, an FFT is applied to each of the HRIR partitions and stored in memory by the Service Module, since the convolution is done in the frequency domain.

The following diagram illustrates the processing performed on the HRTF table before it is stored. This processing is performed offline, ensuring that, in real-time, we have a regular table that is faster to access.



BRIR offline process.

FUNCTIONAL OVERVIEW

The methods in the HRBRIR class provide the tools necessary for configuring and managing Head-Related BRIR data within the renderer. This service supports loading multiple BRIRs, offering flexibility in spatial audio setups. To load a BRIR file, a dedicated [Reader](#) class is necessary to parse the file and add the BRIRs to the HRBRIR class step by step. The process begins with

initializing the configuration using **Begin Setup** and concludes with **End Setup**, which locks in the loaded data. Additional methods enable setting sampling rates, managing file names, defining sampling steps for the BRIR grid, and retrieving partitioned BRIRs for specific directions. This class also includes the `SetWindowingParameters` method, which sets the parameters for the [windowing process](#) to optimize the impulse response.

CONFIGURATION OPTIONS

The methods provided by this service are as follows.

- **Begin Setup:** Prepares the HRBRIR object for initializing and starts the configuration.
- **End Setup:** Finalizes the setup process and store in the configured HRBRIR data.
- **Set Sampling Rate:** Specifies the sampling rate for the HRBRIR data
- **Add BRIR:** Adds a Head-Related Impulse Response (HRBRIR) to the HRBRIR dataset for a specific position.
- **Set/Get file name:** Assigns or retrieves the file name of the HRBRIR data to be used.
- **Set/Get sampling step for the grid:** Defines or retrieves the angular step size used to sample the HRBRIR grid.
- **Get BRIR Partitioned:** Retrieves the interpolated and partitioned stored HRBRIR for an specific direction.
- **SetWindowingParameters:** Sets the parameters for the windowing process, defining the midpoint and rise time for both fade-in and fade-out windows in the impulse response.

 **C++ developer**

Section under construction

Directivity Service Module

The Directivity TF service module is responsible for handling the reading, storage, and processing of the transfer functions of the directivity of a sound source. These functions are essential for enabling the source model to render the directionality of a sound source ([read more](#)). This module optimizes the accessibility and organization of the Directivity TF data, ensuring efficient processing and integration within the overall rendering pipeline.

A Directivity TF table consists of a collection of Directivity Transfer Functions, each associated with specific measurement directions, including azimuth (left-right) and elevation (up-down) and distance. The BRT is designed to handle arbitrary distributions of measurement, meaning it does not rely on a regular or complete directional grid and imposes no minimum density requirements. The Directivity Service Module is in charge of estimating the Directivity transfer function for the exact direction and distance of the listener if these are not explicitly included in the loaded table. For this estimation, the algorithm selects the three nearest points at which the Directivity was measured, and performs a barycentric interpolation among the transfer functions corresponding to these three locations. In this way, this module stores the Directivity TF in the form of a regular grid, enabling more efficient use of resources during binaural rendering. For more details about this grid, click [here](#).

FUNCTIONAL OVERVIEW

Directivity class methods manage the setup, configuration, and retrieval of directivity transfer functions (TFs). They allow initialization with specific parameters, adding TFs for given directions, retrieving existing TFs with optional runtime interpolation, and setting or querying the resampling step. The process of loading a new directivity table begins by calling **BeginSetup**, followed by adding transfer functions one by one using **AddDirectivityTF**, and finalizing the configuration with **EndSetup**. To load the directivity data, a [Reader](#) class is used to read the data files and load them into the class's directivity table. These features support efficient handling and processing of directional audio data.

CONFIGURATION OPTIONS

The methods provided by this service are as follows.

- **Begin Setup**: Initializes the configuration with parameters for transfer function length and extrapolation method.
- **End Setup**: Finalizes the setup process and validates the configuration.
- **Set/Get ResamplingStep**: Defines and retrieves the resampling step size for directivity data grid.
- **Get Directivity TF Length**: Returns the length of the directivity transfer functions.
- **AddDirectivityTF**: Adds a directivity transfer function (TF) for a specific azimuth and elevation.
- **GetDirectivityTF**: Retrieves a directivity transfer function for a given azimuth and elevation, with optional runtime interpolation.

DIRECTIVITY TF FORMAT

A set of requirements must be met to the directionality data to work properly in the BRT:

- **Frequency distribution**: the frequency distribution must be linear from 0 to the Nyquist frequency minus one. The number of samples must be the same as configured in the BRT. By default, BRT has a number of samples of 256, but it is configurable using the BeRTA application.
- **Casual filters**: the filters defining directivity must be causal. Therefore, the transfer functions must have a phase that corresponds to a causal filter.

C++ developer

Section under construction

## 2.7 Readers

---

### 2.7.1 Readers

---

There are two file loaders integrated into the library, one for loading Spatially Oriented Format for Acoustics files ([SOFA](#)) and another for loading OBJ files.

- [SOFA Reader](#): The SOFA Reader class is designed to handle the reading and parsing of audio resource data from files in the SOFA format (different data types and conventions are supported).
- [OBJ Reader](#): The OBJ loader allows you to load the geometry and acoustic parameters of rooms.

## 2.7.2 SOFA Reader

### Overview

*SOFA Reader* is the main entry point in BRT for loading [SOFA](#) files into the internal data structures used by the rendering engine. SOFA format is widely adopted in spatial audio applications due to its flexibility and compatibility with 3D audio systems.

It provides a unified interface to read different types of spatial audio data from SOFA files (e.g., HRTFs, BRIRs, directivity, filters) and converts them into BRT service modules such as: `CSphericalFIRTable`, `CSphericalInterpolatedFIRTable` or `CSphericalSOSTable`. Internally, it relies on a wrapper around [libmysofa](#) to parse SOFA files and then adapts the data to the BRT conventions.

### Loaders Overview

The `SOFA Reader` class provides several specialized loaders depending on the **intended use of the data**.

#### HRTF LOADERS

- Load Head-Related Transfer Functions.
- Can produce:
  - **Interpolated HRTF datasets:** A grid of interpolated data is generated from the IR values read from the file. For further information, please see [here](#).
  - **Raw datasets:** The stored data matches that read from the file <sup>1</sup>. For further information, please see [here](#).
- It supports any HRTF convention using *FIR* or *FIR-E* data types, although it has been extensively tested with [SimpleFreeFieldHRIR](#).

#### BRIR LOADER

- Loads Binaural Room Impulse Responses.
- It supports any BRIR convention using *FIR* or *FIR-E* data types, although it has been extensively tested with [SingleRoomMIMOSRIR](#).
- It allows the BRIR to be adjusted using fade-in and fade-out parameters. To achieve this, a rectangular window with smooth edges is applied, using high-cosine progressive input and output transitions with a configurable slope. This process allows early impulse components to be attenuated and/or late impulse components to be clipped. For further details, see [here](#).

#### FIR FILTER LOADER

- Loads generic impulse responses from SOFA files.
- Used for arbitrary IR-based processing.

- It supports any convention using *FIR* or *FIR-E* data types, although it has been extensively tested with [GeneralFIR](#).

#### SOS FILTER LOADER

- Loads IIR filters (Second Order Sections).
- Typically used for:
  - Near-Field Correction filters
  - Perform any filtering based on second-order sections
- It supports any SOS convention, although it has been extensively tested with [SimpleFreeFieldHRSOS](#).

#### DIRECTIVITY LOADERS

- Load source directivity patterns.
- Requires specific SOFA convention: `SourceDirectivityFIR`.
- Can produce:
  - **Interpolated directivity:** A grid of interpolated data is generated from the IR values read from the file. For further information, please see [here](#).
  - **Raw directivity:** The stored data matches that read from the file <sup>1</sup>. For further information, please see [here](#).
- It works with the new `SourceDirectivityFIR` convention. This convention is currently under development, so this loader will adapt to any changes that may occur. It is currently compatible with version 0.3.

### Supported SOFA Data Types

The reader supports the following SOFA data types:

Data Type	Description
<code>FIR</code>	Finite Impulse Response filters
<code>FIR-E</code>	FIR with additional metadata (extended format)
<code>SOS</code>	Second Order Sections (IIR filters)

C++ developer

GENERAL METHODS

Method	Description
<code>GetLastError()</code>	Returns the last error message and resets it
<code>GetSampleRateFromSofa(const std::string&amp;)</code>	Returns the sampling rate of the SOFA file
<code>GetDataTypeFromSofa(const std::string&amp;)</code>	Returns the detected SOFA data type

HRTF LOADERS

Method	Description
<code>ReadHRTFFromSofa(...)</code>	Loads an interpolated HRTF dataset into <code>CSphericalInterpolatedFIRTable</code>
<code>ReadHRTFRawFromSofa(...)</code>	Loads raw HRTF data into <code>CSphericalFIRTable</code>

BRIR LOADER

Method	Description
<code>ReadBRIRFromSofa(...)</code>	Loads BRIR data and applies fade-in/out to separate direct and reverberant components

FIR FILTER LOADER

Method	Description
<code>ReadFIRFilterFromSofa(...)</code>	Loads a generic FIR dataset into a spherical FIR table

DIRECTIVITY LOADERS

Method	Description
<code>ReadDirectivityFromSofa(...)</code>	Loads interpolated source directivity
<code>ReadDirectivityRawFromSofa(...)</code>	Loads raw directivity without interpolation

SOS FILTER LOADER

Method	Description
<code>ReadSOSFiltersFromSofa(...)</code>	Loads SOS filters into <code>CSphericalSOSTable</code>

1. Please note that responses to the trigger are always processed and stored at the frequency and partitioned. ←←

2.7.3 OBJ & MTL File Reader

This section documents how the BRT interprets and loads room geometries defined in **Wavefront OBJ** files together with their associated **MTL** material files. The loading process is implemented through the `CObjReader` class.

Overview

The OBJ loader provides a way to define room geometry using external modeling tools. The objective is to convert each face in the mesh into a `CWall` object and assemble a complete `CRoom` instance.

The loader is based on the **rapidobj** parser and supports a minimal subset of the OBJ/MTL specification—enough to define walls and their acoustic absorption properties.

1. OBJ File Requirements and Behaviour

1.1 SUPPORTED OBJ DIRECTIVES

The loader processes a standard OBJ file and extracts:

Directive	Used?	Purpose
<code>v</code>	✓	Defines vertex positions (treated as 3D coordinates in meters).
<code>f</code>	✓	Defines polygonal faces. Each face becomes a <code>CWall</code> .
<code>o</code>	✓ (optional)	If multiple objects exist, <b>only the first one</b> is used.
<code>usemtl</code>	✓	Associates a material with each face (if available).
<code>g</code> , <code>s</code> , others	✗ ignored	Groups, smoothing groups and other OBJ directives are ignored.

1.2 COORDINATE SYSTEM AND UNITS

- Vertex coordinates map **1:1** to the `CWall` and `CRoom` structures.

- **No scaling, axis reordering, or transformation** is applied.
- Units are assumed to be **meters**.

1.3 GEOMETRY INTERPRETATION

Once parsed by rapidobj, the loader converts:

- All vertex positions → internal room vertex list.
- Each face (triangles, quads, or n-gons) → a `CWall` polygon.

Important Requirements (from CWall)

Each wall must satisfy the constraints of the `CWall` class:

- Vertices are expected in **counter-clockwise order as seen from inside the room**.
- Walls must be **planar**; if a polygon is slightly non-planar, the first three vertex define the wall's plane and extra vertices are projected onto the wall's plane automatically.
- Walls are assumed to be **convex polygons**.

⚠ **It is the responsibility of the OBJ file author to ensure that the room is geometrically valid** (closed, convex, and properly oriented). Non-convex or open meshes may result in incorrect simulation behaviour.

Recommendation on Vertex Count

- Although any number of vertices per face is supported, **we recommend defining walls using exactly four vertices** (like architectural room faces).
- Performance or acoustic correctness with highly tessellated meshes has **not been tested**.

1.4 MULTIPLE MESHES AND OBJECTS

- If the OBJ file contains multiple meshes/shapes, **only the first one is processed**.
- If no mesh is found, loading fails.

1.5 ROOM ORIGIN AND ORIENTATION

- Any coordinate origin is acceptable; the room does **not** need to be centered at (0,0,0).
- No attempt is made to reposition or normalize the geometry.

2. MTL File Requirements and Acoustic Properties

The loader supports standard MTL file structure but only uses one custom acoustic directive.

2.1 SUPPORTED MTL DIRECTIVE

Custom Directive: Acoustic Absorption Coefficients

Each material may contain the BRT-specific directive:

```
X-acoustic-coeffs a1 a2 a3 a4 a5 a6 a7 a8 a9
```

Where the nine floating-point values represent octave-band absorption coefficients at: **62.5 Hz, 125 Hz, 250 Hz, 500 Hz, 1 kHz, 2 kHz, 4 kHz, 8 kHz \* 16 kHz**. All coefficients must be within **[0, 1]**.

These values are assigned to the wall via:

```
CRoom::SetWallAbsortion(faceIndex, coefficientsVector)
```

## 2.2 UNSUPPORTED STANDARD MTL PROPERTIES

All standard optical or texture-related MTL properties are **ignored** (e.g., colors, reflectivity, texture maps).

## 2.3 MATERIAL ASSIGNMENT RULES

- Materials may be assigned per face using `usemtl`.
- If a face has **no material** or the material does not define `X-acoustic-coeffs`, the wall defaults to **zero absorption in all bands**.

## 2.4 ERROR HANDLING

- Missing MTL file: no error, but all walls get default absorption (0).
- Malformed or incomplete `X-acoustic-coeffs`: the directive is ignored and defaults applied.
- Values outside [0,1] are rejected and replaced with default absorption.

## 3. Conversion to the Room Model

Once the OBJ and MTL content has been parsed, a room (CRoom) is created and the corresponding absorption coefficients are applied to each wall of the room. All walls are **active** by default, unless they are subsequently deactivated manually.

## 4. Example

You can see all this in the following example, [link](#).

## 5. Summary of Constraints

To ensure correct binaural reverberation simulations:

- OBJ must describe a **closed, convex room**.
- Faces must be oriented **counter-clockwise from the inside**.
- Use simple geometry (preferably four-vertex walls).
- Optional MTL files may specify acoustic absorption per face.
- All walls default to **no absorption** if no materials are specified.

## 2.8 Infrastructure Entities

---

### 2.8.1 Infrastructure Entities

---

The **Infrastructure Entities** encompass the medium- and low-level modules of the BRT Library. These modules form the foundational elements upon which the high-level models are built and operate. While they are essential to the library's functionality and architecture, understanding them is not required unless you plan to extend the library or modify its architecture.

#### MIDDLE LAYER

The **Middle Layer** includes a collection of signal processing and service modules that provide the necessary building blocks for high-level models. Signal processing modules handle tasks such as convolution and filtering,

while service modules manage critical data like impulse responses. Developers looking to contribute new algorithms or models to the library will need to understand this layer, as it serves as the foundation for extending the library's functionality.

#### BOTTOM LAYER

The **Bottom Layer** represents the core of the library. It consists of fundamental classes and templates that define the modular architecture and implement mechanisms for interconnecting modules. This layer ensures the flexibility and scalability of the library, making it possible to adapt its architecture to various needs. Developers interested in modifying or extending the library's architecture will primarily work with this layer.

## 2.8.2 Processing Modules

### Processing Modules

The **Processing Modules** encompass a variety of signal processing components used by high-level models to perform simulations and audio rendering tasks. These modules handle essential operations such as convolution, filtering, and encoding, acting as the core signal processing units for the BRT Library's functionality.

#### Modular Design

To promote flexibility and ease of development, the implementation of these modules typically separates signal processing from the connectivity required within the library. This design choice facilitates both the development of new modules and their reuse in other contexts. Each processing module is generally implemented with two distinct classes or files <sup>1</sup>:

- A **signal processing class**, which is independent of the BRT Library's architecture but may rely on common components such as type definitions and vector classes. E.g. *MySignalProcessing.hpp*
- A **processor class**, which extends the signal processing class by integrating connectivity features required for interaction with other library modules. E.g. *MySignalProcessingProcessor.hpp*

For instance, in the case of the **HRTF Convolver**, the class `HRTFConvolver.hpp` implements the convolution logic without depending on connections to any other BRT module. On the other hand, the class `HRTFConvolverProcessor.hpp` integrates this functionality within the library while adding the necessary mechanisms for connecting to other modules.

#### Reusability and Integration

This modular design not only streamlines development within the BRT Library but also enhances the reusability of signal processing components. By decoupling core signal processing functions from library-specific connectivity, these modules can be easily integrated into external systems or reused in other audio processing contexts, making them highly versatile and portable.

#### LIST OF SIGNAL PROCESSOR MODULES

- **Ambisonic Domain Convolver & Processor**: The module performs ambisonic convolution for the N channels of each of the M different input sources, resulting in a monaural signal.
- **Ambisonic Encoder**: Encodes sound sources into an ambisonic format for spatial audio processing and rendering.
- **Bilateral Ambisonic Encode & Processor**: Process an audio source by applying delay and spatial expansion,

simulating near-field effects through binaural filtering, and finally encoding the signals into left and right Ambisonic channels for spatial audio rendering.

- **Binaural Filter**: Implement binaural filtering from second-order stages.
- **Directivity TF Convolver**: Process audio signals by applying a source's directional transfer function (Directivity TF) to simulate realistic sound propagation.
- **Distance Attenuator**: Implements distance-based attenuation to simulate the natural decrease in sound intensity over distance.
- **Uniform Partitioned Convolution**: Implements efficient frequency-domain convolution using uniformly partitioned overlap-save methods, optimized for long impulse responses.
- **HRTF Convolver & Processor**: Manage the convolution of the input signals, using the [Uniform Partitioned Convolution](#).
- **Near Field Effect Processor**: Integrate connectivity features to manage binaural filter processing

---

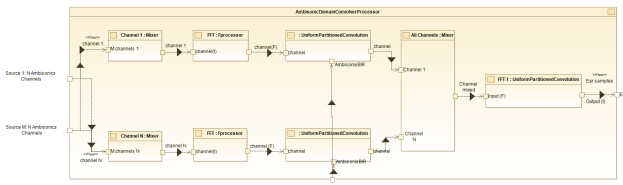
1. There are processing modules that are used internally by other modules or signal processors. In this type of module, the class that adds connectivity functionality has not been implemented. ←

### Ambisonic Domain Convolver Processor

The **Ambisonic Domain Convolver Processor** is a system designed for processing Ambisonic audio channels, performing frequency-domain transformations, convolutions, and mixing to produce spatially accurate sound for binaural listening. The processor handles multiple input channels and applies several processing stages, including FFT, convolution with Ambisonic BIR, and final mixing to create the output for the listener's ears.

#### ARCHITECTURE

The internal block diagram of this class is as follows:



Ambisonic Domain Convolver Processor - Internal diagram.

As it is shown in the diagram, the **Ambisonic Domain Convolver Processor** takes multiple Ambisonic channels, applies FFT processing, convolves with Ambisonic BIR to create spatial effects, mixes the processed channels, and delivers the final output as binaural ear samples, simulating an immersive listening experience for the listener.

#### Key Components and Flow:

- Channel Mixer:** Each mixer processes the respective channels ( `channel 1` through `channel N` ) from each sound source, for further frequency-domain transformation.
- FFT Processor:** The *FFT Processor* blocks apply a Fast Fourier Transform. Each processed channel goes to Uniform Partitioned Convolution.
- Uniform Partitioned Convolution:** The frequency-domain channels are passed into the *Uniform Partitioned Convolution* blocks. These blocks convolve the signals with the *AmbisonicBIR*.
- All Channels - Mixer:** The convolved outputs from all channels ( `Channel 1` through `Channel N` ) are combined in the *All Channels - Mixer*. This step ensures the final output integrates contributions from all Ambisonic channels.
- Final Convolution and Output:** The mixed signal ( `Channel mixed` ) undergoes a final convolution step in the *FFT : UniformPartitionedConvolution* block. The result is output as *Ear samples*, simulating spatial audio perception at the listener's ears.

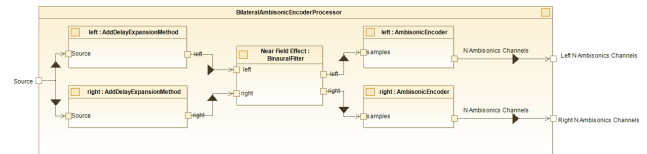
**C++ developer**  
(In progress)

### Bilateral Ambisonic Encoder Processor

The **Bilateral Ambisonic Encoder Processor** processes an audio source by applying delay and spatial expansion, simulating near-field effects through binaural filtering, and finally encoding the signals into left and right Ambisonic channels for spatial audio rendering.

#### ARCHITECTURE

The internal block diagram of this class is as follows:



Bilateral Ambisonic Encoder Processor - Internal diagram.

#### Key Components and Flow

The **Bilateral Ambisonic Encoder Processor diagram** describes the internal workflow for encoding a source signal into Ambisonic channels for both left and right audio streams.

- AddDelayExpansionMethod:** The source signal is first passed through the `left` and `right` *AddDelayExpansionMethod* blocks. These methods apply a delay and spatial expansion to the signal to account for spatial positioning and propagation effects.
- Near Field Effect - Binaural Filter:** The processed `left` and `right` signals are then passed through a *Binaural Filter*. This stage simulates near-field audio effects implementing binaural filtering from second-order stages.
- AmbisonicEncoder:** The filtered `left` and `right` signals are fed into their respective *AmbisonicEncoder* blocks. These encoders process the samples and output *N Ambisonic Channels* for both left and right streams.
- Output:** The final output consists of *Left N Ambisonics Channels* and *Right N Ambisonics Channels*, which represent spatially encoded versions of the source signal.

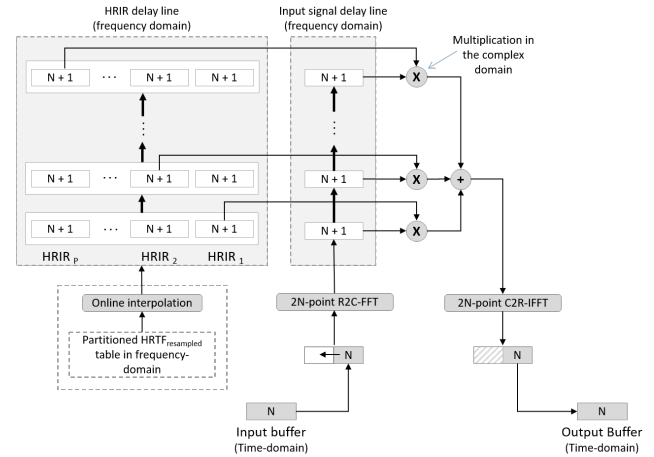
**C++ developer**  
(In progress)

### Uniformly Partitioned Convolution

For both anechoic and reverberation paths, the BRT utilizes the Uniformly Partitioned Overlap-Save (UPOLS) convolution method in the frequency domain, as described by [Wefers \(2015\)](#). This FFT-based approach divides the impulse response (IR) into multiple blocks, each matching the frame size ( $N$ ). These blocks are processed individually as separate IRs using a standard overlap-save technique. This method enables efficient convolution with long IRs by breaking them into shorter, more manageable segments. It is particularly advantageous for long IRs, such as BRIRs.

The figure shows the whole process of the UPOLS in the case of the HRIR, but it is the same for the BRIR. The partitions and FFTs of the HRTF are computed offline, where the whole HRTF is partitioned in blocks of length  $N$  (same length as the frame size) and stored. At run-time, for every audio frame, a new input buffer arrives, the content of the input block buffer is shifted  $N$  samples to the left and the new input of  $N$  samples is then placed on the right (see bottom-left part of the diagram). Then, the whole input buffer is transformed into the frequency domain using a 2N-point real-to-complex FFT and stored in a delay-line, following an overlap-save scheme. In this way, in each audio frame, the input signal stored in the delay line is shifted up by one frame slot. During the audio frame, each delayed input buffer is convolved with each HRIR segment, by a multiplication in the complex domain. Finally, all the multiplications results are mixed and transformed back into the time-domain. To do so, the BRT implement a 2N-point complex-to-real IFFT, where the first  $N$  points are discarded, as we are using an overlap-save scheme. These sample removal and the size used for the FFT and IFFT of 2N-point are implemented to avoid aliasing in the time domain. In addition, zero-padding is used to complete the signal buffer in case it is needed.

Additionally, UPOLS assumes a stable impulse response, which is suitable for static sources and listeners. To accommodate the changes in the HRIR caused by the movement of sources or the listener, the BRT introduces a new delay line for the partitioned impulse response (top-left grey box). For each audio frame, both delay lines (HRIR and input buffer signal) are shifted by one frame slot. This allows a new HRIR, corresponding to the source's position, to be inserted into the delay line at each frame, with its first segment multiplied by the current audio frame. In subsequent frames, the HRIR remains in the delay line, convolving its remaining segments with the incoming audio frames. This method significantly reduces the number and intensity of artifacts caused by the movement of sources.



Uniformly Partitioned Overlap-Save (UPOLS).

#### FUNCTIONAL OVERVIEW

The methods of the UPOLS convolution class are designed to efficiently manage and execute frequency-domain convolution for both anechoic and reverberation paths. The process begins with the **Setup** method, where the input size, block size and number of blocks are configured to prepare the system for operation. Convolution can then be performed using **ProcessUPConvolution** method, which processes the input signal with the given impulse responses (IRs) and outputs the result. For scenarios with moving source and listener, **Process UPConvolution With Memory** extends this functionality. These methods collectively enable efficient convolution with long IRs by leveraging the Uniformly Partitioned Overlap-Save (UPOLS) algorithm.

#### CONFIGURATION OPTIONS

The methods provided by this class are as follows.

- **Setup:** Configures the system for UPOLS convolution with specified input and IR settings.
- **Process UPConvolution:** Executes convolution of the input signal with the provided impulse responses, generating the output.
- **Process UPConvolutionWithMemory:** Performs convolution while retaining intermediate states, for scenarios with moving sources and listener.
- **Calculate IFFT:** Computes the inverse FFT of a buffer for frequency-to-time domain transformation.
- **Reset:** Resets the system, clearing all internal states and buffers for a fresh start.

**C++ developer**

Section under construction

## 2.8.3 Common

### Common

- **Biquad filter:** Provides tools for applying second-order IIR filtering to audio signals. It supports filter setup using specified coefficients.
- **Buffer:** Manages temporary storage of audio data during processing, ensuring efficient data flow through the renderer.
- **Conventions:** Defines and enforces data format standards and spatial audio conventions, such as coordinate systems.
- **Cranial Geometry:** Models the geometry of the human head to enhance spatial audio accuracy by accounting for head-related effects.
- **Envelope Detector:** Extracts the amplitude envelope of an audio signal.
- **Error Handler:** Manages error detection and reporting during audio processing to ensure robustness and debugging support.
- **FFSG:** Implement the fast Fourier transforms by Takuya OOURA Copyright(C) 1996-2001.
- **Filters Chain:** A modular pipeline that allows chaining multiple audio filters for sequential processing of sound signals.
- **FFT calculator:** Audio signal processor that calculates FFTs and IFFTs.
- **IR Windowing:** Applies a window function to impulse responses (IRs).
- **Profiler:** Monitors and measures the performance of various components within the renderer, tracking the CPU usage of the rendering process.
- **Quaternion:** Represents rotational transformations in 3D space, crucial for accurately positioning sound sources and listeners.
- **Room:** Enable the creation and management of 3D room geometries and acoustic properties, supporting predefined shapes and wall configurations.
- **Transform:** Handles spatial transformations like translation, rotation, and scaling for positioning sources or listeners.
- **Vector3:** Represents 3D vectors, often used for positions, directions, or velocity in spatial audio calculations.
- **Wall:** Models a physical wall to simulate reflections and obstructions in room acoustics.
- **Wave Guide:** Simulates wave propagation in 3D space, often used for modeling sound in complex environments.

## 3. Applications

### 3.1 BRT Applications: General Overview

The **Binaural Rendering Toolbox (BRT)** includes a collection of applications designed to provide researchers and developers with ready-to-use tools for creating, testing, and deploying immersive audio experiences. These applications leverage the core functionalities of the BRT Library and extend them into practical, real-time tools for diverse use cases. This section introduces the main applications, their features, and intended use cases.

#### 3.1.1 Introduction

The BRT Applications are standalone software tools built around the BRT Library, enabling seamless integration of binaural rendering into various workflows. These applications are tailored for both experimental research and real-world implementation, offering high configurability and interoperability with existing systems. From immersive auditory experiments to wearable solutions, BRT Applications are designed to meet the demands of modern psychoacoustics and virtual acoustics research.

#### 3.1.2 Key Features

1. **Interoperability:**
- All BRT Applications support the **Open Sound Control (OSC)** protocol for external control, allowing integration with other tools and frameworks such as MATLAB, PureData, and Unity 3D.
  - Compatible with standard file formats like **SOFA** and **WAV**, ensuring smooth data exchange and reproducibility.
2. **Reproducibility:**
- Applications record all relevant parameters of binaural simulations, facilitating accurate reproduction and analysis of experimental results.
  - Support for a new SOFA-based convention for annotated audio, capturing dynamic spatial configurations alongside binaural audio.

#### 3.1.3 Applications

##### BeRTA Renderer (BRT Audio Renderer)

**BeRTA Renderer** is the flagship application of the BRT Toolbox, providing a high degree of configurability for real-time spatial audio rendering. Key functionalities include:

- **Real-time Rendering:**
  - Dynamic control of sound source locations, listener positions, and other spatial parameters via OSC commands.
  - Support for multiple sound sources and HRTF swapping in real time.
- **Configurable Virtual Scenarios:**
  - User-defined configurations for different buffer sizes, sample rates, and rendering parameters.
  - GUI-based feedback for monitoring application status, including rendering states and OSC commands.
- **Data Saving and Export:**
  - Saves all parameters and audio in the annotated audio format for reproducibility.
  - Exports simulation data for further analysis or use in auditory modeling.

### Wearable Solutions

BRT Toolbox also includes applications designed for portable, low-latency rendering on wearable devices:

- **BeRTA-mini:**
  - A lightweight version of BeRTA tailored for wearable platforms, combining a BELA audio board, IMU sensors (gyroscope, magnetometer, accelerometer), and headphones.
  - Designed for on-the-go binaural rendering with high responsiveness and accuracy.
- **Tracker Interoperability:**
  - Integrates with MBIENTLAB's MetaMotionRL and other motion tracking devices.
  - Converts tracking data into OSC commands for seamless interaction with other BRT applications.

### Future Applications

Several additional tools are under development or planned for future releases, including:

- Multi-listener rendering applications.
- Tools for simulating advanced reverberation models (e.g., convolution-based and hybrid approaches).
- Hearing loss and hearing aid simulations.

### 3.1.4 Usage

BRT Applications are ideal for:

- Psychoacoustic experiments requiring dynamic spatial configurations.
- Integration into existing experimental setups with OSC-controlled workflows.
- Real-time virtual acoustics rendering for VR/AR applications and wearable devices.

These applications are designed to minimize the coding effort required to deploy sophisticated audio rendering capabilities, making them accessible to a wide range of users.

---

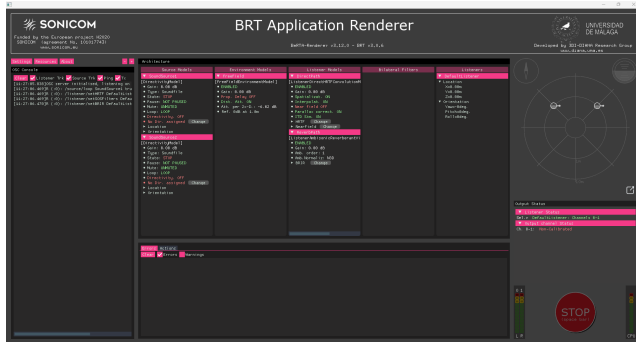
For more information about the BRT Applications and their functionalities, visit the [BRT GitHub Repository](#).

## 3.2 BeRTA Rendeder

### 3.2.1 BeRTA Renderer

BeRTA Renderer is the flagship application of the BRT Toolbox, providing a high degree of configurability for real-time spatial audio rendering. The application integrates the BRT library and works for Windows x64 (Windows 10 and 11) and MacOS Universal.

```markdown



BeRTA Renderer Interface

Key functionalities include:

- **Real-time Rendering:**
 - Dynamic control of sound source locations, listener positions, and other spatial parameters via OSC commands.
 - Support for multiple sound sources and HRTF swapping in real time.
- **Configurable Virtual Scenarios:**
 - User-defined configurations for different buffer sizes, sample rates, and rendering parameters.
 - GUI-based feedback for monitoring application status, including rendering states and OSC commands.
- **Data Saving and Export:**
 - Saves all parameters and audio using the proposed annotated audio SOFA convention format for reproducibility.
 - Exports simulation data for further analysis or use in auditory modeling.

The application is fully controlled through **OSC** (Open Sound Control), resulting in a graphical interface focused primarily on reporting rendering status. The GUI consists of terminal-like sections that display critical information, such as rendering parameters (e.g., enabled or disabled features) and the settings of audio sources and the listener. Additional terminals provide detailed logs of OSC commands received and sent, actions executed by BeRTA, and error messages detected by BeRTA's built-in diagnostic system.

Some of the relevant menus are:

- **Settings menu:** allows users to view and configure general application parameters.
- **Resources menu:** displays all resources currently loaded in the application.

Other important features:

- **Settings file:** How to define an application configuration file that is read when the application starts.
- **Calibration:** BeRTA provides a calibration option, which allows establishing the relationship between the system's dBFS (Decibels Full Scale) and the SPL (Sound Pressure Level) in dB of the output signal.
- **Safety Limiter:** BeRTA includes a Safety Limiter that allows users to define a maximum SPL threshold, preventing output levels from exceeding safe limits.

PARAMETERS

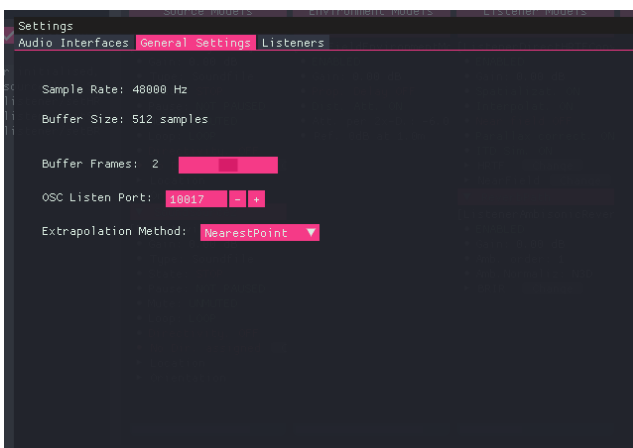
3.2.2 Settings Menu

The **Settings** menu allows users to view and configure application parameters, audio interfaces and listener output routing. It contains three tabs:

- **Audio Interfaces:** Select the input and output devices and the number of available channels.
- **General Settings:** View and configure global application parameters.
- **Listeners:** Assign listeners to physical output pairs, control their Active/Standby state and configure calibration references for virtual listeners.

General settings

This section displays global parameters defined in the `settings.json` configuration file. Some of these parameters can be modified directly from the interface, while others are read-only.



BeRTA Renderer - General Settings menu

| Parameter | Description | Editable |
|----------------------|--|---|
| Sample rate | Defines the sampling rate used by the application. All loaded audio files and audio devices must operate at this same rate. |  No |
| Buffer size | Number of frames per processing block. Larger values increase latency but provide more processing time, enabling more complex simulations (e.g., reverberation). |  No |
| Buffer Frames | Parameter related to the underlying audio backend (e.g., RtAudio on Windows). It defines the number of internal buffers used before sending audio to the output device. The operating system will try to honor this value, but it is not guaranteed. Higher values may improve stability but increase latency. |  Yes |
| OSC Listen Port | UDP port where BeRTA listens for incoming OSC messages. |  Yes |
| Extrapolation Method | Defines the extrapolation strategy used during |  Yes |

| Parameter | Description | Editable |
|-----------|---|----------|
| | interpolation when loading HRTFs or directivity data. Available options: <ul style="list-style-type: none">ZeroInsertion: Missing data points are filled with zeros. This method avoids introducing artificial information but may produce discontinuities.NearestPoint: Uses the closest available data point. This provides smoother results but may introduce spatial bias. | |

Audio Interfaces

This section allows selecting the audio devices used for input and output.

Two lists are displayed:

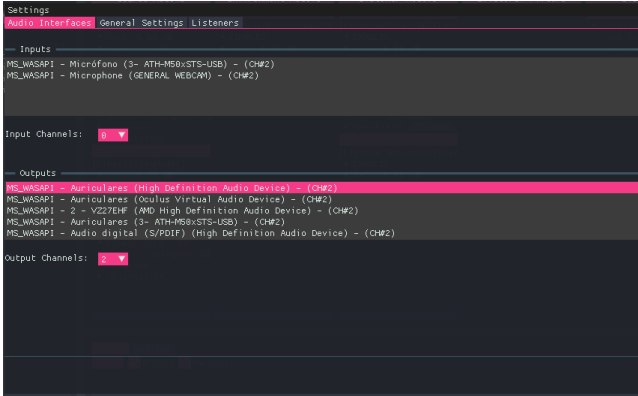
- Available **input devices**
- Available **output devices**

Each device entry includes:

- Audio backend (e.g., ASIO, WASAPI)
- Device name
- Number of available channels

**Info**

Both input and output devices must use the **same audio backend/API**. This is a limitation imposed by the underlying audio library (audio backend).



BeRTA Renderer - Audio Interfaces menu

INPUT INTERFACE

The input interface is used for **line-in sound sources**.

Input Channels

The **Input Channels** dropdown defines how many channels from the selected input device will be available for use.

- Channels are assigned **sequentially starting from 1**
- If set to **0**, no input channels are used
- If set to **N**, the first **N** channels are enabled

When creating a line-in source, the parameter `linein_channel` specifies which of these enabled channels is used.

Example:

- Device with 4 input channels
- Input Channels set to **2**
- Available channels for use: **1 and 2**
- Valid `linein_channel` values: **1** or **2**

OUTPUT INTERFACE

The output interface defines the audio device used for playback.

Output Channels

The **Output Channels** dropdown determines how many physical output channels are available for listener routing.

- Output channels are enabled in stereo pairs: 2, 4, 6, etc.
- Output channel numbering starts at **0**. Four enabled channels provide pairs **0-1** and **2-3**.
- Each pair can reproduce the signal of one active listener at a time.
- Several listeners may share an assigned pair, but only one of them can be Active.
- Virtual listeners do not use physical output channels.

The number of output pairs limits how many listeners can be heard simultaneously. It does not limit the total number of listeners in the scene.

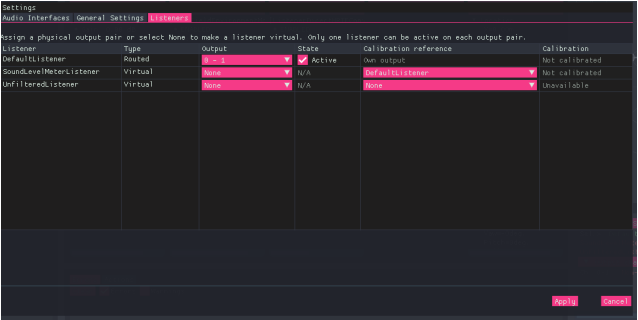
Example:

With **4** output channels enabled, two listeners can be heard simultaneously through separate pairs. Additional listeners may share those pairs in Standby or remain virtual.

Configure these assignments in the **Listeners** tab described below.

Listeners

The **Listeners** tab configures how existing listeners connect to the audio interface and how they obtain the calibration used to estimate sound levels. Each listener produces its own stereo signal from its position and orientation in the scene. The output configuration determines whether that signal is sent to a physical output pair, kept in Standby or used only for virtual measurements. This tab configures existing listeners. It does not create or delete them.



BeRTA Renderer - Listener menu

TABLE COLUMNS

| Column | Description |
|-----------------------|--|
| Listener | Identifier of the existing listener. Read-only in this tab. |
| Type | <code>Routed</code> or <code>Virtual</code> , determined by the output assignment. |
| Output | Physical stereo pair, or <code>None</code> (<code>Virtual</code>) to remove the assignment. |
| State | Checkbox controlling <code>Active</code> or <code>Standby</code> for <code>Routed</code> listeners. Virtual listeners display <code>N/A</code> . |
| Calibration reference | Own output for <code>Routed</code> listeners. For <code>Virtual</code> listeners, another <code>Routed</code> listener or <code>None</code> . |
| Calibration | Availability of the calibration resolved through the listener's own output or its reference. |

LISTENER TYPES

The **Type** column is determined automatically by the **Output** assignment, and can be *Routed* or *Virtual*.

Routed

A **Routed** listener has a physical stereo output pair assigned to it, such as `0 - 1`.

It can be:

- **Active:** Its signal is sent to the assigned output pair.
- **Standby:** The assignment is retained, but its signal is not sent to the physical output.

A `Routed` listener uses the calibration of its own assigned output pair when calculating its theoretical sound level in dB SPL.

Virtual

A **Virtual** listener has no physical output assignment. Its **Output** is `None`, and its **State** is `N/A`. Its signal remains available for digital level measurements in dBFS. It can also provide theoretical dB SPL levels if it has a valid calibration reference.

Virtual listeners are useful for evaluating the signal at additional positions in the scene without using more physical output channels.

Standby and Virtual have different meanings

A `Standby` listener retains a physical output assignment and can be activated later. A `Virtual` listener has no physical output assignment. Neither state disables the calculation of the listener's own signal levels.

ASSIGNING A PHYSICAL OUTPUT PAIR

1. Locate the listener in the table.
2. Open its **Output** dropdown.
3. Select an available pair, such as `0 - 1` or `2 - 3`.
4. Enable the **State** checkbox if the listener should be `Active`.
5. Press **Apply**.

Points to note:

- The available pairs depend on the output channel count selected in **Audio Interfaces**.
- Assigning a different pair puts the listener into **Standby**. Activate it explicitly when you want it to take control of that pair.
- Assigning a physical output also removes any previous calibration reference: `Routed` listeners use their own output pair.

SHARING AN OUTPUT PAIR

Several `Routed` listeners may be assigned to the same output pair, but only one can be `Active` at a time. When you activate one listener, the other listeners assigned to that pair automatically change to `Standby` in the table.

When you change this in the interface, you must click **Apply** for the change to take effect.

For example:

| Listener | Output | State |
|----------|--------------------|----------------------|
| A | <code>0 - 1</code> | <code>Active</code> |
| C | <code>0 - 1</code> | <code>Standby</code> |

Points to note:

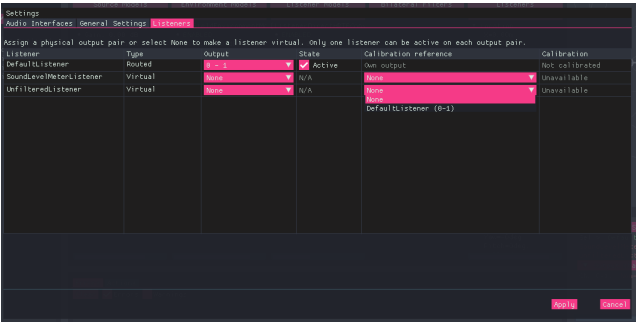
- Activating C changes A to `Standby`. After applying the changes, channels 0-1 reproduce C's signal.
- Listeners assigned to different pairs can be `Active` simultaneously. Signals from listeners sharing a pair are not mixed together.
- If all listeners assigned to a pair are in `Standby`, none of their signals is sent to that pair.

CONFIGURING A VIRTUAL LISTENER

1. Open the listener's **Output** dropdown.
2. Select **None (Virtual)**.
3. If theoretical dB SPL measurements are required, select a Routed listener in **Calibration reference**.
4. Otherwise, leave the reference as **None**.
5. Press **Apply**.

Points to note:

- Selecting **None (Virtual)** removes the physical assignment, deactivates output routing and clears the listener's previous calibration reference. Choose the desired reference afterwards.
- The reference dropdown lists eligible Routed listeners together with their assigned pairs, for example **A (0-1)** . A listener cannot reference itself or another Virtual listener.
- A Virtual listener without a calibration reference is a valid configuration. Its dBFS measurements remain available, but its theoretical dB SPL level is unavailable.



BeRTA Renderer - Listener menu - Virtual listener's Calibration reference dropdown

HOW CALIBRATION REFERENCES WORK

Calibration belongs to a physical output pair. A Virtual listener obtains its calibration through a reference to a Routed listener:

Virtual listener B → Routed listener A → A's current output pair → calibration

B uses **its own signal level** and the calibration associated with A's output pair. It does not use A's signal level.

The theoretical level is calculated as:

listener level in dB SPL = listener level in dBFS + calibration offset in dB

For more information about establishing the calibration, see [Calibration](#).

The reference follows the listener

If A moves from channels 0-1 to channels 2-3, B's reference still points to A. Once the change is applied, B uses the calibration of channels 2-3.

If the new pair is not calibrated, B's theoretical dB SPL measurement becomes unavailable until calibration is provided.

The referenced listener may be in Standby

A does not need to be Active to provide a calibration reference.

For example:

| Listener | Type | Output | State |
|----------|---------|--------|---------|
| A | Routed | 0 - 1 | Standby |
| C | Routed | 0 - 1 | Active |
| B | Virtual | None | Standby |

In this configuration:

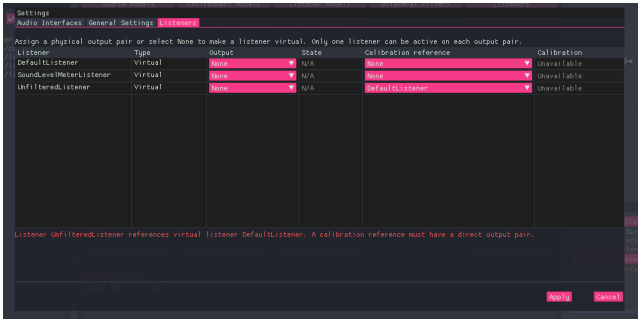
- Channels 0-1 reproduce C's signal.
- A's theoretical dB SPL level uses A's signal and the calibration of channels 0-1.
- B's theoretical dB SPL level uses B's signal and the same calibration.
- B continues to reference A, even though C is currently audible.

Converting a referenced listener into Virtual

A Virtual listener cannot provide a calibration reference to another Virtual listener.

If you change A to Virtual while B still references A, the GUI reports an invalid configuration. Before applying it, change B's reference to another Routed listener or select **None**.

- This column describes calibration availability. It does not indicate whether a listener is audible or whether a current signal measurement is available.
- Routing may be configured before calibration is performed.
- Calibration is needed to convert digital levels into estimated dB SPL levels.
- The **Listeners** tab selects the calibration source; it does not perform the calibration procedure.



BeRTA Renderer - Listener menu - Calibration error

 Session configuration

Applying listener changes updates the running application. It does not automatically save those changes to `settings.json`.

Listener output configuration can also be set using `/control/setListenerOutput`. See the [OSC control documentation](#) for its arguments and behaviour.

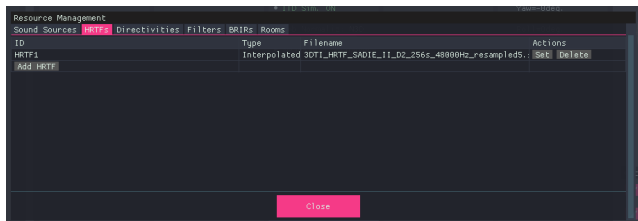
3.2.3 Resources Menu

The **Resources** menu displays all resources currently loaded in the application.

Resources can be loaded in three ways:

- Dynamically during execution via **OSC commands**
- At startup, from the resource lists defined in the `settings.json` configuration file
- Using the 'Add' buttons in this menu of the user interface

The content of this menu is updated **in real time** as new resources are loaded.



BeRTA Renderer - Resources menu

Resource Lists

Resources are organized into separate tabs, one for each resource type:

- **Sound Sources**
- **HRTFs**
- **Directivities**
- **Filters**
- **BRIRs**
- **Rooms**

Each tab contains a table listing the resources of that type.

SOUND SOURCES

Displayed Information

Each resource is represented as a row in the corresponding table. The following information is shown:

| Column | Description |
|---------------------------------|--|
| Source ID | Unique identifier of the sound source |
| Source Model | Specify the type of microphone used: directional or omnidirectional. |
| Input type | Indicate whether the source has been loaded from an audio file or is being read in real time from an input channel on the interface. |
| WAV File / Input Channel | Displays the audio file or input channel currently in use. |
| Actions | Button to delete the sound source from the scene |

SOFA RESOURCES

Displayed Information

Each resource is represented as a row in the corresponding table. The following information is shown:

| Column | Description |
|--------------------|--|
| ID | Unique identifier of the resource within the application |
| Type | Describes the nature of the resource. Its meaning depends on the resource category (see below) |
| Source File | Path or name of the file from which the resource was loaded |
| Actions | Buttons to delete or set that resource on a compatible model. |

Note

The **Type** column has a different meaning depending on the resource category.

Type Column by Resource

The interpretation of the **Type** column varies depending on the selected tab:

- **HRTFs / Directivities**: Indicates whether the dataset has been **interpolated** or loaded as **raw data**
- **Filters**: Indicates the filter structure.
- **FIR** (Finite Impulse Response)
- **IIR** (Infinite Impulse Response)
- **Rooms**: Indicates the room model type.
- **Shoebox** (parametric rectangular room)
- **Free-form** (geometry loaded from an external `.obj` file)
- **BRIRs**
- This category does **not include a Type column**
- Only **ID** and **Source File** are displayed

Actions Buttons

- **Add X**: Opens a pop-up menu to load a new resource of type X.
- **Set**: Set that resource to a compatible model from those loaded in the scene.
- **Delete**: Remove that resource.

Notes

- The same resource file can be loaded several times using different IDs.

3.2.4 Configuration File Setup

This section explains how to define a configuration file in JSON format that is read when the application starts. If the program is launched without arguments, it will look for a file called `settings.json` in the same folder. Alternatively, a custom configuration file can be provided as an argument, for example, `mysettings.json`. This file defines the audio interface settings, the model architecture, and, optionally, a set of resources to be re-loaded at the beginning and a complete scenario with sources and listeners.

General Structure

The JSON configuration file is organized into several key sections that define settings such as general parameters, model architecture, resources (HRTFs, BRIRs, etc.), and scene configuration. Below is a guide on how to construct each section.

1. GENERAL SETTINGS

These settings control general application parameters.

- **SampleRate:** The sample rate in Hz. Example: `48000`.
- **BufferSize:** The size of the buffer in samples. Example: `512`.
- **BufferFrames:** Controls stream latency. Larger values may provide better performance but increase latency.
- **OSCListenPort:** The port on which the OSC server listens for messages.
- **ExtrapolationMethod:** The method of extrapolation. Available options: `ZeroInsertion`, `NearestPoint`.
- **DefaultInputAudioInterface:** The identifier of the input audio interface to be selected by default. These identifiers can be viewed in the application console when the application is running. If set to `-1` (default) or if the identifier cannot be found, the interface selected by default in the operating system is chosen.
- **DefaultOutputAudioInterface:** The identifier of the output audio interface to be selected by default. These identifiers can be viewed in the application console when the application is running. If set to `-1` (default) or if the identifier cannot be found, the interface selected by default in the operating system is chosen.
- **LogFile :** Selects whether to save a log file or not. `true`, `false`.
- **LogFilePath :** Folder where the log file is stored. By default the application saves the log files in the subfolder 'data' inside the installation folder. On Windows, if the installation folder is 'Program Files', you will need to run the program with administrator permissions.

Example:

```
"GeneralSettings": {
  "SampleRate": 48000,
  "BufferSize": 512,
  "BufferFrames": 4,
  "OSCListenPort": 10017,
  "ExtrapolationMethod": "NearestPoint",
  "DefaultInputAudioInterface": -1,
  "DefaultOutputAudioInterface": -1,
  "LogFile": true,
  "LogFilePath": "",
},
```

2. MODELS ARCHITECTURE

This section defines the models used for sound rendering, including listeners and environment models.

- **Listeners:** Defines the physical listener(s) by ID. Example: `["DefaultListener"]`.
- **ListenerModels:** Specifies the listener models, identified by an ID and model type. The available types include `ListenerDirectHRTFConvolution`, `ListenerDirectBRIRConvolutionModel`, `ListenerAmbisonicVirtualLoudspeakersModel`, and `ListenerAmbisonicReverberantVirtualLoudspeakersModel`.
- **EnvironmentModels:** Specifies the environment models, such as the `SDNEnvironmentModel` and the `FreeFieldEnvironmentModel`.
- **Bilateral Filters:** Specifies the bilateral filter used. The two binaural filter model available : `SOSBinauralFilter` and `FIRBilateralFilter`.
- **ConnectSourcesTo:** Defines the default models (listener or environment) to which audio sources are connected upon loading, unless otherwise specified. If, when loading a source using the `loadSoundSource` OSC command or via the 'Sound Sources' section of this file, the models to be connected are not specified, the list defined here is used.
- **Model2ModelConnections:** Defines how are the interconnections between the different models (environment, listening and binaural filters). The outputs of the `OriginID` models will be connected to the inputs of the `DestinationID` models.
- **ConnectToListener:** Specifies how the listener models are connected to the listener. The output of the listener models is connected to the listener for mixing.

Example:

```
"ModelsArchitecture": {
  "Listeners": ["DefaultListener"],
  "ListenerModels": [
    {"ID": "DirectPath", "Model":
"ListenerDirectHRTFConvolution"},
    {"ID": "ReverbPath", "Model":
"ListenerAmbisonicReverberantVirtualLoudspeakersModel"}
  ],
  "EnvironmentModels": [{"ID": "FreeField", "Model":
"FreeFieldEnvironmentModel"}],
  "BilateralFilters": [{"ID": "Bilateral1", "Model":
"SOSBilateralFilter"}],
  "ConnectSourcesTo": ["FreeField", "ReverbPath"],
  "Model2ModelConnections": [{"OriginID": "FreeField",
```

```
"DestinationID": "DirectPath"}],
  "ConnectToListener": [
    {"ModelID": "DirectPath", "ListenerID": "DefaultListener"},
    {"ModelID": "ReverbPath", "ListenerID": "DefaultListener"}
  ],
}
```

3. RESOURCES

This section lists the resources such as HRTFs, BRIRs, NearField Filters, and DirectivityTFs that are loaded when the application starts.

- **HRTFs:** A list of HRTFs (Head-Related Transfer Functions) to load. If no HRTFs should be loaded, leave the list empty ([]).
- **BRIRs:** A list of BRIRs (Binaural Room Impulse Responses) to load. BRIRs can provide spatial audio information specific to a particular room. If none should be loaded, leave this list empty ([]).
- **Filters:** Defines the filters to load. The filters can be either SOS or FIR filters. If no should be loaded, leave the list empty ([]).
- **Directivities:** A list of directivity to apply to sound sources. These describe how sound is emitted in different directions from a sound source. If no Directivity should be loaded, leave the list empty ([]).
- **Rooms:** A list of room definitions to be loaded. You can specify the loading of an OBJ file, or define a shoebox-type room (indicating the dimensions). If no Rooms should be loaded, leave the list empty ([]).

Example:

```
"Resources" : {
  "HRTFs": [{"ID": "HRTF1", "fileName": "resources//HRTF//
3DTI_HRTF_SADIE_II_D2_256s_48000Hz_resampled5.sofa",
"spatialResolution": 5}],
  "BRIRs": [{"ID": "BRIR1", "fileName": "resources//BRIR//
3DTI_BRIR_Trapezoid_48000Hz_3D.sofa", "spatialResolution": 15,
"fadeInWindowThreshold" : 0, "fadeInWindowRiseTime": 0,
"fadeOutWindowThreshold" : 0, "fadeOutWindowRiseTime": 0}],
  "Filters": [
    {"ID": "DefaultNFFilters", "fileName": "resources//
SOSFilters//NearFieldCompensation_ILD_1.2m_48KHz.sofa"},
    {"ID": "JBLCompensation", "fileName": "resources//Filters//
Phone_Compensation_JBLQuantum100_48000.sofa"}
  ],
  "Directivities": [
    {"ID": "Directivity1", "fileName": "resources//
Directivity//
Cardioid_LP_30dB_512s_resampled10_normalized_fir_512.sofa"}
  ],
  "Rooms": [
    {"ID": "Room1", "length": 3, "width": 5.5, "height": 3},
    {"ID": "Room2", "fileName": "resources//OBJ//
A108_room.obj"}
  ]
}
```

4. SOUND SOURCES

This section defines the sound sources to be loaded at startup. Each source must specify the full filename and the model used for rendering. The available source model options are `OmnidirectionalModel` or `DirectivityModel`.

- **ID:** A unique identifier for each sound source.
- **fileName:** The full path to the sound file to be used.
- **sourceModel:** Specifies the rendering model for the sound source. The two available models are `OmnidirectionalModel` and `DirectivityModel`.
- **modelToConnectTo:** This is an optional parameter that specifies the listener or environment models to which the source will be connected. If it is not specified, the source will be connected to the models listed in the `'ConnectSourcesTo'` field of the `'ModelsArchitecture'` section of this configuration file.

Example:

```
"SoundSources": [
  {"ID": "SoundSource1", "fileName": "resources//
MusArch_Sample_48kHz_Anechoic_FemaleSpeech.wav",
"sourceModel": "OmnidirectionalModel", "modelToConnectTo":
"DirectPath"},
  {"ID": "SoundSource2", "fileName": "resources//
MusArch_Sample_48kHz_Anechoic_MaleSpeech.wav",
"sourceModel": "OmnidirectionalModel"}
]
```

5. SCENE CONFIGURATION

This section allows you to define the scene by specifying commands in OSC (Open Sound Control) format. These commands configure various parameters like looping sound sources, setting HRTFs, and configuring environmental properties such as room dimensions and wall absorption coefficients.

- **command:** The OSC command to execute.
- **parameters:** The parameters that accompany each OSC command.

Example:

```
"SceneConfiguration": [
  {"command": "/source/loop", "parameters": ["SoundSource1",
"true"]},
  {"command": "/listener/setHRTF", "parameters":
["DefaultListener", "HRTF1"]},
  {"command": "/listener/setSOSFilters", "parameters":
["DefaultListener", "DefaultNFFilters"]},
  {"command": "/listener/setBRIR", "parameters":
["DefaultListener", "BRIR1"]}
]
```

3.2.5 Calibration

BeRTA provides a calibration procedure that establishes a relationship between digital signal levels in dBFS and measured sound pressure levels in dBSPL.

Calibration is associated with a **physical stereo output pair** of the selected audio interface. Each pair stores its own calibration reference, independently of the listener selected in the GUI. Once a pair is calibrated, BeRTA can use this reference to estimate sound pressure levels from digital signal levels. These values are estimates based on the playback chain used during calibration, not continuous acoustic measurements.

Calibration does not automatically adjust the playback hardware or guarantee a particular perceived loudness.

Output pairs and calibration references

Output channels are numbered from **0** and grouped into stereo pairs:

| Left channel argument | Output pair |
|-----------------------|-------------|
| 0 | 0-1 |
| 2 | 2-3 |
| 4 | 4-5 |

Calibration commands require the left channel of an available pair. The value must be a non-negative even number, and both channels must be enabled in **Settings → Audio Interfaces**.

The requested pair is used independently of the listener selected in the GUI. A pair can be calibrated even when no listener is assigned to it.

BeRTA currently stores **one calibration relationship per stereo pair**, shared by its left and right channels. It does not store independent calibration offsets for the two sides.

Calibration relationship

A calibration reference contains:

- The digital RMS level used to reproduce the reference signal, in dBFS.
- The corresponding sound pressure level measured externally, in dBSPL.

BeRTA calculates the calibration offset as:

$$\text{calibration offset} = \text{reference dBSPL} - \text{reference dBFS}$$

During normal operation, the estimated sound pressure level is:

$$\text{estimated dBSPL} = \text{signal dBFS} + \text{calibration offset}$$

For example, if a reference reproduced at **−40 dBFS** produces a measured level of **60 dBSPL**:

$$\text{calibration offset} = 60 - (-40) = 100 \text{ dB}$$

A subsequent signal at **−30 dBFS** therefore corresponds to an estimated level of **70 dBSPL**, before any physical output limiting is applied.

Calibration process in BeRTA

The procedure consists of reproducing a reference signal, measuring its acoustic output, storing the relationship and optionally verifying it.

Choose the output device, enabled channels and playback hardware before starting. Keep the hardware gain and volume settings unchanged throughout calibration and subsequent use.

1. PLAY THE REFERENCE SIGNAL

Send `/control/playCalibration` with the desired digital RMS level and the left channel of the output pair.

For example:

```
/control/playCalibration -40 0
```

This starts the reference signal at **−40 dBFS** on output channels **0-1**.

BeRTA plays `resources/Calibration/CalibrationPinkNoise.wav` in a continuous loop. It reads the file's left channel and sends identical samples to both channels of the selected output pair. Other output channels remain silent.

When loading the reference file, BeRTA:

- Checks that its sample rate matches the application's configured sample rate.
- Calculates the RMS of its left channel to determine the required playback gain.
- Checks that the requested gain will not produce digital clipping.

If these checks fail, the playback request is rejected.

Reference signal

The reference is reproduced directly through the selected output pair. It does not pass through listener spatialisation or the normal scene-processing chain.

2. MEASURE THE ACOUSTIC OUTPUT

Measure the reference signal using a sound level meter and a measurement arrangement appropriate to the playback transducer.

Record the measured sound pressure level and the digital level used in the previous step. Keep the measurement settings and physical arrangement consistent when verifying the calibration. For headphones, the microphone placement and coupling to the earpiece affect the result. The calibration reference represents the conditions under which that measurement was made.

Shared calibration for both channels

Both channels receive identical digital samples, but the corresponding acoustic levels may differ because of the playback hardware.

BeRTA applies one calibration offset to both channels of the pair. A measurement taken from one side does not independently calibrate or compensate the other side. Check both sides when their agreement matters to the application.

3. STORE THE CALIBRATION

Send `/control/setCalibration` with:

1. The digital RMS level used for the reference.
2. The measured sound pressure level.
3. The left channel of the same output pair.

For example, if the reference played at -40 dBFS was measured at 60 dB SPL:

```
/control/setCalibration -40 60 0
```

BeRTA stores this relationship for channels **0-1**, stops calibration playback and exits calibration mode.

Starting or stopping reference playback does not replace an existing calibration. The stored values change when `/control/setCalibration` is applied.

4. VERIFY THE CALIBRATION

Once a calibration is available, use `/control/playCalibrationTest` to request a reference level in dB SPL:

```
/control/playCalibrationTest 60 0
```

BeRTA converts the requested level to dBFS using the stored calibration and reproduces the reference signal on channels **0-1**.

Measure the acoustic output externally and compare it with the requested level.

The command requires an existing calibration for the selected pair. A requested level that would cause digital clipping is rejected.

5. STOP THE TEST

To stop reference or verification playback, send the left channel of the pair currently being tested:

```
/control/stopCalibrationTest 0
```

The GUI Stop control and the space bar also stop calibration playback.

Stopping the test preserves the stored calibration. Normal scene playback does not resume automatically.

For argument types and response formats, see the [OSC calibration commands](#).

Command acknowledgement

A successful OSC response confirms that BeRTA accepted and performed the software operation. It does not confirm the accuracy of the external acoustic measurement.

Behaviour during calibration

Calibration playback is an exclusive operating mode:

- Only one output pair can be tested at a time.
- Starting reference or verification playback stops normal scene playback.
- New playback and recording operations are blocked until calibration mode ends.
- Calibration playback cannot start while a recording is in progress.
- A new calibration playback request interrupts the previous test. If the new request fails, the previous test is not resumed.
- Applying `/control/setCalibration` ends calibration playback.
- Stopping calibration does not automatically resume the scene.

SAFETY LIMITER AND MEASUREMENTS

The normal scene-processing path is bypassed during both reference playback and verification playback.

Consequently:

- The physical safety limiter does not act on the calibration signal.
- Normal output-level measurements are paused.
- The output VUmeters do not display the calibration signal.
- Listener measurements do not represent the calibration signal.

The reference-file RMS calculation performed when loading the WAV is used to set playback gain. It is separate from the normal live measurements, which remain paused during calibration.



Calibration playback bypasses the safety limiter

The sound-level limit configured for normal scene playback does not constrain reference or verification playback.

Choose the requested test level deliberately. BeRTA's digital clipping check does not establish an acoustic exposure limit.

The existing calibration data remain stored while a test is running. They are replaced only when new calibration values are applied.

How listeners use calibration

Calibration belongs to an output pair. Listeners obtain the calibration used for their theoretical dBSPL measurements through their output configuration.

| Listener configuration | Calibration used |
|--------------------------------|---|
| Routed, Active | Its assigned output pair |
| Routed, Standby | Its assigned output pair |
| Virtual with a valid reference | The current output pair of the referenced Routed listener |
| Virtual without a reference | No calibration available |

For example:

Virtual listener B → Routed listener A → output pair 0-1 → calibration

Listener B uses **its own digital signal level** with the calibration of channels 0-1. It does not use Listener A's signal level.

The reference follows Listener A. If A is reassigned to channels 2-3, B uses the calibration of channels 2-3 after the routing change is applied.

A may be in Active or Standby: its routing state does not affect its ability to provide a calibration reference.

Virtual listeners without a valid calibration reference still provide digital RMS measurements in dBFS. Their theoretical dBSPL measurements are unavailable.

These references are configured in [Settings → Listeners](#).

Monitoring sound levels

BeRTA distinguishes between digital output metering, estimated physical output levels and listener signal levels.

OUTPUT VUMETERS

The multichannel output VUmeters display the digital envelope level of each enabled output channel, in dBFS.

- The measurement is taken after the physical safety limiter.
- Digital metering does not require calibration.
- The meters are independent of the listener selected in the GUI.
- The envelope detector provides the meter's attack and release behaviour; it is not the same measurement as listener RMS.

During calibration playback, the meters show a paused indication instead of measuring the reference signal.

OUTPUT STATUS

The **Output pairs - dBSPL** section displays one row per physical output pair:

| Column | Meaning |
|--------------|---|
| Pair | Physical output pair, such as 0-1 |
| Cal | OK when calibration is available, or No otherwise |
| Level | Estimated left / right output levels in dBSPL, after limiting |

Amber levels followed by * indicate that the safety limiter is acting. The numeric levels remain visible.

Hover over a pair for its calibration reference, offset and available limiter information. When limiting is active, the tooltip also includes pre-limiter levels.

The display distinguishes:

- `-inf` : a valid silent signal.
- `--` : an unavailable measurement.
- `TEST` : calibration playback is active on this pair.
- `Paused` : calibration playback is active on another pair and normal measurements are paused.

Calibration availability and measurement availability are separate. A calibrated pair may temporarily have no current measurement. A known silent output may be reported as silence even without calibration.

OSC QUERIES

The former `/control/getSoundLevel` command has been removed. Use the following queries:

| Command | Measurement | Calibration required | Includes physical limiting |
|---|---|--------------------------------------|----------------------------|
| <code>/control/getOutputSoundLeveldB SPL</code> | Estimated physical output level for a stereo pair | Required for a non-silent signal | Yes |
| <code>/control/getListenerSoundLeveldB SPL</code> | Theoretical level calculated from the listener's own RMS signal level | Yes, directly or through a reference | No |
| <code>/control/getListenerSoundLeveldB FS</code> | The listener's own digital RMS level | No | No |

The Active/Standby state does not affect a listener's own RMS measurement or its theoretical dB SPL calculation.

A listener's theoretical dB SPL level may exceed the physical safety limit. The limiter changes the signal sent to the physical output, not the listener's theoretical measurement.

Each query response includes the requested identifier, a `valid` flag, left and right levels, and an error string.

- `valid=true` indicates an available measurement. A silent channel may have a level of negative infinity.
- `valid=false` indicates an unavailable measurement. Inspect the error string rather than interpreting the numeric fields as silence.

During calibration playback, the output-level query for the pair being tested reports that the measurement is unavailable.

See [OSC sound-level commands](#) for complete syntax and examples.

Interpretation and limitations

BeRTA's dB SPL values are estimates derived from a digital signal measurement and a stored calibration offset. They are not live acoustic measurements and do not directly quantify perceived loudness.

Their agreement with an external measurement depends on factors including:

- **Playback gain:** Changes to interface volume, amplifier alter the relationship between digital and acoustic levels. Replacing the interface, amplifier, or speakers may require a new calibration. A single calibration offset does not compensate for room-dependent differences between digital and other audio.
- **Measurement location:** Meter weighting, microphone placement, headphone coupling and the distance to the reference measurement.
- **Signal processing:** Clipping, saturation and nonlinearities in the signal chain can invalidate the relationship between digital and acoustic levels.
- **Calibration scope:** A shared stereo-pair calibration cannot compensate differences between

Keep the playback configuration consistent with the conditions used for calibration. Recheck the acoustic output after relevant hardware, gain or measurement changes.

Calibration provides a reproducible level reference within those conditions. External measurements remain necessary to establish and verify that reference.

3.2.6 Safety Limiter

Importance of Setting a Maximum Signal Level

In spatial audio applications, ensuring safe listening levels is crucial to preventing hearing damage and avoiding unintended excessive sound levels. Prolonged exposure to high sound pressure levels (SPL) can cause hearing fatigue or even permanent hearing loss. Additionally, sudden loud sounds can be uncomfortable or startling for users, potentially compromising the immersive experience. To mitigate these risks, BeRTA includes a **Safety Limiter** that allows users to define a maximum SPL threshold, preventing output levels from exceeding safe limits.

Functionality

Once the calibration process has been completed, users can configure a **safety control** in BeRTA. This control continuously monitors the signal level in real time. If the

signal exceeds the predefined safety threshold, BeRTA automatically limits it to the maximum allowed level, displays a warning in the graphical interface, and sends an alert command to notify connected clients.

The safety threshold is set using the **setSoundLevelLimit** OSC command, where the user specifies the maximum allowed SPL level. For further details, refer to the [OSC Commands](#) section.

When the signal surpasses the limit, BeRTA sends an alert message via the **soundLevelAlert** OSC command to all clients that have established an OSC connection with the BeRTA Renderer. This ensures that external applications are immediately informed of potential safety violations.

By implementing the Safety Limiter, BeRTA helps maintain a controlled and safe listening environment, protecting users while ensuring consistent playback levels across different use cases.

3.3 OSC Commands

3.3.1 OSC commands defined in BRT

Part of the BRT components is the definition of a set of OSC commands which the BRT Renderer application understands to dynamically configure the virtual scene. This is a complete list of all commands:

OSC CONTROL COMMANDS

- `/control/actionResult` : Retrieve the result of the action performed in another application after receiving an OSC command.
- `/control/bufferFrames` : Query the number of buffered audio frames.
- `/control/connect` : Establish a connection with an IP and port.
- `/control/disconnect` : Terminate the connection and unsubscribe from updates.
- `/control/frameSize` : Request the audio frame size per channel.
- `/control/ping` : Check if BeRTA is listening by sending an echo.
- `/control/sampleRate` : Request the current audio sample rate.
- `/control/version` : Retrieve the current BeRTA version.

Listener output configuration

- `/control/setListenerOutput` : Configure an existing listener's output assignment.

Calibration

- `/control/playCalibration` : Start the calibration process.
- `/control/setCalibration` : Set the calibration values.
- `/control/playCalibrationTest` : Play the calibration test sound at a specified volume.
- `/control/stopCalibrationTest` : Stop the calibration test sound playback.

Sound levels

- `/control/getOutputSoundLeveldB SPL` : Get the theoretical sound level in dB SPL for a physical stereo output pair.
- `/control/getListenerSoundLeveldB SPL` : Get the theoretical sound level in dB SPL for a listener.
- `/control/getListenerSoundLeveldB FS` : Get the digital signal level in dB FS for a listener's left and right channels.

Safety Limiter

- `/control/setSoundLevelLimit` : Set the sound level limit to a specified volume.
- `/control/soundLevelAlert` : Alert sent when the output sound level exceeds the threshold set in the limiter.

OSC OVERALL COMMANDS

- `/play` : Starts playback of file sources and streaming from input channels.
- `/pause` : Pauses file sources and stops streaming from input channels.
- `/stop` : Stops playback of file sources and streaming from input channels.
- `/removeAllSources` : Removes all sound sources.
- `/playAndRecord` : Records spatialized sound and data during playback.
- `/record` : Records spatialised sound and data without real-time playback.
- `/enableModel` : Enables or disables a model.
- `/modelGain` : Sets the output gain of the model in dB.
- `/recordIR` : Record the system's response to the stimulus as it is configured at that moment.

RESOURCES COMMANDS

- `/resources/loadHRTF` : Loads a new HRTF, with interpolation, from a SOFA file and assigns an identifier.
- `/resources/loadHRTFRaw` : Loads a new HRTF, without interpolation, from a SOFA file and assigns an identifier.
- `/resources/removeHRTF` : Removes an HRTF from the loaded resources.
- `/resources/getHRTFInfo` : Gets information about one of the loaded HRTFs.
- `/resources/setHRTFHeadRadius` : Sets the head radius to be stored in the HRTF.
- `/resources/getHRTFHeadRadius` : Retrieves the head radius of one of the loaded HRTFs.
- `/resources/restoreHRTFHeadRadius` : Sets the HRTF head radius to the value stored in the SOFA file.
- `/resources/enableWoodworthITD` : Enables or disables the calculation of the ITD using the Woodworth formula.
- `/resources/loadBRIR` : Loads a new Binaural Room Impulse Response from a SOFA file.
- `/resources/removeBRIR` : Removes a Binaural Room Impulse Response from the loaded resources.

- `/resources/getBRIRInfo` : Gets information about one of the loaded Binaural Room Impulse Responses (BRIR).
- `/resources/loadDirectivity` : Loads a Directivity Transfer Function from a SOFA file.
- `/resources/loadDirectivityRaw` : Loads a Directivity Transfer Function without interpolation from a SOFA file.
- `/resources/removeDirectivity` : Removes a Directivity Transfer Function from the loaded resources.
- `/resources/getDirectivityInfo` : Gets information about one of the loaded Directivity Transfer Functions.
- `/resources/loadDirectivityTF` : Loads a Directivity Transfer Function from a SOFA file. *(Removed, use '/resources/loadDirectivity' instead.)*
- `/resources/removeDirectivityTF` : Removes a Directivity Transfer Function from the loaded resources. *(Removed, use '/resources/removeDirectivity' instead.)*
- `/resources/getDirectivityTFInfo` : Gets information about one of the loaded Directivity Transfer Functions. *(Removed, use '/resources/getDirectivityInfo' instead.)*
- `/resources/loadFilter` : Load a new set of filter coefficients (IIR or FIR) from a SOFA file.
- `/resources/removeFilter` : Removes a set of filter coefficients from the loaded resources.
- `/resources/getFilterInfo` : Gets some information about one of the loaded set of filters coefficients.
- `/resources/loadSOSFilters` : Loads a set of SOS filters from a SOFA file. *(Deprecated, use '/resources/loadFilter' instead.)*
- `/resources/removeSOSFilters` : Removes a set of SOS filters from the loaded resources. *(Deprecated, use '/resources/removeFilter' instead.)*
- `/resources/getSOSFiltersInfo` : Gets information about one of the loaded sets of SOS filters. *(Deprecated, use '/resources/getFilterInfo' instead.)*
- `/resources/loadRoom` : Loads a new room from a OBJ file
- `/resources/loadShoeBoxRoom` : Set up a shoebox-shaped room with specified dimensions.
- `/resources/removeRoom` : Removes a loaded room resource.
- `/resources/setRoomWallAbsorption` : Sets absorption coefficients for the walls of a room resource.
- `/resources/enableRoomWall` : Enables or disables specific room walls in a room resource.
- `/source/location` : Set the global location of a sound source.
- `/source/loadSource` : Loads a new sound file and assigns an identifier.
- `/source/loop` : Set loop mode for a sound source.
- `/source/mute` : Mutes a specific sound source.
- `/source/orientation` : Set the orientation (yaw, pitch, roll) of a sound source.
- `/source/pause` : Pauses a specific sound source.
- `/source/play` : Plays a specific sound source.
- `/source/playAndRecord` : Record a file of specified duration with the source's audio and spatial information.
- `/source/record` : Record a file of specified duration with the source's audio and spatial information without real-time playback.
- `/source/removeSource` : Removes a sound source from the system.
- `/source/setDirectivity` : Assign a directivity to a sound source.
- `/source/solo` : Mutes all sources except the specified one.
- `/source/stop` : Stops a specific sound source.
- `/source/unmute` : Unmutes a previously muted sound source.
- `/source/unsolo` : Unmute all sources except the specified one.

SOURCE MODELS COMMANDS

- `/source/addLineIn` : Adds a new source from an audio input channel.
- `/source/enableDirectivity` : Enable or disable the directivity of a sound source.
- `/source/gain` : Set the gain of a sound source in dB.

LISTENER MODELS COMMANDS

- `/listener/enableInterpolation` : Enable or disable interpolation among HRIRs.
- `/listener/enableITD` : Enable or disable the simulation of Interaural Time Difference (ITD).
- `/listener/enableNearFieldEffect` : Enable or disable Near Field Compensation (NFC) with HRTF.
- `/listener/enableParallaxCorrection` : Enable or disable parallax correction for direction of arrival.
- `/listener/enableSpatialization` : Enables or disables listener spatialization processing.
- `/listener/location` : Set the global location of the listener in x, y, z coordinates.
- `/listener/orientation` : Set the orientation of the listener in yaw, pitch, roll.
- `/listener/setAmbisonicsNormalization` : Set the Ambisonics normalization type.
- `/listener/setAmbisonicsOrder` : Set the Ambisonic encoding order.
- `/listener/setBRIR` : Set a Binaural Room Impulse Response (BRIR) for the listener.

- `/listener/enableDistanceAttenuation` : Enable or disable distance attenuation for listener models that support it.
- `/listener/setDistanceAttenuationFactor` : Set the attenuation factor used for distance attenuation.
- `/listener/setHRTF` : Set a Head-Related Transfer Function (HRTF) for the listener.
- `/listener/setSOSFilters` : Set filters for Near Field Compensation (NFC) after HRTF convolution.

ENVIRONMENT MODELS COMMANDS

- `/environment/enableDirectPath` : Toggle the direct path source in the environment model.
- `/environment/enableReverbPath` : Toggle the reverb path sources in the environment model.
- `/environment/enablePropagationDelay` : Enable or disable propagation delay.
- `/environment/enableDistanceAttenuation` : Enable or disable distance attenuation.

- `/environment/setDistanceAttenuationFactor` : Establish the attenuation factor, which is used for calculating attenuation.
- `/environment/setRoom` : Set the room resource to be used by the environment model.
- `/environment/setReflectionOrder` : Set the order of reflection used in the simulation.
- `/environment/setMaxDistanceSourcesToListener` : Set the maximum distance between the sources and the listener.
- `/environment/setFadeZoneMargin` : Set the size of the fade zone used by some models for simulation.

BILATERAL FILTERS COMMANDS

- `/binauralFilter/setSOSFilter` : Set up sos filter for an specific listener. (*Deprecated, use '/bilateralFilter/setFilter' instead.*)
- `/bilateralFilter/setFilter` : Set up the filter coefficient database.

3.3.2 Control commands

The OSC control commands allow to establish and manage connections with BeRTA over the network or locally. Below are detailed the basic commands for initiating and closing connections, as well as obtaining system and audio configuration information. Each command is designed to provide immediate feedback.

/CONTROL/CONNECT

Open an OSC connection to the indicated IP and port to send the information back. After opening the communicating the following response is sent back: same command with the IP and listening port of BeRTA. Once the connection is stablished, the sender is considered as a suscriber to all the updates which berta sends back.

Syntax

```
/control/connect <string ip> <int port>
```

ip : IP address of the sender which BeRTA will use to send back messages. Use `localhost` if it is running in the same machine

port : OSC port in which the sender is listening for reply and update messages. BeRTA will send all replies and update messages to this port.

Return

A message is sent back to the sender indicating IP and listening port of BeRTA: `/control/connect <string ip> <int port>`. Although this information is already known by the sender, this is away of acknowledging that the connection is stablished and the sender is subscribed to updates.

Example

BeRTA receives: `/control/connect localhost 10011`.

BeRTA sends back to the sender:
`/control/connect localhost 10017`.

/CONTROL/DISCONNECT

Stops the communication through which BeRTA sent information back. This is equivalent to unsibscribe the sender from updates sent back by BrRTA. When this message is received the same message is sent back.

Syntax

```
/control/disconnect
```

Return

An echo is sent back to the sender to confirm the reception of the command: `/control/disconnect`.

Example

BeRTA receives: `/control/disconnect`.

BeRTA sends back to the sender: `/control/disconnect`.

/CONTROL/PING

Responds with the same message to indicate that BeRTA is listening to the external app.

Syntax

```
/control/ping
```

Return

An echo is sent back to the sender to confirm the reception of the command: `/control/ping`.

Example

BeRTA receives: `/control/ping`.

BeRTA sends back to the sender: `/control/ping`.

/CONTROL/VERSION

Responds with a message to indicate the version of BeRTA.

Syntax

```
/control/version
```

Return

An echo is sent back to the sender including the version of BeRTA: `/control/version <string version>`.

Example

BeRTA receives: `/control/version`.

BeRTA sends back to the sender: `/control/version BeRTA-Renderer v3.0.0`.

/CONTROL/SAMPLERATE

Asks for the sample rate which BeRTA is using. This parameter is defined in the used [settings file](#).

Syntax

```
/control/sampleRate
```

Return

An echo is sent back to the sender including the sample rate: `/control/sampleRate <int sample_rate>`.

`sample_rate` : Sample rate used by BeRTA in samples per second.

Example

BeRTA receives: `/control/sampleRate`
BeRTA sends back to the sender: `/control/sampleRate 48000`.

`/CONTROL/FRAMESIZE`

Asks for the size of the audio frame which BeRTA is using. This parameter is defined in the used [settings file](#).

Syntax

`/control/frameSize`

Return

An echo is sent back to the sender including the frame size:
`/control/frameSize <int frame_size>`.

`frame_size` : size of the audio frame in bytes per channel (BeRTA produces a stereo output for each listener. This parameter indicated the size of the frame of only one channel).

Example

BeRTA receives: `/control/frameSize`
BeRTA sends back to the sender: `/control/frameSize 512`.

`/CONTROL/BUFFERFRAMES`

Asks for the number of audio frames that are stored in the buffer to prevent audio dropout or system failure due to frame underrun. Since the operating system is not real-time, there can be instances where it prioritizes other tasks, potentially delaying the provision of a new audio frame. By setting an adequate buffer size, the system ensures continuous audio playback, even if the operating system momentarily allocates resources to higher-priority processes. This buffer acts as a safeguard against timing inconsistencies, allowing the audio processing thread to retrieve preloaded frames, maintaining uninterrupted audio output. However, as a trade-off, increasing the number of buffered frames results in higher system latency. To minimize latency, this parameter should be set to 0, which eliminates buffering but requires the system to deliver new audio frames in real time without delay. This parameter is defined in the used [settings file](#).

Syntax

`/control/bufferFrames`

Return

An echo is sent back to the sender including the number of frames buffered: `/control/bufferFrames <int buffer_frames>`.

`buffer_frames` : number of buffered frames.

Example

BeRTA receives: `/control/bufferFrames`
BeRTA sends back to the sender: `/control/bufferFrames 2`.

`/CONTROL/ACTIONRESULT`

Command received when an action has been carried out from the application (triggered by a previously received OSC command). It informs about the outcome of the executed action.

Syntax

`/control/actionResult <string actionCommand> <string id> <bool success> <string description>`

Example

BeRTA receives:
`/control/actionResult /resources/removeHRTF HRTF1 true HRTF1 removed`

Listener output configuration

`/CONTROL/SETLISTENEROUTPUT`

Available from BeRTA v3.15.0

Configure an existing listener's output assignment, Active/Standby state and calibration reference.

This command provides the output-routing configuration available in the Listeners tab of the Settings GUI. It does not create a listener. The listener type is determined by its output assignment: a non-negative channel selects a physical output pair, while `-1` makes the listener virtual.

Syntax

`/control/setListenerOutput <string listener_id> <int leftOutputChannel> <bool outputActive> <string calibrationReferenceListenerID>`

All four arguments are required, including an empty string when no calibration reference is used.

`listener_id` : Identifier of the existing listener to configure.

`leftOutputChannel` : Zero-based left channel of an available physical stereo output pair. It must be a non-negative even number, such as `0` for channels 0-1 or `2` for channels 2-3. Use `-1` for a virtual listener.

`outputActive` : For a listener with a physical output assignment, `true` selects Active and `false` selects Standby. Virtual listeners must use `false`. This controls physical output routing, not whether the listener's signal levels are calculated. Send this argument as an OSC boolean.

`calibrationReferenceListenerID` : For a virtual listener, the ID of another listener with a direct output assignment, or `""` for no reference. A listener with a physical output assignment must use `""`.

Behaviour

- Activating a listener automatically puts other listeners assigned to the same output pair into Standby.
- Listeners assigned to different output pairs may be active simultaneously.
- A virtual listener may reference a routed listener in either Active or Standby.
- A virtual listener cannot reference itself or another virtual listener.
- Calibration references follow the referenced listener's current output assignment.
- Converting a routed listener into a virtual listener automatically clears references from other listeners that depended on it.
- An output pair does not need to be calibrated to configure its routing. Calibration is required to obtain a listener's theoretical dB SPL level.
- The resulting routing configuration is validated before it is applied. If validation fails, the current routing configuration is preserved.

Return

```
/control/actionResult /control/setListenerOutput <string  
listener_id> <bool success> <string description>
```

`listener_id` : Listener being configured. If the request cannot provide a usable listener ID, the response uses `listenerOutput`.

`success` : `true` if the configuration was applied; otherwise, `false`.

`description` : Description of the applied configuration or the reason for failure.

Examples

- Assign Listener A to channels 0-1 and activate its output:
BeRTA receives: `/control/setListenerOutput A 0 true ""`
BeRTA sends: `/control/actionResult /control/
setListenerOutput A true "Listener A routed to output
channels 0-1 (Active)."`
- Keep the assignment but put Listener A into Standby:

BeRTA receives: `/control/setListenerOutput A 0 false ""`

BeRTA sends: `/control/actionResult /control/
setListenerOutput A true "Listener A routed to output
channels 0-1 (Standby)."`

- Configure Listener B as virtual, using Listener A as its calibration reference:

BeRTA receives: `/control/setListenerOutput B -1 false A`

BeRTA sends: `/control/actionResult /control/
setListenerOutput B true "Listener B configured as
Virtual with A as calibration reference."`

- Remove Listener B's calibration reference while keeping it virtual:

BeRTA receives: `/control/setListenerOutput B -1 false ""`

BeRTA sends: `/control/actionResult /control/
setListenerOutput B true "Listener B configured as
Virtual without a calibration reference."`

- Attempt to activate a virtual listener:

BeRTA receives: `/control/setListenerOutput B -1 true A`

BeRTA sends: `/control/actionResult /control/
setListenerOutput B false "ERROR: Virtual listener B
cannot have an active physical output."`

Calibration

/CONTROL/PLAYCALIBRATION

Available since BeRTA v3.4.0

Updated in BeRTA v3.15.0

 Changes in BeRTA v3.15.0

The `channelNumber` argument is now required and explicitly selects the physical stereo output pair. The command no longer falls back to the first configured listener.

Calibration playback now uses the selected audio interface and enters an exclusive mode: normal playback is stopped, and new playback or recording cannot start until calibration ends. The command is rejected if recording is already in progress.

Start the calibration process by playing an audio file at the dBFS volume specified in the parameter. For more details, refer to the [calibration](#) section.

Syntax

```
/control/playCalibration <float leveldBFS> <int  
channelNumber>
```

`leveldBFS` : The signal level at which the calibration audio was played, measured in dBFS.

`channelNumber` : Available from BeRTA v3.7.0 This indicates the output channels of the audio interface to which the

command applies. This value must always be an even number. For example, if channel 0 is selected, channels 0-1 are affected; if 2 is selected, channels 2-3 are affected. This argument is required. The command applies to the specified output pair independently of the listener selected in the GUI.

Return

```
/control/actionResult /control/playCalibration <string  
actionName> <boolean success> <string description>.
```

The return confirmation refers to the action `calibration`, indicating `success=true` if the calibration has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

Example

```
BeRTA receives: /control/playCalibration -40 0  
  
BeRTA sends back to the sender: /control/actionResult /  
control/playCalibration calibration true "Calibration sound  
played".
```

/CONTROL/SETCALIBRATION

Available since BeRTA v3.4.0 Updated in BeRTA v3.15.0

Changes in BeRTA v3.15.0

The `channelNumber` argument is now required and explicitly selects the physical stereo output pair. The command no longer falls back to the first configured listener.

Set the calibration values using the dBFS playback volume and the dB SPL output level measured in the headphones, both specified in the parameters. For more details, refer to the [calibration](#) section.

Syntax

```
/control/setCalibration <float leveldBFS> <float leveldB SPL>  
<int channelNumber>
```

`leveldBFS` : The signal level at which the calibration audio was played, measured in dBFS.

`leveldB SPL` : The output sound level measured at the headphones (expressed in dB SPL) when the system is calibrated and delivering the dBFS specified in the `leveldBFS` parameter.

`channelNumber` : Available from BeRTA v3.7.0 This indicates the output channels of the audio interface to which the command applies. This value must always be an even number. For example, if channel 0 is selected, channels 0-1 are affected; if 2 is selected, channels 2-3 are affected. This argument is required. The command applies to the specified

output pair independently of the listener selected in the GUI.

Return

```
/control/actionResult /control/setCalibration <string  
actionName> <boolean success> <string description>.
```

The return confirmation refers to the action `calibrationDone`, indicating `success=true` if the calibration has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

Example

```
BeRTA receives: /control/setCalibration -40 60 0  
  
BeRTA sends back to the sender: /control/actionResult /  
control/setCalibration calibrationDone true "Calibration  
done with -40 dB FS and 60 dB SPL".
```

/CONTROL/PLAYCALIBRATIONTEST

Available since BeRTA v3.4.0 Updated in BeRTA v3.15.0

Changes in BeRTA v3.15.0

The `channelNumber` argument is now required and explicitly selects the physical stereo output pair. The command no longer falls back to the first configured listener.

Calibration playback now uses the selected audio interface and enters an exclusive mode: normal playback is stopped, and new playback or recording cannot start until calibration ends. The command is rejected if recording is already in progress.

Play the calibration test sound at a volume adjusted to match the dB SPL specified in the parameter, allowing verification with the sound level meter. See the [calibration](#) section for more details.

Syntax

```
/control/playCalibrationTest <float leveldB SPL> <int  
channelNumber>
```

`leveldB SPL` : The signal level (in dB SPL) measured at the headphones when reproducing the calibration audio.

`channelNumber` : Available from BeRTA v3.7.0 This indicates the output channels of the audio interface to which the command applies. This value must always be an even number. For example, if channel 0 is selected, channels 0-1 are affected; if 2 is selected, channels 2-3 are affected. This argument is required. The command applies to the specified output pair independently of the listener selected in the GUI.

Return

```
/control/actionResult /control/playCalibrationTest <string  
actionName> <boolean success> <string description>.
```

The return confirmation refers to the action `calibrationTest`, indicating `success=true` if the calibration has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

Example

```
BeRTA receives: /control/playCalibrationTest 80 2
```

```
BeRTA sends back to the sender: /control/actionResult /  
control/playCalibrationTest calibrationTest false "ERROR  
BeRTA has not been calibrated yet".
```

/CONTROL/STOPCALIBRATIONTEST

Available since BeRTA v3.4.0 Updated in BeRTA v3.15.0

Changes in BeRTA v3.15.0

The `channelNumber` argument is now required and explicitly selects the physical stereo output pair. The command no longer falls back to the first configured listener.

Stop the calibration test sound playback.

Syntax

```
/control/stopCalibrationTest <int channelNumber>
```

`channelNumber` : Available from BeRTA v3.7.0 This indicates the output channels of the audio interface to which the command applies. This value must always be an even number. For example, if channel 0 is selected, channels 0-1 are affected; if 2 is selected, channels 2-3 are affected. This argument is required. The command applies to the specified output pair independently of the listener selected in the GUI.

Return

```
/control/actionResult /control/stopCalibrationTest <string  
actionName> <boolean success> <string description>.
```

The return confirmation refers to the action `calibrationTest`, indicating `success=true` if the stop of the calibration has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

Example

```
BeRTA receives: /control/stopCalibrationTest 0
```

```
BeRTA sends back to the sender: /control/actionResult /  
control/stopCalibrationTest calibrationTest true  
"Calibration test stopped.".
```

Sound levels

The following commands distinguish between the estimated physical output level after safety limiting, the theoretical calibrated level of a listener, and the digital signal level of a listener.

They replace the removed `/control/getSoundLevel` command.

Each query returns the requested output channel or listener ID, followed by `valid`, the left and right levels, and an `error` string.

A valid silent signal is represented by `valid=true` and negative infinity for the corresponding level. An unavailable measurement is represented by `valid=false`, negative infinity for both levels, and an explanatory error message.

In the examples below, `-inf` represents a floating-point negative infinity value, not an OSC string. `" "` represents an empty OSC string.

/CONTROL/GETOUTPUTSOUNDLEVELDBSPL

Available from BeRTA v3.15.0

Get the latest estimated sound level in dB SPL for a physical stereo output pair, after applying the safety limiter. The estimate uses the output calibration and the processed digital signal. It is not a live acoustic measurement from a sound level meter.

This command queries the output pair independently of the listener selected in the GUI.

Syntax

```
/control/getOutputSoundLeveldB SPL <int leftOutputChannel>
```

`leftOutputChannel` : Zero-based index of the left output channel. It must be a non-negative even number identifying an available stereo pair in the selected audio interface. For example, `0` selects channels 0-1 and `2` selects channels 2-3.

Return

```
/control/getOutputSoundLeveldB SPL <int leftOutputChannel>  
<bool valid> <float leftChannelSoundLevel> <float  
rightChannelSoundLevel> <string error>
```

`leftOutputChannel` : Output channel requested.

`valid: true` if a current output measurement is available; otherwise, `false`.

`leftChannelSoundLevel`, `rightChannelSoundLevel`: Estimated left and right output levels in dB SPL, including the effect of the safety limiter.

`error`: Empty when `valid=true`; otherwise, describes why the measurement is unavailable.

When a valid silence measurement has been published for the output pair, both levels are `-inf`. This includes output pairs receiving no signal, and does not require calibration. If a signal is being processed but the output pair is not calibrated, the measurement is invalid. Invalid output pairs and unavailable current measurements also return `valid=false`.

Examples

Assuming that BeRTA receives: `/control/getOutputSoundLeveldB SPL 0`. Then:

- For an uncalibrated output pair processing a signal:

BeRTA sends:

```
/control/getOutputSoundLeveldB SPL 0 false -inf -inf
"Output channels 0-1 have an active signal but are not calibrated."
```

- If channels 0-1 are calibrated, they have a calibration offset of 100 dB and their digital levels are -24.6 dBFS and -18.7 dBFS. And that the safety limiter is not configured or is not working:

BeRTA sends: `/control/getOutputSoundLeveldB SPL 0 true 75.4 81.3 ""`

- If channels 0-1 are calibrated, they have a calibration offset of 100 dB and their digital levels are -24.6 dBFS and -18.7 dBFS. And that the safety limiter is reducing their output to 80 dB SPL:

BeRTA sends: `/control/getOutputSoundLeveldB SPL 0 true 75.4 80 ""`

- For a valid silent output pair:

BeRTA sends: `/control/getOutputSoundLeveldB SPL 0 true -inf -inf ""`

/CONTROL/GETLISTENERSOUNLEVELDB SPL

Available from BeRTA v3.15.0

Get the latest theoretical sound level in dB SPL for a listener. The result is calculated from the listener's own digital signal level in dBFS and the calibration offset of its resolved output pair:

`listener level in dB SPL = listener level in dBFS + calibration offset in dB`

For a listener with a *direct output assignment*, its assigned pair provides the calibration. For a *virtual listener*, its calibration reference identifies another listener with a direct output assignment. The current output pair of that referenced listener provides the calibration. The reference follows the listener if its output assignment changes.

The safety limiter is *not applied* to this result. The Active/Standby state of either listener does not affect the calculation. Consequently, the theoretical level may exceed the physical safety limit.

Syntax

```
/control/getListenerSoundLeveldB SPL <string listener_id>
```

`listener_id`: Identifier of an existing listener.

Return

```
/control/getListenerSoundLeveldB SPL <string listener_id>
<bool valid> <float leftChannelSoundLevel> <float rightChannelSoundLevel> <string error>
```

`listener_id`: Listener requested.

`valid: true` if a current listener measurement and a valid calibration are available; otherwise, `false`.

`leftChannelSoundLevel`, `rightChannelSoundLevel`: Theoretical left and right levels in dB SPL, before physical output limiting.

`error`: Empty when `valid=true`; otherwise, describes why the measurement is unavailable.

The result is invalid if the listener does not exist, has no current digital measurement, has no direct output or valid calibration reference, or resolves to an unavailable or uncalibrated output pair.

A silent listener returns `valid=true` and `-inf` levels only when the required measurement and calibration are available.

Example

- Assuming:
- that Listener A is assigned to channels 0-1 in Active.
- that Channels 0-1 have a calibration offset of 100 dB.
- that Listener A has digital levels of -29 dBFS and -27 dBFS.

BeRTA receives: `/control/getListenerSoundLeveldB SPL B`

BeRTA sends:

```
/control/getListenerSoundLeveldB SPL B true 71 73 ""
```

- Assuming:
- that Listener A is assigned to channels 0-1 in Standby
- that Listener B is virtual and references Listener A.
- that Channels 0-1 have a calibration offset of 100 dB.
- that Listener B has digital levels of -30 dBFS and -28 dBFS.

BeRTA receives: `/control/getListenerSoundLeveldB SPL B`

BeRTA sends:

```
/control/getListenerSoundLeveldB SPL B true 70 72 ""
```

The result uses Listener B's signal and the calibration of channels 0-1. Listener A being in Standby does not affect it.

If Listener B has no calibration reference:

BeRTA sends:

```
/control/getListenerSoundLeveldB SPL B false -inf -inf
"Listener B has no direct output or valid calibration
reference."
```

/CONTROL/GETLISTENERSOUNLEVELDBFS

Available from BeRTA v3.15.0

Get the latest digital signal level in dBFS for a listener's left and right channels, before physical output limiting. This command applies to both routed and virtual listeners. It does not require an output assignment or calibration, and the listener's Active/Standby state does not affect the result.

Syntax

```
/control/getListenerSoundLeveldBFS <string listener_id>
```

listener_id: Identifier of an existing listener.

Return

```
/control/getListenerSoundLeveldBFS <string listener_id>
<bool valid> <float leftChannelSoundLevel> <float
rightChannelSoundLevel> <string error>
```

listener_id: Listener requested.

valid: `true` if a current digital measurement is available; otherwise, `false`.

leftChannelSoundLevel, **rightChannelSoundLevel**: Left and right digital signal levels in dBFS.

error: Empty when `valid=true`; otherwise, describes why the measurement is unavailable.

A valid silent signal returns `-inf` for the corresponding channel. A missing listener or an unavailable current measurement returns `valid=false`.

Example

- BeRTA receives: `/control/getListenerSoundLeveldBFS B`

BeRTA sends: `/control/getListenerSoundLeveldBFS B true -30 -28 ""`

- For a valid silent listener:

BeRTA sends:

```
/control/getListenerSoundLeveldBFS B true -inf -inf ""
```

- If the requested listener does not exist:

BeRTA receives: `/control/getListenerSoundLeveldBFS`

Unknown

BeRTA sends:

```
/control/getListenerSoundLeveldBFS Unknown false -inf -
inf "Listener Unknown does not exist."
```

Safety Limiter

/CONTROL/SETSOUNDLEVELLIMIT

Available from BeRTA v3.4.0

Set the sound level limit to the dB SPL value specified in the parameter. See the [safety limiter](#) for more details.

Syntax

```
/control/setSoundLevelLimit <float levelLimitdB SPL> <int
channelNumber>
```

levelLimitdB SPL: The maximum allowable sound level, in dB SPL, used by the limiter to ensure safe listening levels for the user.

channelNumber: Available from BeRTA v3.7.0 This indicates the output channels of the audio interface to which the command applies. This value must always be an even number. For example, if channel 0 is selected, channels 0-1 are affected; if 2 is selected, channels 2-3 are affected. This argument is required. The command applies to the specified output pair independently of the listener selected in the GUI.

Return

```
/control/actionResult /control/setSoundLevelLimit <string
actionName> <boolean success> <string description>.
```

The return confirmation refers to the action

`calibrationTest`, indicating `success=true` if the set has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

Example

BeRTA receives: `/control/setSoundLevelLimit 100 0`

BeRTA sends back to the sender: `/control/actionResult /control/setSoundLevelLimit calibrationTest false "ERROR`

Sound level limit has not been set because BeRTA has not been calibrated.".

/CONTROL/SOUNDLEVELALERT

Available from BeRTA v3.4.0

Sent when the output sound level exceeds the threshold set in the sound level limiter. This is a send-only command from BeRTA. It does not receive any parameters but triggers an

alert when the limit is surpassed. See the [safety limiter](#) for more details.

Syntax

/control/soundLevelAlert

Example

BeRTA sends: /control/soundLevelAlert 60 62

3.3.3 Overall commands

These commands affects all sound sources and therefore, they don't need the `/source` prefix.

/PLAY

Starts playing back all sources comming from audio files and starts streaming all sources comming form input channels.

Syntax

`/play`

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/play`.

/STOP

Stops playing back all sources comming from audio files and also stops streaming all sources comming form input channels.

Syntax

`/stop`

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/stop`.

/PAUSE

Pauses all sources comming from audio files and stops streaming all sources comming form input channels. While file sources will resume with a later play at the same point they were paused, for sources connected to an input channel the effect of `/pause` is identical to the effect of `/stop`.

Syntax

`/pause`

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/pause`.

/REMOVEALLSOURCES

Removes all sources.

Syntax

`/removeAllSources`

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/removeAllSources`.

/PLAYANDRECORD

Records a file of the specified duration with spatialised sound (wav) and other data (mat) corresponding to the playback of all sources from the beginning. If the type is mat, the produced file will follow the structure of the [AnnotatedReceiverAudio SOFA convention](#). Before starting the recording, all sources are stopped and played back since the begining.

Syntax

`/playAndRecord <String filename> <String type> <float time>`

`filename`: indicates the name of the file and must include the path, either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable. The extension will be added by the application if necessary. If there is a file with the same name, it'll not be overwritten, an ordinal number will be added to the end of the new file name.

`type`: indicates the extension of the file: **mat** (MATLAB binary data container) or **wav** (32-bit IEEE floating-point stereo).

`time`: indicates the duration of the recording in seconds. If time is -1 the recording will finish automatically when the source stops.

Return

```
/control/actionResult /playAndRecord <string filename>
<boolean success> <string description>.
```

The return confirmation refers to the `filename`, indicating `success=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

It sends an echo to all subscribers excepting the sender: `/playAndRecord <String filename> <String type> <float time>`

Example

BeRTA receives and echoes back to all subscribers but the sender: `/playAndRecord c:/tmp/file.mat mat 10`

When the recording is finished (in this example after 10 seconds), BeRTA sends back to all subscribers: `/control/actionResult /playAndRecord c:/tmp/file.mat true Recording completed. File saved : c:/tmp/file.mat` or `/control/actionResult /playAndRecord c:/tmp/file.mat false ERROR:Recording failed. File could not be created.`

/RECORD

Available from BeRTA v3.7.0

Generates a file of the specified duration with spatialised sound (wav) and other data (mat) by processing all sources from the beginning *without real-time playback*. This allows the operation to be executed significantly faster. If the type is mat, the produced file will follow the structure of the [AnnotatedReceiverAudio SOFA convention](#). All sources are stopped and processed from the beginning before the recording starts.

Syntax

```
/record <String filename> <String type> <float time>
```

`filename`: indicates the name of the file and must include the path, either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable. The extension will be added by the application if necessary. If there is a file with the same name, it'll not be overwritten—an ordinal number will be added to the end of the new file name.

`type`: indicates the extension of the file: **mat** (MATLAB binary data container) or **wav** (32-bit IEEE floating-point stereo).

`time`: indicates the duration of the recording in seconds.

This value must be greater than 0. Unlike

`/playAndRecord`, the special value `-1` is *not* allowed.

Return

```
/control/actionResult /record <string filename> <boolean
success> <string description>.
```

The return confirmation refers to the `filename`, indicating `success=true` if the action has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

It sends an echo to all subscribers except the sender: `/record <String filename> <String type> <float time>`

Example

BeRTA receives and echoes back to all subscribers except the sender: `/record c:/tmp/file.mat mat 10`

When the processing is finished (in this example after 10 seconds), BeRTA sends back to all subscribers: `/control/actionResult /record c:/tmp/file.mat true Recording completed. File saved : c:/tmp/file.mat` or `/control/actionResult /record c:/tmp/file.mat false ERROR:Recording failed. File could not be created.`

/ENABLEMODEL

Available from BeRTA v3.6.0

This command enables or disables a model. This function is implemented in all listener, environment models and in all binaural filters. The model to be enabled or disabled is identified by an identifier defined in the [configuration](#) file used. When a model is deactivated it does not process the input signal and provides:

- *Silence* on its output in the case of listener models.
- Let the signal *pass through unaltered* in the case of environment models and binaural filters.

Syntax

```
/enableModel <string model_id> <boolean enable>
```

`model_id`: identifier assigned to the model.

`enable`: If true (1), enables the model. If false (0), the model is disabled and its output will be silent.

Return

```
/control/actionResult /enableModel <string model_id> <boolean>
<string description>
```

The return confirmation refers to the `model_id`, indicating `enable=true` if the model has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/enableModel DirectPath true`

BeRTA sends back to the sender: `/control/actionResult /enableModel DirectPath true "Listener model DirectPath enabled."`

/MODELGAIN

Available from BeRTA v3.6.0

This command sets the output gain of the model in dB. This is a gain that is applied to the model output after processing. A gain of 0 dB, default value, indicates that no gain is applied to the output samples after processing. The model to be enabled or disabled is identified by an identifier defined in the used [settings file](#).

Syntax

`/modelGain <string model_id> <float gain>`

`model_id`: identifier assigned to the model.

`gain`: A floating value, expressed in dB, representing the gain. The model output is multiplied by $(10^{\{gain / 20\}})$.

Return

`/control/actionResult /modelGain <string model_id> <bool setGain> <string description>`

The return confirmation refers to the `model_id`, indicating `setGain=true` if the action has been successfully performed and `setGain=false` if not. In both cases a description is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/modelGain DirectPath -3`

BeRTA sends back to the sender: `/control/actionResult /modelGain DirectPath true "Listener model DirectPath gain updated to -3dB."`

/RECORDIR

Available from BeRTA v3.13.0

Generates a file of the specified duration containing the **system's impulse response**, as configured at that moment. The applied impulse can be adjusted using various

parameters. Any sound sources present in the simulation will be removed to carry out the measurement. This processing and recording will take place *without real-time playback*. If the generated file is of the wav type, only the impulse response is saved. If the type is mat, additional data will be saved; the generated file will follow the structure of the [SOFA AnnotatedReceiverAudio convention](#).

Syntax

`/recordIR <String filename> <String type> <float time> <int period> <int delay> <float x> <float y> <float z> <String directivity> <bool enable_directivity>`

`filename`: indicates the name of the file and must include the path, either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable. The extension will be added by the application if necessary. If there is a file with the same name, it'll not be overwritten—an ordinal number will be added to the end of the new file name.

`type`: indicates the extension of the file: **mat** (MATLAB binary data container) or **wav** (32-bit IEEE floating-point stereo).

`time`: indicates the duration of the recording in seconds. *This value must be greater than 0.* The special value `-1` is not allowed.

`period`: indicates the number of samples between pulses. If this value is zero, the response to a single pulse is recorded. However, if you wish to record the response to a *impulse train*, this value must indicate the number of samples between impulses. This value must be less than the duration of the recording.

`delay`: indicates the number of samples by which the impulse is delayed relative to the origin. This value must be less than the duration of the recording.

`x`: global coordinate of the impulse's position on the X-axis, expressed in metres. As a reference, X axis is positive to the front.

`y`: global coordinate of the impulse's position on the Y-axis. As a reference, Y axis is positive to the left.

`z`: global coordinate of the impulse's position on the Z-axis, expressed in metres. As a reference, Z axis is positive to up.

`directivity`: indicates the ID of the directivity to be applied to the impulse source. If the specified ID corresponds to a previously loaded directivity, the recording is made using that directivity. If the specified ID is not valid or does not correspond to a loaded directivity, the recording is made without any directivity assigned. If this parameter is left empty, the impulse source is considered omnidirectional.

`enable_directivity`: indicates whether directivity is enabled for the impulse source. If set to true, the impulse source is created with the specified directivity enabled. If set to false, the impulse source is created with the specified directivity disabled. If this parameter is not specified, it defaults to true. If no valid directivity is specified, this parameter has no effect.

Return

```
/control/actionResult /recordIR <string filename> <boolean success> <string description>.
```

The return confirmation refers to the `filename`, indicating `success=true` if the action has been successfully performed and `success=false` if not. In both cases a `description` is added to give more details.

It sends an echo to all subscribers except the sender: `/recordIR <String filename> <String type> <float time> <int`

```
period> <int delay> <float x> <float y> <float z> <String directivity> <bool enable_directivity>
```

Example

BeRTA receives and echoes back to all subscribers except the sender:

```
/recordIR "record/sytem_conf_IR.wav" "wav" 2 0 0 1 0 0
Directivity1 true
```

When the processing is finished (in this example after 2 seconds), BeRTA sends back to all subscribers:

```
/control/actionResult /recordIR record/system_conf_IR.wav
true Recording completed. File saved: record/
sytem_conf_IR.wav
```

or

```
/control/actionResult /recordIR record/sytem_conf_IR.wav
false ERROR:Recording failed. File could not be created. or
/control/actionResult /recordIR Directivity1 false
ERROR:setting Directivity to the source. Directivity not
found into the directivity list.
```

3.3.4 Resources commands

These commands are responsible for managing the resources used in the virtual audio scene.

HRTF

/RESOURCES/LOADHRTF

Loads a new HRTF in a SOFA file from the specified path and assign an identifier to it. If there is already another HRTF with the same identifier, it is substituted by the new one. The SOFA file must use FIR or FIR-E as data type but can follow any convention using these data types. When loading, the HRTF is spatially resampled to fit a grid using the [SphericalInterpolatedFIRTable](#) service module. This way the real time processor can get fast enough the three HRIRs to be interpolated and convolved.

Syntax

```
/resources/loadHRTF <string HRTF_id> <string
HRTF_SOFAFile_path> <float spatialResolution>
```

HRTF_id : Identifier to be assigned to the HRTF for later references to it. If there is already another HRTF with the same identifier, it is substituted by the new one. Otherwise, a new HRTF is created.

HRTF_SOFAFile_path : Specifies a SOFA file containing an HRTF. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

spatialResolution : This parameter indicates the interpolation step in the horizontal plane of zero elevation. To learn more about this grid and the interpolation mechanism, please refer to the [SphericalInterpolatedFIRTable](#) service module.

Return

```
/control/actionResult /resources/loadHRTF <string HRTF_id>
<boolean loaded> <string description>
```

The return confirmation refers to the **HRTF_id**, indicating **loaded=true** if the HRTF is successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadHRTF HRTF1 c:/tmp/hrtf.sofa 5`

BeRTA sends back to the sender: `/control/actionResult /`

```
resources/loadHRTF HRTF1 true HRTF HRTF1 loaded
successfully, with a spatial resolution of 5, from the file:
c:/tmp/hrtf.sofa OR /control/actionResult /resources/
loadHRTF HRTF1 false HRTF sofa file couldn't be loaded from
HRTF1.
```

/RESOURCES/LOADHRTFRAW

Loads a new HRTF in a SOFA file from the specified path and assign an identifier to it. If there is already another HRTF with the same identifier, it is substituted by the new one. The SOFA file must use FIR or FIR-E as data type but can follow any convention using these data types. The data is stored without any resampling, using the [SphericalFIRTable](#) service module.

Syntax

```
/resources/loadHRTFraw <string HRTF_id> <string
HRTF_SOFAFile_path>
```

HRTF_id : Identifier to be assigned to the HRTF for later references to it. If there is already another HRTF with the same identifier, it is substituted by the new one. Otherwise, a new HRTF is created.

HRTF_SOFAFile_path : Specifies a SOFA file containing an HRTF. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

Return

```
/control/actionResult /resources/loadHRTFraw <string
HRTF_id> <boolean loaded> <string description>
```

The return confirmation refers to the **HRTF_id**, indicating **loaded=true** if the HRTF is successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadHRTF HRTF1 c:/tmp/hrtf.sofa`

BeRTA sends back to the sender: `/control/actionResult /resources/loadHRTF HRTF1 true HRTF HRTF1 loaded successfully from the file: c:/tmp/hrtf.sofa OR /control/actionResult /resources/loadHRTF HRTF1 false HRTF sofa file couldn't be loaded from HRTF1.`

/RESOURCES/REMOVEHRTF

Removes an HRTF from the loaded resources.

Syntax

```
/resources/removeHRTF <string HRTF_id>
```

HRTF_id : Identifier of the HRTF to be removed.

Return

```
/control/actionResult /resources/loadHRTF <string HRTF_id>
<bool removed> <string description>
```

The return confirmation refers to the **HRTF_id**, indicating **removed=true** if the HRTF is successfully removed and **removed=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeHRTF HRTF1`

BeRTA sends back to all subscribers: `/control/actionResult /resources/removeHRTF HRTF1 true HRTF HRTF1 removed` or `/control/actionResult /resources/removeHRTF HRTF1 false error: HRTF not found in the list`

/RESOURCES/GETHRTFINFO

Gets some information about one of the loaded HRTFs. this information consists of the filename and the spatial resolution used to resample it in the [HRTF grid](#).

Syntax

```
/resources/getHRTFInfo <string HRTF_id>
```

HRTF_id : Identifier of the HRTF of which we are requesting information.

Return

```
/resources/getHRTFInfo <string HRTF_id> <string
HRTF_SOFAFile> <float spatialResolution>
```

The return message is sent back to the sender and refers to the **HRTF_id**, indicating the name of the SOFA file (**HRTF_SOFAFile**) from which the HRTF was loaded, as well as the **spatialResolution** used to resample it when loaded.

Example

BeRTA receives: `/resources/getHRTFInfo HRTF1`

BeRTA sends back:

```
/resources/getHRTFInfo HRTF1 hrtf.sofa 5
```

/RESOURCES/SETHRTFHEADRADIUS

Sets the head radius to be stored into the HRTF. This value affects: (1) the position of the ears used in the parallax correction when getting HRIRs; and (2) the calculation of the ITD when using the simulation based on the Woodworth formula which is activated or deactivated by the `/resources/enableWoodworthITD` command. If this command is not received, the position of receivers in the HRTF SOFA file is used as the default value. See `/resources/restoreHRTFHeadRadius` as a related command.

Syntax

```
/resources/setHRTFHeadRadius <string HRTF_id> <float
headRadius>
```

HRTF_id : Identifier of the HRTF of which we are setting the new head radius.

headRadius : New head radius in meters. Only positives values are accepted.

Return

```
/control/actionResult /resources/setHRTFHeadRadius <string
HRTF_id> <bool set> <string description>
```

The return confirmation refers to the **HRTF_id**, indicating **set=true** if the HRTF is successfully set and **set=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/setHRTFHeadRadius HRTF1 0.09`

BeRTA sends back to the sender: `/control/actionResult /resources/setHRTFHeadRadius HRTF1 true success`

/RESOURCES/GETHRTFHEADRADIUS

Gets the head radius of one of the HRTFs loaded as a resource.

Syntax

```
/resources/getHRTFHeadRadius <string HRTF_id>
```

HRTF_id : Identifier of the HRTF of which we are requesting the head radius.

Return

A message is sent back to the sender `/resources/getHRTFHeadRadius <string HRTF_id> <float headRadius>`, indicating the **headRadius** of the HRTF **HRTF_id**.

Example

BeRTA receives: `/resources/getHRTFHeadRadius HRTF1`

BeRTA sends back to the sender: `/resources/
getHRTFHeadRadius HRTF1 0.09`

`/RESOURCES/RESTOREHRTFHEADRADIUS`

Sets the head radius of the HRTF to the one stored in the SOFA file as the position of receivers. This value affects: (1) the position of the ears used in the parallax correction when getting HRIRs; and (2) the calculation of the ITD when using the simulation based on the Woodworth formula which is activated or deactivated by the `/resources/enableWoodworthITD` command.

Syntax

`/resources/restoreHRTFHeadRadius <string HRTF_id>`

HRTF_id: Identifier of the HRTF for which we are setting the head radius.

Return

`/control/actionResult /resources/restoreHRTFHeadRadius
<string HRTF_id> <bool restored> <string description>`

The return confirmation refers to the **HRTF_id**, indicating `restored=true` if the HRTF is successfully set and `restored=false` if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/restoreHRTFHeadRadius HRTF1`

BeRTA sends back to the sender: `/control/actionResult /
resources/restoreHRTFHeadRadius HRTF1 true success`

`/RESOURCES/ENABLEWOODWORTHITD`

Enables or disables the calculation of the ITD based on the Woodworth formula, considering head radius, or it is extracted from the delays in the HRTF SOFA file. The Woodworth formula needs the head radius set by commands `/resources/setHRTFHeadRadius` and `/resources/restoreHRTFHeadRadius`. By default, the use of the Woodworth formula is disabled.

Syntax

`/resources/enableWoodworthITD <string HRTF_id> <boolean
enable>`

HRTF_id: Identifier of the HRTF for which we are setting the head radius.

enable: If `enable` is true (1), the ITD calculation is performed using the Woodworth formula. If `enable` is false (0), the ITD is simulated from the delays read from the SOFA file.

Return

`/control/actionResult /resources/enableWoodworthITD <string
HRTF_id> <bool enabled> <string description>`

The return confirmation refers to the **HRTF_id**, indicating `enabled=true` if the HRTF is successfully set and `enabled=false` if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/enableWoodworthITD HRTF1 true`

BeRTA sends back to the sender: `/control/actionResult /
resources/enableWoodworthITD HRTF1 true success`

BRIR

`/RESOURCES/LOADBRIR`

Loads a new Binaural Room Impulse Response (BRIR) from a SOFA file at the specified path and assign an identifier to it. If there is already another BRIR with the same identifier, it is substituted by the new one. The SOFA file must use FIR or FIR-E as data type but can follow any convention using these data types. The data is stored without any resampling, using the [ShericalFIRTable](#) service module.

Syntax

`/resources/loadBRIR <string BRIR_id> <string
BRIR_SOFAFile_path> <float fadeInBegin> <float riseTime>
<float fadeOutCutoff> <float fallTime>`

BRIR_id: Identifier to be assigned to the BRIR for later references to it. If there is already another BRIR with the same identifier, it is substituted by the new one. Otherwise, a new BRIR is created.

BRIR_SOFAFile_path: Specifies a SOFA file containing a BRIR. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

The loaded BRIR can be windowed. For this purpose, a fade-in and fade-out mechanism has been enabled using raised-cosines. The parameter `fadeInBegin` indicates the

midpoint (50%) of the rising slope of the window, in seconds, the `riseTime` indicates the rising time, also in seconds. Analogously, the parameter `fadeOutCutoff` indicates the midpoint (50%) of the falling slope of the window, and `fallTime` indicates the falling time, all in seconds.

Return

```
/control/actionResult /resources/loadBRIR <string BRIR_id>
<boolean loaded>
```

The return confirmation refers to the `BRIR_id`, indicating `loaded=true` if the BRIR is successfully loaded and `loaded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadBRIR BRIR1 c:/tmp/BRIR.sofa 0.005 0.001 0.5 0.01`

BeRTA sends back to the sender: `/control/actionResult /resources/loadBRIR BRIR1 true BRIR BRIR1 loaded successfully from the file: c:/tmp/BRIR.sofa` OR `/control/actionResult /resources/loadBRIR BRIR1 false BRIR sofa file couldn't be loaded from c:/tmp/BRIR.sofa`.

Changes from BeRTA v3.11.0

From BeRTA v3.11.0 onwards, the *spatial resolution* parameter has been removed, as a BRIR is no longer interpolated when loaded.

/RESOURCES/REMOVEBRIR

Removes a Binaural Room Impulse Response (BRIR) from the loaded resources.

Syntax

```
/resources/removeBRIR <string BRIR_id>
```

`BRIR_id`: Identifier of the BRIR to be removed.

Return

```
/control/actionResult /resources/removeBRIR <string BRIR_id>
<bool removed> <string description>
```

The return confirmation refers to the `BRIR_id`, indicating `removed=true` if the BRIR successfully removed and `removed=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeBRIR BRIR1`

BeRTA sends back to all subscribers: `/control/actionResult /resources/removeBRIR BRIR1 true BRIR BRIR1 removed` or just to the sender: `/control/actionResult /resources/removeBRIR BRIR1 false ERROR deleting BRIR. BRIR1 not found in the list`

/RESOURCES/GETBRIRINFO

Gets some information about one of the loaded Binaural Room Impulse Response (BRIR). This information consists of the name of the file from which it was loaded, the spatial resolution and all the parameters of the window with which the BRIR is windowed.

Syntax

```
/resources/getBRIRInfo <string BRIR_id>
```

`BRIR_id`: Identifier of the BRIR of which we are requesting information.

Return

```
/resources/getBRIRInfo <string BRIR_id> <string BRIR_SOFAFile> <float spatialResolution> <float fadeInWindowThreshold> <float fadeInWindowRiseTime> <float fadeOutWindowThreshold> <float fadeOutWindowFallTime>
```

The return message is sent back to the sender and refers to the `BRIR_id`, indicating the name of the SOFA file (`BRIR_SOFAFile`) from which the filters were loaded, the spatial resolution at which it is resampled and all the parameters defining the windowing:

`fadeInWindowThreshold` and `fadeInWindowRiseTime` for the fade-in rising slope; and `fadeOutWindowThreshold` and `fadeOutWindowFallTime` for the fade-out falling slope.

Example

BeRTA receives: `/resources/getBRIRInfo BRIR1`

BeRTA sends back: `/resources/getBRIRInfo BRIR1 BRIR.sofa 15 0.005 0.001 0.5 0.01`

Directivity

/RESOURCES/LOADDIRECTIVITY

Loads a new Directivity Transfer Function in a SOFA file from the specified path and assign an identifier to it. If there is already another Directivity with the same identifier, it is substituted by the new one. The SOFA file must use

FreeFieldDirectivityFIR convention. When loading, the directivity is spatially resampled to fit a grid using the [SphericalInterpolatedFIRTable](#) service module.

Syntax

```
/resources/loadDirectivity <string directivity_id> <string directivity_SOFAFile_path> <float spatialResolution>
```

directivity_id: Identifier to be assigned to the directivityTF for later references to it. If there is already another directivityTF with the same identifier, it is substituted by the new one. Otherwise, a new directivityTF is created.

directivity_SOFAFile_path: Specifies a SOFA file containing a directivityTF. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

spatialResolution: This parameter indicates the interpolation step in the horizontal plane of zero elevation. To learn more about this grid and the interpolation mechanism, please refer to the [SphericalInterpolatedFIRTable](#) service module.

Return

```
/control/actionResult /resources/loadDirectivity <string directivity_id> <boolean loaded> <string description>
```

The return confirmation refers to the **directivity_id**, indicating **loaded=true** if the directivityTF is successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadDirectivity Directivity1 c:/tmp/directivity.sofa 15`

BeRTA sends back to the sender: `/control/actionResult /resources/loadDirectivity Directivity1 true Directivity1 loaded` OR `/control/actionResult /resources/loadDirectivity Directivity1 false Directivity sofa file couldn't be loaded from c:/tmp/directivityTF.sofa.`

/RESOURCES/LOADDIRECTIVITYRAW

Loads a new Directivity Transfer Function in a SOFA file from the specified path and assign an identifier to it. If there is already another Directivity with the same identifier, it is substituted by the new one. The SOFA file must use FreeFieldDirectivityFIR convention. The data is stored

without any resampling, using the [ShericalFIRTable](#) service module.

Syntax

```
/resources/loadDirectivityRaw <string directivity_id> <string directivity_SOFAFile_path>
```

directivity_id: Identifier to be assigned to the directivityTF for later references to it. If there is already another directivityTF with the same identifier, it is substituted by the new one. Otherwise, a new directivityTF is created.

directivity_SOFAFile_path: Specifies a SOFA file containing a directivityTF. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

Return

```
/control/actionResult /resources/loadDirectivityRaw <string directivity_id> <boolean loaded> <string description>
```

The return confirmation refers to the **directivity_id**, indicating **loaded=true** if the directivityTF is successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadDirectivity Directivity1 c:/tmp/directivity.sofa`

BeRTA sends back to the sender: `/control/actionResult /resources/loadDirectivity Directivity1 true Directivity1 loaded` OR `/control/actionResult /resources/loadDirectivity Directivity1 false Directivity sofa file couldn't be loaded from c:/tmp/directivity.sofa.`

/RESOURCES/REMOVEDIRECTIVITY

Removes a Directivity Transfer Function from the loaded resources.

Syntax

```
/resources/removeDirectivity <string directivity_id>
```

directivity_id: Identifier of the directivity to be removed.

Return

```
/control/actionResult /resources/removeDirectivity <string directivity_id> <bool removed>
```

The return confirmation refers to the **directivity_id**, indicating **removed=true** if the directivityTF is successfully

removed and `removed=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeDirectivity directivity1`

BeRTA sends back to the sender: `/control/actionResult /resources/removeDirectivity directivity1 true Directivity directivity1 removed` OR `/control/actionResult /resources/removeDirectivity directivity1 false ERROR deleting Directivity. directivity1 not found in the list`

/RESOURCES/GETDIRECTIVITYINFO

Gets some information about one of the loaded Directivity Transfer Functions. This information consists of the filename and the spatial resolution used if it was loaded using interpolation.

Syntax

`/resources/getDirectivityInfo <string directivity_id>`

`directivity_id`: Identifier of the Directivity of which we are requesting information.

Return

`/resources/getDirectivityInfo <string directivity_id> <string directivity_SOFAFile> <float spatialResolution>`

The return message is sent back to the sender and refers to the `directivity_id`, indicating the name of the SOFA file (`directivity_SOFAFile`) from which the directivity was loaded, as well as the `spatialResolution` used to resample it when loaded.

Example

BeRTA receives: `/resources/getDirectivityInfo directivity1`

BeRTA sends back: `/resources/getDirectivityInfo directivity1 directivity.sofa 15`

/RESOURCES/LOADDIRECTIVITYTF

Loads a new Directivity Transfer Function in a SOFA file from the specified path and assign an identifier to it. If there is already another Directivity with the same identifier, it is substituted by the new one. The SOFA file must use TF as data type with the same number of frequency bins as the frame size configured in BeRTA. When loading, the DirectivityTF is spatially resampled to fit the [BRT grids](#). This way the real time processor can get fast enough the three Directivities to be interpolated and convolved.

Syntax

`/resources/loadDirectivityTF <string directivityTF_id> <string directivityTF_SOFAFile_path> <float spatialResolution>`

`directivityTF_id`: Identifier to be assigned to the directivityTF for later references to it. If there is already another directivityTF with the same identifier, it is substituted by the new one. Otherwise, a new directivityTF is created.

`directivityTF_SOFAFile_path`: Specifies a SOFA file containing a directivityTF. it can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

`spatialResolution`: This parameter indicates the interpolation step in the horizontal plane of zero elevation. To learn more about this grid and the interpolation mechanism, please refer to the [BRT grids](#).

Return

`/control/actionResult /resources/loadDirectivityTF <string directivityTF_id> <boolean loaded> <string description>`

The return confirmation refers to the `directivityTF_id`, indicating `loaded=true` if the directivityTF is successfully loaded and `loaded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadDirectivityTF DirectivityTF1 c:/tmp/directivityTF.sofa 15`

BeRTA sends back to the sender: `/control/actionResult /resources/loadDirectivityTF DirectivityTF1 true Directivity directivityTF1 loaded` OR `/control/actionResult /resources/loadDirectivityTF DirectivityTF1 false Directivity sofa file couldn't be loaded from c:/tmp/directivityTF.sofa .`

/RESOURCES/REMOVEDIRECTIVITYTF

Removes a Directivity Transfer Function from the loaded resources.

Syntax

`/resources/removeDirectivityTF <string directivityTF_id>`

`directivityTF_id`: Identifier of the directivityTF to be removed.

Return

```
/control/actionResult /resources/removeDirectivityTF <string
directivityTF_id> <bool removed>
```

The return confirmation refers to the `directivityTF_id`, indicating `removed=true` if the `directivityTF` is successfully removed and `removed=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeDirectivityTF directivityTF1`

BeRTA sends back to the sender: `/control/actionResult /resources/removeDirectivityTF directivityTF1 true DirectivityTF directivityTF1 removed or /control/actionResult /resources/removeDirectivityTF directivityTF1 false ERROR deleting DirectivityTF. directivityTF1 not found in the list`

`/RESOURCES/GETDIRECTIVITYTFINFO`

Gets some information about one of the loaded Directivity Transfer Functions. This information consists of the filename and the spatial resolution used to resample it in the [BRT grid](#). Deleted from BeRTA v3.14.0

Syntax

```
/resources/getDirectivityTFInfo <string directivityTF_id>

directivityTF_id : Identifier of the DirectivityTF of which we
are requesting information.
```

Return

```
/resources/getDirectivityTFInfo <string directivityTF_id>
<string directivityTF_SOFAFile> <float spatialResolution>
```

The return message is sent back to the sender and refers to the `directivityTF_id`, indicating the name of the SOFA file (`directivityTF_SOFAFile`) from which the directivity was loaded, as well as the `spatialResolution` used to resample it when loaded.

Example

BeRTA receives: `/resources/getDirectivityTFInfo directivityTF1`

BeRTA sends back: `/resources/getDirectivityTFInfo directivityTF1 directivityTF.sofa 15`

Filters Coeffs. (SOS and FIR)

`/RESOURCES/LOADFILTER`

Load a new set of filter coefficients (IIR or FIR) from a SOFA file, from the specified path, and assign it an identifier. This SOFA file can be: Loadable service: SOS Coefficients (IIR or FIR) Available since BeRTA v3.10.0.

- SOS type, i.e. it contains second-order sections (SOS) coefficients. They are loaded into an [SOS-type](#) service module.
- FIR type, i.e. it contains impulse responses. They are loaded into a [GeneralFIR-type](#) service module.

On the other hand, if another set of filter coefficients with the same identifier already exists, it is replaced by this new one.

Syntax

```
/resources/loadFilter <string filter_id> <string
filter_SOFAFile_path>
```

`filter_id` : Identifier to be assigned to filter coefficients for later references to it. If there is already another set of coefficients with the same identifier, it is substituted by the new one. Otherwise, a new set of filters coefficients is created.

`filter_SOFAFile_path` : Specifies a SOFA file containing a set of filters coefficients, either IIR (SOS data type) or FIR (FIR or FIR-E data type). It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

Return

```
/control/actionResult /resources/loadFilter <string
filter_id> <boolean loaded>
```

The return confirmation refers to the `filter_id`, indicating `loaded=true` if SOS NFC Filters are successfully loaded and `loaded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadFilters EarMuffFilter resources//SOSFilters//EarMuffFilter_48Khz.sofa`

BeRTA sends back to the sender: `/control/actionResult /resources/loadFilters EarMuffFilter true SOS filter coef. (EarMuffFilter) loaded successfully, from the file: resources//SOSFilters//EarMuffFilter_48Khz.sofa or /resources/loadSOSFilters EarMuffFilter false File path could`

```
not be found: resources//SOSFilters//
EarMuffFilter_48Khz.sofa..
```

/RESOURCES/REMOVEFILTER

Removes a set of filter coefficients from the loaded resources. Works for any type of SOS or FIR filter. If this set of filter coefficients was loaded into any model, they are unlinked from the model before being deleted.

Syntax

```
/resources/removeFilter <string filter_id>
```

filter_id: Identifier of the filters to be removed.

Return

```
/control/actionResult /resources/removeFilter <string
filter_id> <bool removed> <string description>
```

The return confirmation refers to the **filter_id**, indicating **removed=true** if the set of filters coefficients are successfully removed and **removed=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeFilter EarMuffFilter`

BeRTA sends back to the sender: `/control/actionResult /resources/removeFilter EarMuffFilter true Filter coeffs. (EarMuffFilter) removed from the resource list. OR /control/actionResult /resources/removeFilter EarMuffFilter false ERROR deleting coeffs. (EarMuffFilter) not found in the list.`

/RESOURCES/GETFILTERINFO

Gets some information about one of the loaded set of filters coefficients. This information consists of the name of the file from which it was loaded.

Syntax

```
/resources/getFilterInfo <string filter_id>
```

filter_id: Identifier of the set of filters coeffs. of which we are requesting information.

Return

```
/resources/getFilterInfo <string filter_id> <string
filtersCoefs_SOFAFile>
```

The return message is sent back to the sender and refers to the **filter_id**, indicating the name of the SOFA file (**filtersCoefs_SOFAFile**) from which the filters were loaded.

Example

BeRTA receives: `/resources/getFilterInfo NFCF1`

BeRTA sends back: `/resources/getFilterInfo NFCF1
NFCFilters.sofa`

/RESOURCES/LOADSOSFILTERS

Loads a new set of Second Order Section (SOS) filters in a SOFA file from the specified path and assign an identifier to it. If there is already another set of SOS filters with the same identifier, it is substituted by the new one. The SOFA file must use SOS as data type.

Syntax

```
/resources/loadSOSFilters <string SOSFilters_id> <string
SOSFilters_SOFAFile_path>
```

SOSFilters_id: Identifier to be assigned to the NFC Filters for later references to it. If there is already another set of NFC Filters with the same identifier, it is substituted by the new one. Otherwise, a new set of NFC Filters is created.

SOSFilters_SOFAFile_path: Specifies a SOFA file containing a set of NFC Filters. It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

Return

```
/control/actionResult /resources/loadSOSFilters <string
SOSFilters_id> <boolean loaded>
```

The return confirmation refers to the **SOSFilters_id**, indicating **loaded=true** if SOS NFC Filters are successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadSOSFilters SOSF1 c:/tmp/SOSilters.sofa`

BeRTA sends back to the sender: `/control/actionResult /resources/loadSOSFilters NFCF1 true SOS filter SOSF1 loaded successfully, from the file: c:/tmp/SOSilters.sofa OR /resources/loadSOSFilters SOSF1 false ERROR deleting SOS filter. SOSF1 not found in the list.`

/RESOURCES/REMOVESOSFILTERS

Removes a set of Near Field Compensation (NFC) filters
Compatible services: SOS-CompFidant
 Deprecated since BeRTA v3.10.0, use '/resources/removeFilter' instead.

Syntax

```
/resources/removeSOSFilters <string SOSFilters_id>
```

SOSFilters_id: Identifier of the NFC Filters to be removed.

Return

```
/control/actionResult /resources/removeSOSFilters <string  

SOSFilters_id> <bool removed> <string description>
```

The return confirmation refers to the **SOSFilters_id**, indicating **removed=true** if the NFC Filters are successfully removed and **removed=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeSOSFilters SOSF1`

BeRTA sends back to the sender: `/control/actionResult /resources/removeSOSFilters SOSF1 true SOSF1 removed from the list` or `/control/actionResult /resources/removeSOSFilters SOSF1 false ERROR deleting SOS Filters. SOSF1 not found in the SOS Filters list`

/RESOURCES/GETSOSFILTERSINFO

Gets some information about one of the loaded set of Near Field Compensation (NFC) filters.
Compatible services: SOS-CompFidant
 Deprecated since BeRTA v3.10.0, use '/resources/getFilterInfo' instead.

This information consists of the name of the file from which it was loaded.

Syntax

```
/resources/getSOSFiltersInfo <string SOSFilters_id>
```

SOSFilters_id: Identifier of the NFC Filters of which we are requesting information.

Return

```
/resources/getSOSFiltersInfo <string SOSFilters_id> <string  

SOSFilters_SOFAFile>
```

The return message is sent back to the sender and refers to the **SOSFilters_id**, indicating the name of the SOFA file (**SOSFilters_SOFAFile**) from which the filters were loaded.

Example

BeRTA receives: `/resources/getSOSFiltersInfo NFCF1`

BeRTA sends back: `/resources/getSOSFiltersInfo NFCF1 NFCFilters.sofa`

Room**/RESOURCES/LOADROOM**

Loads a new room from a OBJ file from the specified path and assign an identifier to it. If there is already another set of room with the same identifier, it is substituted by the new one. Please note the considerations regarding OBJ files specified in the OBJ services section.

Syntax

```
/resources/loadRoom <string Room_id> <string OBJ_File_path>
```

Room_id: Identifier to be assigned to the Room for later references to it. If there is already another room with the same identifier, it is substituted by the new one.

OBJ_File_path: Specifies an OBJ file containing a room (see the OBJ Services section for more information). It can be either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable.

Return

```
/control/actionResult /resources/loadRoom <string Room_id>  

<boolean loaded>
```

The return confirmation refers to the **Room_id**, indicating **loaded=true** if room are successfully loaded and **loaded=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/loadRoom room1 c:/tmp/Room1.obj`

BeRTA sends back to the sender: `/control/actionResult /resources/loadRoom room1 true Room (room1) loaded successfully, from the file: c:/tmp/Room1.obj` OR `/resources/loadRoom room1 false ERROR File path could not be found: c:/tmp/Room1.obj.`

/RESOURCES/LOADSHOEBOXROOM

Sets up a shoebox room with six walls (four walls, plus floor and ceiling). The six walls are automatically created with

Added from BeRTA v3.9.0

the centre of the room at (0, 0, 0). The following ID are assigned for each wall:

- 0 front wall (positive x)
- 1 left wall (positive y)
- 2 right wall (negative y)
- 3 back wall (negative x)
- 4 floor (negative z)
- 5 ceiling (positive z)

Syntax

```
/resources/setShoeBoxRoom <string Room_id> <float length>
<float width> <float height>
```

Room_id: identifier to be assigned to the Room for later references to it. If there is already another room with the same identifier, it is substituted by the new one. .

length: floating value expressing the length of the room on the X axis, expressed in metres.

width: Floating value expressing the length of the room on the Y axis, expressed in metres.

height: Floating value expressing the length of the room on the Z axis, expressed in metres.

Return

```
/control/actionResult /resources/setShoeBoxRoom <string
Room_id> <bool set> <string description>
```

The return confirmation refers to the **Room_id**, indicating **set=true** if the action has been successfully performed and **set=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/setShoeBoxRoom room1 6 8 3`

BeRTA sends back to the sender: `/control/actionResult /environment/setShoeBoxRoom room1 true "ShoeBox Room (room1) created succesfully with dimensions Length 6m, Width 8m, Height 3m`

/RESOURCES/REMOVEROOM

Removes a room from the loaded resources. Available from BeRTA v3.9.0.

Syntax

```
/resources/removeRoom <string Room_id>
```

Room_id: Identifier of the room to be removed.

Return

```
/control/actionResult /resources/removeRoom <string Room_id>
<bool removed> <string description>
```

The return confirmation refers to the **Room_id**, indicating **removed=true** if the room are successfully removed and **removed=false** if not. In both cases a **description** is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/removeRoom room1`

BeRTA sends back to the sender: `/control/actionResult /resources/removeRoom room1 true Room (room1) removed from the resources list` OR `/control/actionResult /resources/removeRoom room1 false ERROR deleting Room. Room1 not found in list.`

/RESOURCES/SETROOMWALLABSORPTION

Compatible mode: SDN-ISM
Sets the absorption coefficients of a room wall. It supports two modes of operation:
Available from BeRTA v3.9.0, previously called /environment/WallAbsorption.

- Specify a frequency-independent absorption coefficient. That is, using a single number between 0 and 1, which is used for all bands.
- Specify absorption coefficients for each of the 9 simulated bands. In other words, in this case, nine numbers between 0 and 1 are used, with absorptions corresponding to 62.5 Hz, 125 Hz, 250 Hz, 500 Hz, 1 kHz, 2 kHz, 4 kHz, 8 kHz and 16 kHz.

Syntax

Two options available:

- `/resources/setRoomWallAbsorption <string Room_id> <int wall_id> <float absorption_fullband>`
- `/resources/setRoomWallAbsorption <string Room_id> <int wall_id> <float abs62> <float abs125> <float abs250> <float abs500> <float abs1K> <float abs2K> <float abs4K> <float abs8K> <float abs16K>`

Room_id: identifier of the room.

wall_id: index of the wall to which the absorption applies.

absorption_fullband: absorption coefficient which is frequency independent.

abs62: absorption coefficient of the band centered at 62.5Hz.

`abs125` : absorption coefficient of the band centered at 125Hz.

`abs250` : absorption coefficient of the band centered at 250Hz.

`abs500` : absorption coefficient of the band centered at 500Hz.

`abs1K` : absorption coefficient of the band centered at 1KHz.

`abs2K` : absorption coefficient of the band centered at 2KHz.

`abs4K` : absorption coefficient of the band centered at 4KHz.

`abs8K` : absorption coefficient of the band centered at 8KHz.

`abs16K` : absorption coefficient of the band centered at 16KHz.

Return

```
/control/actionResult /resources/setRoomWallAbsorption
<string Room_id> <bool set> <string description>
```

The return confirmation refers to the `Room_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Examples

BeRTA receives and echoes back to all subscribers but the sender: `/resources/setRoomWallAbsorption Room1 0 0.1 0.2 0.3 0.4 0.5 0.4 0.3 0.2 0.1`

BeRTA sends back to the sender: `/control/actionResult /resources/setRoomWallAbsorption Room1 true "The absorption coefficients of the wall (0) of the room (Room1) have been updated: [0.100000, ...]"`

/RESOURCES/ENABLEROOMWALL

Enables or disables a wall in the room. Disabling it has the same effect as not having defined it. By default, all walls are activated after defining the geometry of a room.

Syntax

```
/resources/enableRoomWall <string Room_id> <int wall_id>
<boolean enable>
```

`Room_id` : identifier assigned to the model.

`wall_id` : index of the wall to which the absorption applies.

`enable` : If false (0), disable the simulation of the indicated room wall. If true (1), enables the simulation of the indicated room wall. By default, all walls are enabled.

Return

```
/control/actionResult /resources/enableRoomWall <string
Room_id> <bool set> <string description>
```

The return confirmation refers to the `Room_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/resources/enableRoomWall Room1 1 false`

BeRTA sends back to the sender: `/control/actionResult /resources/enableRoomWall Room1 true "The wall (1) of the room (Room1) has been disabled."`

3.3.5 Source commands

These commands provide a comprehensive set of tools for managing and controlling audio sources within the virtual scene. These commands allow for the addition, manipulation, and control of sound sources, including their playback, position, orientation, and various audio properties.

/SOURCE/LOADSOURCE

Adds a new source loading a sound file (wav, mp3 and aif formats supported). Adding a new source does not imply playing it back. When adding the source an ID string is associated. That ID can be used later to control the source using other OSC commands.

Syntax

```
/source/loadSource <string source_id> <string source_path>
<string source_model> [<string modelToConnectTo>]
```

Specifying a sound file with `source_path`, a new sound file is loaded from the specified path and an identifier `source_id` is assigned to it. If there is already another source with the same identifier, it is substituted by the new one. Otherwise, a new source is created.

`source_model` indicates the source model to be used, possible values are: `OmnidirectionalModel`, `DirectivityModel`. See [Source Models](#) for more details.

`modelToConnectTo` is optional and specifies which model (listener or environment) this source will connect to. If left blank, the source will connect to the default models, i.e. those specified via the 'ConnectSourcesTo' field in the 'ModelsArchitecture' [section](#) of the application's configuration file (settings.json).

Return

```
/control/actionResult /source/loadSource <string source_id>
<bool loaded> <string description> .
```

The return confirmation refers to the `source_id`, indicating `loaded=true` if the source is successfully loaded and `loaded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/loadSource source1 speech.wav`

BeRTA sends back to the sender: `/control/actionResult /source/loadSource source1 true Audio source source1 loop`

```
enabled or /source/loadSource source1 ERROR: The audio
source source1 doesn't exist.
```

/SOURCE/ADDLINEIN

Adds a new source getting it from an audio input channel. Adding a new source does not imply playing it back. When adding the source an ID string is associated. That ID can be used later to control the source using other OSC commands.

Syntax

```
/source/addLineIn <string source_id> <int linein_channel>
<string source_model> [<string modelToConnectTo>]
```

`source_id`: identifier assigned to the sound source for further references to it

`linein_channel`: indicates the input channel of the selected audio device to be linked to this sound source. The channels are numbered starting from 1.

`source_model` indicates the source model to be used, possible values are: `OmnidirectionalModel`, `DirectivityModel`. See [Source Models](#) for more details.

`modelToConnectTo` is optional and indicates to which listener model this source will be connected. If left blank, the source will be connected to all listener models that have been indicated in the application [settings menu](#).

Return

```
/control/actionResult /source/addLineIn <string source_id>
<bool loaded> <string description>
```

The return confirmation refers to the `source_id`, indicating `loaded=true` if the source is successfully loaded and `loaded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/addLineIn source1 1 DirectivityModel`

BeRTA sends back to the sender: `/control/actionResult /source/addLineIn source1 true AUDIO LINE-IN source1 configured succesfully or /source/addLineIn source1 false error: not a valid channel`

/SOURCE/REMOVESOURCE

Removes a sound source from the system.

Syntax

```
/source/removeSource <string source_id>
```

`source_id` : identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/removeSource <string  
source_id> <bool removed> <string description>
```

The return confirmation refers to the `source_id`, indicating `removed=true` if the source is successfully removed and `removed=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/remove source1`

BeRTA sends back to the sender : `/control/actionResult /source/remove source1 true Audio source removed: source1` OR `/control/actionResult /source/remove source1 false ERROR: The audio source source1 doesn't exist.`

/SOURCE/PLAY

Plays a specific source, starting to send frames to the library from either the audio file or the input channel. In the first case, it starts from the beginning of the file unless the source was previously started and a `/source/pause` command was sent before. In such case it resumes at the point it was paused.

Syntax

```
/source/play <string source_id>
```

`source_id` : identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/play <string source_id> <bool  
played> <string description>
```

The return confirmation refers to the `source_id`, indicating `played=true` if the source is successfully played and `played=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/play source1`

BeRTA sends back to the sender: `/control/actionResult /source/play source1 true Audio source source1 played back`

/SOURCE/STOP

Stops a specific source. In the case of a source coming from a file, this resets the source so that the next `/source/play` command will start at the beginning of the file.

Syntax

```
/source/stop <string source_id>
```

`source_id` : identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/stop <string source_id> <bool  
stopped> <string description>
```

The return confirmation refers to the `source_id`, indicating `stopped=true` if the action has been successfully performed and `stopped=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/stop source1`

BeRTA sends back to the sender: `/control/actionResult /source/stop source1 true Audio source source1 removed.`

/SOURCE/PAUSE

Pauses a specific source. In the case of a source coming from a file, the next `/source/play` command will resume, starting at the same point where it was paused. If the `source_id` does not exist, the command is ignored.

Syntax

```
/source/pause <string source_id>
```

`source_id` : identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/pause <string source_id> <bool  
paused> <string description>
```

The return confirmation refers to the `source_id`, indicating `paused=true` if the action has been successfully performed and `paused=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/pause source1`

BeRTA sends back to the sender: `/control/actionResult /source/pause source1 true Audio source source1 paused.`

/SOURCE/MUTE

Mutes a specific source. In the case of a source coming from a file, the application keeps getting the sound stream from the file at the same rate, but silently. The `/source/unmute` command will make the source sound again. If the `source_id` does not exist, the command is ignored.

Syntax

```
/source/mute <string source_id>
```

`source_id`: identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/mute <string source_id> <bool muted> <string description>
```

The return confirmation refers to the `source_id`, indicating `muted=true` if the action has been successfully performed and `muted=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/mute source1`

BeRTA sends back to the sender: `/control/actionResult /source/mute source1 true Audio source source1 muted.`

/SOURCE/UNMUTE

Unmutes a specific source. In the case of a source coming from a file previously muted, the application has kept getting the sound stream from the file at the same rate, but silently. The `/source/unmute` command will make the source sound again. If the `source_id` does not exist, the command is ignored.

Syntax

```
/source/unmute <string source_id>
```

`source_id`: identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/unmute <string source_id> <bool unmuted> <string description>
```

The return confirmation refers to the `source_id`, indicating `unmuted=true` if the action has been successfully performed and `unmuted=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/unmute source1`

BeRTA sends back to the sender: `/control/actionResult /source/unmute source1 true Audio source source1 unmuted.`

/SOURCE/SOLO

Mutes all sources but the one indicated by this command. In the case of sources coming from files, the application keeps getting the sound stream from them at the same rate, but silently. The `/source/unsolo` command will make all the sources sound again. Individual commands `/source/unmute` addressed to those sources muted as a consequence of an `/source/solo` command will have the same effect making them sound again. If the `source_id` does not exist, the command is ignored.

Syntax

```
/source/solo <string source_id>
```

`source_id`: identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/solo <string source_id> <bool solo> <string description>
```

The return confirmation refers to the `source_id`, indicating `solo=true` if the action has been successfully performed and `solo=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/solo source1`

BeRTA sends back to the sender: `/control/actionResult /source/solo source1 true Audio source source1 playing solo.`

`/SOURCE/UNSOLO`

Unmutes all sources but the one indicated by this command. If the `source_id` does not exist, the command is ignored.

Syntax

```
/source/unsolo <string source_id>
```

`source_id`: identifier assigned to the sound source for further references to it

Return

```
/control/actionResult /source/unsolo <string source_id>
<bool unsolo> <string description>
```

The return confirmation refers to the `source_id`, indicating `unsolo=true` if the action has been successfully performed and `unsolo=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/unsolo source1`

BeRTA sends back to the sender: `/control/actionResult /source/unsolo source1 true All audio sources unmuted.`

`/SOURCE/LOOP`

Sets loop mode for the identified source. If loop mode is enabled, when reaching the end, the source will start again. If loop mode is disabled, when reaching the end, the source will stop.

Syntax

```
/source/loop <string source_id> <boolean enable>
```

`source_id`: identifier assigned to the sound source for further references to it

`enable`: if true (1) loop mode is enabled. If false (0), loop mode is disabled

Return

```
/control/actionResult /source/loop <string source_id> <bool
loop> <string description>
```

The return confirmation refers to the `source_id`, indicating `loop=true` if the action has been successfully performed and `loop=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/loop source1`

BeRTA sends back to the sender: `/control/actionResult /source/loop source1 true Audio source source1 enabled.`

`/SOURCE/LOCATION`

Set location of source. Position is set in global x,y,z coordinates, expressed in meters. The new location is sent to all remotes except the one has sent the message.

Syntax

```
/source/location <string source_id> <float x> <float y>
<float z>
```

`source_id`: identifier assigned to the sound source for further references to it

`x`: X global coordinate, expressed in metres. As a reference, X axis is positive to the front.

`y`: Y global coordinate, expressed in metres. As a reference, Y axis is positive to the left.

`z`: Z global coordinate, expressed in metres. As a reference, Z axis is positive to up.

Return

Any confirmation message is sent back to the sender.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/location source1 2 0 0`

`/SOURCE/ORIENTATION`

Set orientation of the source. Orientation is set in egocentric coordinates yaw, pitch and roll, expressed in radians and applied in that order (Yaw, Pitch and Roll). An orientation of (0, 0, 0) corresponds to be oriented towards

positive X. The new orientation is sent to all remotes except the one has sent the message.

Syntax

```
/source/orientation <string source_id> <float yaw> <float pitch> <float roll>
```

source_id: identifier assigned to the sound source for further references to it

yaw: Yaw, expressed in radians. Positive to the right.

pitch: Pitch, expressed in radians. Positive upwards.

roll: Roll, expressed in radians. Positive to the right.

Return

Any confirmation message is sent back to the sender.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/orientation source1 0 0 0`

/SOURCE/GAIN

Sets source gain in dB. 0 dB indicates that the source is kept as it is read from the file or the input channel.

Syntax

```
/source/gain <string source_id> <float gain>
```

source_id: identifier assigned to the sound source for further references to it. If the source doesn't exist, the command is ignored.

gain: Gain, expressed in dB. the source, as read from the file or the input channel is multiplied by $10^{(gain/20)}$

Return

```
/control/actionResult /source/gain <string source_id> <bool setGain> <string description>
```

The return confirmation refers to the `source_id`, indicating `setGain=true` if the action has been successfully performed and `setGain=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/gain source1 -3`

BeRTA sends back to the sender: `/control/actionResult /source/gain source1 true Audio source source1 gain updated to -3 dB`

/SOURCE/ENABLEDIRECTIVITY

Switch on or off the directivity of the source. For directivity to work, a DirectivityTF must first be loaded from a SOFA file (`/resources/loadDirectivityTF`) and then assigned to the source (`/source/setDirectivity`).

Syntax

```
/source/enableDirectivity <string source_id> <boolean enable>
```

source_id: identifier assigned to the sound source. if this the source doesn't exist, the command is ignored

enable: if true (1) directivity is enabled. If false (0), directivity is disabled

Return

```
/control/actionResult /source/enableDirectivity <string source_id> <bool enableDirectivity> <string description>
```

The return confirmation refers to the `source_id`, indicating `enableDirectivity=true` if the action has been successfully performed and `enableDirectivity=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/enableDirectivity source1 true`

BeRTA sends back to the sender: `/control/actionResult /source/enableDirectivity source1 true Audio source source1 directivity enabled`

/SOURCE/SETDIRECTIVITY

Assigns a directivity which has been previously loaded using the command `/resources/setDirectivity`.

Syntax

```
/source/setDirectivity <string source_id> <string directivity_id>
```

source_id: identifier assigned to the sound source for further references to it

directivity_id: identifier of the directivity which was assigned when using the command `/resources/setDirectivity`

Return

```
/control/actionResult /source/setDirectivity <string
directivity_id> <bool setDirectivity> <string description>
```

The return confirmation refers to the `source_id`, indicating `setDirectivity=true` if the action has been successfully performed and `setDirectivity=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/setDirectivity source1 Directivity1`

BeRTA sends back to the sender: `/control/actionResult /source/setDirectivity Directivity1 true setDirectivity: Directivity1 has been set to SoundSource: source1`

/SOURCE/PLAYANDRECORD

Records a file of the specified duration with the delivered binaural sound and all the spatial information of one source and listener (location and orientation). The sound corresponds to the playback of this source from the beginning. The file includes the structure needed to create a SOFA file with the new [AnnotatedReceiverAudio](#) convention

Syntax

```
/source/playAndRecord <string source_id> <string filename>
<string type> <float seconds>
```

`source_id`: identifier of the source to be played back and recorded.

`string filename`: indicates the name of the file and must include the path, either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable. The extension will be added by the application if necessary. If there is a file with the same name, it'll not be overwritten, an ordinal number will be added to the end of the new file name.

`type`: indicates the extension of the file: **mat** (MATLAB binary data container) or **wav** (32-bit IEEE floating-point stereo).

`seconds`: indicates the duration of the recording. If seconds is -1 the recording will finish automatically when the source stops.

Return

```
/control/actionResult /source/playAndRecord <string
source_id> <bool recorded> <string description>
```

The return confirmation refers to the `source_id`, indicating `recorded=true` if the action has been successfully performed and `recorded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/source/playAndRecord SoundSource1 C:/tmp/testingRecord mat 3`

BeRTA sends back to the sender: `/control/actionResult /source/playAndRecord source1 true "Recording completed. File saved: C:/tmp/testingRecord"`

/SOURCE/RECORD

Available from BeRTA v3.7.0

Generates a file of the specified duration with the delivered binaural sound and all the spatial information of one source and listener (location and orientation), by processing the source from the beginning *without real-time playback*. This allows the operation to be executed significantly faster. The file includes the structure needed to create a SOFA file with the new [AnnotatedReceiverAudio](#) convention.

Syntax

```
/source/record <string source_id> <string filename> <string
type> <float seconds>
```

`source_id`: identifier of the source to be processed and recorded.

`filename`: indicates the name of the file and must include the path, either relative or absolute. If a relative path is used it will be calculated from the data folder that can be found in the same folder as the BeRTA executable. The extension will be added by the application if necessary. If there is a file with the same name, it'll not be overwritten—an ordinal number will be added to the end of the new file name.

`type`: indicates the extension of the file: **mat** (MATLAB binary data container) or **wav** (32-bit IEEE floating-point stereo).

`seconds`: indicates the duration of the recording. **This value must be greater than 0.** Unlike `/source/playAndRecord`, the special value -1 is *not* allowed.

Return

```
/control/actionResult /source/record <string source_id>
<bool recorded> <string description>
```

The return confirmation refers to the `source_id`, indicating `recorded=true` if the action has been successfully performed and `recorded=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers except the sender:

```
/source/record SoundSource1 C:/tmp/testingRecord mat 3
```

BeRTA sends back to the sender:

```
/control/actionResult /source/record SoundSource1 true
"Recording completed. File saved: C:/tmp/testingRecord"
```

or

```
/control/actionResult /source/record SoundSource1 false
"ERROR: Recording failed. File could not be created."
```

3.3.6 Listener commands

In this section, we describe the various OSC commands used to control the listener. These commands not only manage basic parameters like position and orientation but also define the models applied for simulation. These models come with a range of controllable parameters and can simulate either just the listener's head through a Head-Related Transfer Function (HRTF), or include the surrounding environment using a Binaural Room Impulse Response (BRIR), as the BRIR models the combined impulse response of a head situated within a room. See [listener models](#) for more information.

/LISTENER/LOCATION

Set location of listener. Position is set in global x,y,z coordinates, expressed in meters. The new location is sent to all remotes except the one has sent the message.

Syntax

```
/listener/location <string listener_id> <float x> <float y>
<float z>
```

listener_id: identifier assigned to the listener.

x: X global coordinate, expressed in metres. As a reference, X axis is positive to the front.

y: Y global coordinate, expressed in metres. As a reference, Y axis is positive to the left.

z: Z global coordinate, expressed in metres. As a reference, Z axis is positive to up.

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/location defaultListener 0 0 0`

/LISTENER/ORIENTATION

Sets orientation of the listener. Orientation is set in egocentric coordinates yaw, pitch and roll, expressed in radians and applied in that order (Yaw, Pitch and Roll). An orinetation of (0, 0, 0) corresponds to be oriented towards positive X. The new orientation is sent to all remotes except the one has sent the message.

Syntax

```
/listener/orientation <string listener_id> <float yaw>
<float pitch> <float roll>
```

listener_id: identifier assigned to the listener.

yaw: Yaw, expressed in radians. Positive to the right.

pitch: Pitch, expressed in radians. Positive upwards.

roll: Roll, expressed in radians. Positive to the right.

Return

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/orientation defaultListener 0.1 0 0`

/LISTENER/SETHRTF

Sets an HRTF using the HRTF id. The HRTF should be load previously using the command `/resources/loadHRTF`. After set the HRTF the command `listener/enableSpatialization` is called automatically. The HRTF is assigned to all the listener models which may accept it. See [listener models](#) for more information. Currently, these are the Listener models accepting it:

- [Listener Model based on Direct Convolution with HRTF](#).
- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)

Syntax

```
/listener/setHRTF <string listener_id> <string HRTF_id>
```

listener_id: identifier assigned to the listener.

HRTF_id: identifier assigned to the HRTF.

Return

```
/control/actionResult /listener/setHRTF <string listener_id>
<bool set> <string description>
```

The return confirmation refers to the `listener_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setHRTF defaultListener HRTF1`

BeRTA sends back to the sender: `/control/actionResult /listener/setHRTF defaultListener true "HRTF HRTF1 has been set to listener defaultListener"`

/LISTENER/SETBIR

Sets a BRIR using the BRIR id. The BRIR should be loaded previously using the command `/resources/loadBRIR`. After receiving this command `listener/enableSpatialization` is called automatically. The BRIR is assigned to all the models linked to the listener which may accept it. See the [Listener+Environment models](#) based on Room Impulse Responses for more information. These models currently are:

- [Listener + Environment Model based on Direct Convolution with BRIR](#).
- [Listener + Environment Model based on convolution with BRIR in the Ambisonic domain](#)

Syntax

```
/listener/setBRIR <string listener_id> <string BRIR_id>
```

`listener_id`: identifier assigned to the listener.

`BRIR_id`: identifier assigned to the BRIR.

Return

```
/control/actionResult /listener/setBRIR <string listener_id>
<bool set> <string description>
```

The return confirmation refers to the `listener_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setBRIR defaultListener BRIR1`

BeRTA sends back to the sender: `/control/actionResult /listener/BRIR1 defaultListener true "BRIR BRIR1 has been set to listener defaultListener"`

/LISTENER/SETSOSFILTERS

Sets a set of filters for the Near Field Compensation to be applied after the convolution with the HRTF. The filters are identified by an id, which was defined when they were previously loaded using the command `/resources/loadSOSFilters`. The NFC filters are assigned to all the models linked to the listener which may accept it. These currently are:

- [Listener Model based on Direct Convolution with HRTF](#).
- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)

Syntax

```
/listener/setSOSFilters <string listener_id> <string
NFCFilters_id>
```

`listener_id`: identifier assigned to the listener.

`NFCFilters_id`: identifier assigned to the Near Field Compensation filters.

Return

```
/control/actionResult /listener/setSOSFilters <string
listener_id> <bool set> <string description>
```

The return confirmation refers to the `listener_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setSOSFilters DefaultListener NFCFilters1`

BeRTA sends back to the sender: `/control/actionResult /listener/setSOSFilters defaultListener true "SOSFilters NFCFilters1 has been set to listener defaultListener"`

/LISTENER/ENABLESPATIALIZATION

In the case of a [Listener Model based on Direct Convolution with HRTF](#) and [Listener Model based on BRIR](#), this command switch on or off the simulation of the spatial audio. In case it is off, the sound is played without spatialization.

Syntax

```
/listener/enableSpatialization <string listener_id> <boolean
enable>
```

`listener_id`: identifier assigned to the listener.

`enable`: If true (1), enables spatialization for the listener. If false (0), spatialization is disabled.

Return

```
/control/actionResult /listener/enableSpatialization <string
listener_id> <bool enable> <string description>
```

The return confirmation refers to the `listener_id`, indicating `enable=true` if the spatialization has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableSpatialization DefaultListener true`

BeRTA sends back to the sender: `/control/actionResult /listener/enableSpatialization defaultListener true`
`"Spatialization enabled"`

/LISTENER/ENABLEINTERPOLATION

In the case of a [Listener Model based on Direct Convolution with HRTF](#) and [Listener Model based on BRIR](#), this command switch on or off the interpolation among HRIRs. In case it is on, a barycentric interpolation is applied among the three nearest HRIRs surrounding the source direction of arrival. See [HRTF grids](#) for more information. If there are several models based on direct convolution with HRTF linked to the listener, this command will affect all of them.

Syntax

```
/listener/enableInterpolation <string listener_id> <boolean enable>
```

`listener_id` : identifier assigned to the listener.

`enable` : If true (1), enables interpolation of the HRTF of the listener. If false (0), interpolation is disabled.

Return

```
/control/actionResult /listener/enableInterpolation <string listener_id> <bool enable> <string description>
```

The return confirmation refers to the `listener_id`, indicating `enable=true` if the model has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableInterpolation DefaultListener true`

BeRTA sends back to the sender: `/control/actionResult /listener/enableInterpolation defaultListener true`
`"Interpolation enabled to defaultListener"`

/LISTENER/ENABLENEARFIELDEFFECT

This command switches on or off the Near Field Compensation (NFC) applied together with the convolution with the HRTF. For the NFC to work, a set of NFC filters must first be loaded from a SOFA file (`/resources/loadNFCFilters`) and then assigned to the listener model (`/`

`listener/setNFCFilters`). If there are several models with the NFC feature linked to the listener, this command will affect all of them. The models with this feature currently are:

- [Listener Model based on Direct Convolution with HRTF](#).
- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)

Syntax

```
/listener/enableNearFieldEffect <string listener_id> <boolean enable>
```

`listener_id` : identifier assigned to the listener.

`enable` : If true (1), enables NFC. If false (0), NFC is disabled.

Return

```
/control/actionResult /listener/enableNearFieldEffect <string listener_id> <bool enable> <string description>
```

The return confirmation refers to the `listener_id`, indicating `enable=true` if the model has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableNearFieldEffect DefaultListener true`

BeRTA sends back to the sender: `/control/actionResult /listener/enableNearFieldEffect DefaultListener true`
`"NearField Effect enabled to DefaultListener"`

/LISTENER/ENABLEITD

This command switches on or off the simulation of ITD separate from the intrinsic ITD which could be included as initial delays in HRIRs. For the ITD simulation to make sense, HRIRs stored in the HRTF SOFA file should be aligned. Then the ITD can be extracted from the initial delays stored in the `Data.Delay` field of the SOFA structure. However, the ITD simulation can also use a spherical head model to synthesize ITD. See also the commands `/resources/enableWoodworthITD` and `/resources/setHRTFHeadRadius` for more information. If there are several models with the ITD

simulation feature, this command will affect all of them. The models with this feature currently are:

- [Listener Model based on Direct Convolution with HRTF](#).
- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)

Syntax

```
/listener/enableITD <string listener_id> <boolean enable>
```

`listener_id` : identifier assigned to the listener.

`enable` : If true (1), enables ITD. If false (0), ITD is disabled.

Return

```
/control/actionResult /listener/enableITD <string  
listener_id> <bool enable> <string description>
```

The return confirmation refers to the `listener_id`, indicating `enable=true` if the model has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableITD DefaultListener true`

BeRTA sends back to the sender: `/control/actionResult /
listener/enableITD DefaultListener true "ITD enabled to
DefaultListener"`

/LISTENER/ENABLEPARALLAXCORRECTION

This command switches on or off the parallax correction used to calculate the direction of arrival to each of the two ears. To do so, the head radius is needed and it is taken from the HRTF SOFA file or the radius provided by the `/resources/setHRTFHeadRadius` command. If there are several models with this parallax correction feature, this command will affect all of them. The models with this feature currently are:

- [Listener Model based on Direct Convolution with HRTF](#).
- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)

Syntax

```
/listener/enableParallaxCorrection <string listener_id>  
<boolean enable>
```

`listener_id` : identifier assigned to the listener.

`enable` : If true (1), enables parallax correction. If false (0), parallax correction is disabled.

Return

```
/control/actionResult /listener/enableITD <string  
listener_id> <bool enable> <string description>
```

The return confirmation refers to the `listener_id`, indicating `enable=true` if the model has been enabled and `enable=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableParallaxCorrection DefaultListener
true`

BeRTA sends back to the sender: `/control/actionResult /
listener/enableParallaxCorrection DefaultListener true
"Parallax Correction enabled to DefaultListener"`

/LISTENER/SETAMBISONICSORDER

Sets the order of the Ambisonic encoding used by the listener model. The listener model is identified by an identifier defined in the used [settings file](#). This command can be only used with models based on Ambisonics. If it is referred to a different model it will be ignored. The models with this feature currently are:

- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)
- [Listener + Environment Model based on convolution with BRIR in the Ambisonic domain](#)

Syntax

```
/listener/setAmbisonicsOrder <string listenerModel_id> <int  
ambisonicsOrder>
```

`listenerModel_id` : identifier assigned to the model.

`ambisonicsOrder` : Ambisonic order. Possible values are 1, 2 and 3.

Return

```
/control/actionResult /listener/setAmbisonicsOrder <string  
listenerModel_id> <bool set> <string description>
```

The return confirmation refers to the `listener_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setAmbisonicsOrder DirectPath 3`

BeRTA sends back to the sender: `/control/actionResult /listener/setAmbisonicsOrder DirectPath true "Ambisonics order set to 3"`

/LISTENER/SETAMBISONICSNORMALIZATION

Sets the normalization type used in the Ambisonic encoding used by the listener model. The listener model is identified by an identifier defined in the used [settings file](#). This command can be only used with models based on Ambisonics. If it is referred to a different model it will be ignored. The models based on Ambisonics currently are:

- [Listener Model based on convolution with HRTF in the Ambisonic domain](#)
- [Listener + Environment Model based on convolution with BRIR in the Ambisonic domain](#)

Syntax

```
/listener/setAmbisonicsNormalization <string
listenerModel_id> <string ambisonicsNormalization>
```

`listenerModel_id`: identifier assigned to the model.

`ambisonicsNormalization`: Ambisonic normalization type. Possible values are: `N3D`, `SN3D`, `maxN`.

Return

```
/control/actionResult /listener/setAmbisonicsNormalization
<string listenerModel_id> <bool set> <string description>
```

The return confirmation refers to the `listener_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setAmbisonicsNormalization ReverbPath N3D`

BeRTA sends back to the sender: `/control/actionResult /listener/setAmbisonicsNormalization DirectPath true "Ambisonics normalization set to N3D"`.

/LISTENER/ENABLEDISTANCEATTENUATION

Available from BeRTA v3.6.0

This command enables or disables the distance simulation in the listener model, *only available in the combined listener/environment models*. This simulation consists in applying a global attenuation, where doubling the distance between the source and the listener reduces the sound level by a predefined attenuation value.

Syntax

```
/listener/enableDistanceAttenuation <string
listenerModel_id> <boolean enable>
```

`listenerModel_id`: identifier assigned to the model.

`enable`: If true (1), enables the simulation of the distance attenuation. If false (0), disable the simulation of the distance attenuation.

Return

```
/control/actionResult /listener/enableDistanceAttenuation
<string listenerModel_id> <bool enabled> <string
description>
```

The return confirmation refers to the `listenerModel_id`, indicating `enabled=true` if the distance attenuation has been enabled and `enabled=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/enableDistanceAttenuation ReverbPath true`

BeRTA sends back to the sender: `/control/actionResult /listener/enableDistanceAttenuation ReverbPath true "Distance attenuation enabled in the model (ReverbPath)".`

/LISTENER/SETDISTANCEATTENUATIONFACTOR

Available from BeRTA v3.6.0

This command allows you to change the attenuation factor per distance, in decibels. This is the amount of attenuation applied to the signal each time the distance is doubled from the reference distance. By default, this attenuation has a value of -3 dB and the default reference distance is 1 meter. The value to set must always be negative. For more information, see the [section](#) on the listener and environment combined models.

Syntax

```
/listener/setDistanceAttenuationFactor <string  
listenerModel_id> <float distanceAttenuationFactor>
```

`listenerModel_id`: identifier assigned to the model.

`distanceAttenuationFactor`: a float value, expressed in dB, representing the distance attenuation factor. This value must be negative.

Return

```
/control/actionResult /listener/setDistanceAttenuationFactor  
<string listenerModel_id> <float distanceAttenuationFactor>  
<string description>
```

The return confirmation refers to the `listenerModel_id`, indicating if the attenuation factor

`distanceAttenuationFactor` has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/listener/setDistanceAttenuationFactor ReverbPath -3`

BeRTA sends back to the sender: `/control/actionResult /listener/setDistanceAttenuationFactor ReverbPath true "Distance Attenuation Factor updated in the model (ReverbPath) to -3".`

3.3.7 Environment commands

This section covers the OSC commands responsible for / controlling the environment. These commands handle parameters related to the characteristics of the space, such as room geometry and the absorption profiles of the walls, particularly in models based on geometrical acoustics.

/ENVIRONMENT/ENABLEDIRECTPATH

~~This command switches on or off the virtual source corresponding to the direct path. Disabling it will produce silence at its output. This feature is only implemented by some models.~~

Syntax

```
/environment/enableDirectPath <string environmentModel_id>
<boolean enable>
```

`environmentModel_id` : identifier assigned to the model.

`enable` : If true (1), enables the direct path source. If false (0), mutes the direct path source.

Return

```
/control/actionResult /environment/enableDirectPath <string
environmentModel_id> <bool enabled> <string description>
```

The return confirmation refers to the `environmentModel_id`, indicating `enabled=true` if the direct path has been enabled and `enabled=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/enableDirectPath SDN false`

BeRTA sends back to the sender: `/control/actionResult /environment/enableDirectPath SDN true "Environment model SDN Direct path enabled"`

/ENVIRONMENT/ENABLEREVERBPATH

~~This command switches on or off all virtual sources related to reverberation; it does not apply to the direct path. When deactivated, all outputs will be silent. This function can be implemented in all environment models.~~

Syntax

```
/environment/enableReverbPath <string environmentModel_id>
<boolean enable>
```

`environmentModel_id` : identifier assigned to the model.

`enable` : If true (1), enables the reverb path sources. If false (0), mutes all reverb path sources.

Return

```
/control/actionResult /environment/enableReverbPath <string
environmentModel_id> <bool enabled> <string description>
```

The return confirmation refers to the `environmentModel_id`, indicating `enabled=true` if the model direct path has been enabled and `enabled=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/enableReverbPath SDN false`

BeRTA sends back to the sender: `/control/actionResult /environment/enableReverbPath SDN true "Environment model SDN Reverb path enabled"`.

/ENVIRONMENT/ENABLEPROPAGATIONDELAY

~~This command enables or disables the simulation of propagation delay throughout the entire environment, allowing users to control whether or not the system accounts for delays in signal propagation during audio rendering.~~

Syntax

```
/environment/enablePropagationDelay <string
environmentModel_id> <boolean enable>
```

`environmentModel_id` : identifier assigned to the model.

`enable` : If true (1), enables the simulation of the propagation delay. If false (0), disable the simulation of the propagation delay.

Return

```
/control/actionResult /environment/enablePropagationDelay
<string environmentModel_id> <bool set> <string
description>
```

The return confirmation refers to the `environmentModel_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/enablePropagationDelay FreeFieldEnvironmentModel true`

BeRTA sends back to the sender: `/control/actionResult /environment/enablePropagationDelay FreeFieldEnvironmentModel true "Propagation delay in the FreeFieldEnvironmentModel model enabled."`

/ENVIRONMENT/ENABLEDISTANCEATTENUATION

~~Compatible models: FreeField~~ Available from BeRTA v2.0.0
This command enables or disables the distance simulation in the environment model, if available. This simulation consists in applying a global attenuation, where doubling the distance between the source and the listener reduces the sound level by a predefined attenuation value.

Syntax

`/environment/enableDistanceAttenuation <string environmentModel_id> <boolean enable>`

`environmentModel_id`: identifier assigned to the model.

`enable`: If true (1), enables the simulation of the distance attenuation. If false (0), disable the simulation of the distance attenuation.

Return

`/control/actionResult /environment/enableDistanceAttenuation <string environmentModel_id> <bool set> <string description>`

The return confirmation refers to the `environmentModel_id`, indicating `set=true` if the action has been successfully performed and `set=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/enableDistanceAttenuation FreeFieldEnvironmentModel true`

BeRTA sends back to the sender: `/control/actionResult /environment/enableDistanceAttenuation FreeFieldEnvironmentModel true "Distance attenuation enabled in the model (FreeFieldEnvironmentModel)."`

/ENVIRONMENT/SETDISTANCEATTENUATIONFACTOR

~~Compatible models: FreeField~~ Available from BeRTA v2.0.0
This command allows you to change the attenuation factor per distance, in decibels. This is the amount of attenuation applied to the signal each time the distance is doubled from

the reference distance. By default, this attenuation has a value of -6.0206 dB and the default reference distance is 1 meter. The value to set must always be negative. For more information, see the [section](#) on the environment model.

Syntax

`/environment/setDistanceAttenuationFactor <string environmentModel_id> <float distanceAttenuationFactor>`

`environmentModel_id`: identifier assigned to the model.

`distanceAttenuationFactor`: a float value, expressed in dB, representing the distance attenuation factor. This value must be negative.

Return

`/control/actionResult /environment/setDistanceAttenuationFactor <string environmentModel_id> <bool setAttenuation> <string description>`

The return confirmation refers to the `environmentModel_id`, indicating if the attenuation factor has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/setDistanceAttenuationFactor FreeField -3`

BeRTA sends back to the sender: `/control/actionResult /environment/setDistanceAttenuationFactor FreeField true "Distance Attenuation Factor updated in the model (FreeField) to -3"`

/ENVIRONMENT/SETFADEZONEMARGIN

~~Compatible models: ISM~~ Available from BeRTA v2.0.0
This command allows you to change the size of the fade zone used by the model for simulation. This value is used to fade out samples from virtual sources that enter that zone. Its value must be a positive number and is expressed in metres. By default, this parameter has a value of 0.6m (equivalent to 2ms). For more information, see the section on environment models, for example, the [ISM](#).

Syntax

`/environment/setFadeZoneMargin <string environmentModel_id> <float fadeZoneMargin>`

`environmentModel_id`: identifier assigned to the model.

`fadeZoneMargin`: a float value, which represents the size of the fading zone used in the model simulation. This value must be positive and is expressed in metres.

Return

```
/control/actionResult /environment/setFadeZoneMargin <string
environmentModel_id> <bool set> <string description>
```

The return confirmation refers to the `environmentModel_id`, indicating if the maximum distance between the sources and the listener has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/setFadeZoneMargin ISM 10.5`

BeRTA sends back to the sender: `/control/actionResult /environment/setFadeZoneMargin ISM true "Max Distance Sources to Listener updated in the model (ISM) to 10.5m."`

/ENVIRONMENT/SETMAXDISTANCESOURCESTOLISTENER

~~This command allows you to change the maximum distance between the sources and the listener used by the model for the simulation. This value is used to prune the number of image sources that are simulated. Its value must be a positive number and is expressed in metres. By default, this parameter has a value of 3.43m. For more information, see the section on environment models, for example, the [ISM](#).~~

Syntax

```
/environment/setMaxDistanceSourcesToListener <string
environmentModel_id> <float maxDistanceSourcesToListener>
environmentModel_id : identifier assigned to the model.

maxDistanceSourcesToListener : a float value, representing
the maximum distance between the sources and the listener
to be used by the model simulation. This value must be
positive.
```

Return

```
/control/actionResult /environment/
setMaxDistanceSourcesToListener <string environmentModel_id>
<bool set> <string description>
```

The return confirmation refers to the `environmentModel_id`, indicating if the maximum distance between the sources and the listener has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/setMaxDistanceSourcesToListener ISM 10.5`

BeRTA sends back to the sender: `/control/actionResult /environment/setMaxDistanceSourcesToListener ISM true "Max Distance Sources to Listener updated in the model (ISM) to 10.5m."`

/ENVIRONMENT/SETREFLECTIONORDER

~~This command allows you to change the reflection order used by the model for simulation. Its value determines the number of virtual sources used in the simulation. Its value must be a positive integer. By default, the reflection order is set to zero in the model, which means that the simulation is not performed. For more information, see the section on environment models, for example the [ISM](#).~~

Syntax

```
/environment/setReflectionOrder <string environmentModel_id>
<int reflectionOrder>

environmentModel_id : identifier assigned to the model.

reflectionOrder : a int value, representing the reflection
order to be used by the model simulation. This value must
be positive.
```

Return

```
/control/actionResult /environment/setReflectionOrder
<string environmentModel_id> <bool set> <string
description>
```

The return confirmation refers to the `environmentModel_id`, indicating if the reflection order has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/setReflectionOrder ISM 3`

BeRTA sends back to the sender: `/control/actionResult /environment/setReflectionOrder ISM true "Reflection Order updated in the model (ISM) to 3."`

/ENVIRONMENT/SETRROOM

~~Establishes a room in an environment model. The definition of the room and the model are referenced by two identifiers. The room must have been previously loaded using a command of the class `/resources/....` After~~

establishing the room, the environment model is reconfigured accordingly, which may require heavy processing and cause rendering to be interrupted. A room can be assigned to more than one environment model at a time. See environment models for more information. Currently, these are the listener models that accept it:

- **SND Environment Model**
- **ISM Model**

Syntax

```
/environment/setRoom <string environmentModel_id> <string room_id>
```

`environmentModel_id`: identifier assigned to the model.

`room_id`: identifier assigned to a room definition.

Return

```
/control/actionResult /environment/setRoom <string room_id>
<bool set> <string description>
```

The return confirmation refers to the `room_id`, indicating if the room definition has been set or not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/environment/setRoom SDN room1`

BeRTA sends back to the sender: `/control/actionResult /environment/setRoom SDN false "ERROR setting Room to the environment room1 not found in the Room list']`.

3.3.8 Bilateral Filter commands

This section covers the OSC commands responsible for controlling the binaural filters using second order section filters.

/BINAURALFILTER/SETSOSFILTER

~~This command sets the resource (the Second Order Section filters) to the simulation of the binaural filter model.~~
Deprecated since BeRTA v3.10.0, use '/binauralFilter/setFilter' instead.

Syntax

```
/binauralFilter/setSOSFilter <string model_id> <string
sosfilter_id>

model_id : identifier assigned to the model.

sosfilter_id : identifier assigned to the SOS filter resource.
```

Return

```
/control/actionResult /binauralFilter/setSOSFilter <string
model_id> <bool success> <string description>
```

The return confirmation refers to the `model_id`, indicating `success=true` if the set has been done successfully and `enabled=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

Example

BeRTA receives and echoes back to all subscribers but the sender: `/binauralFilter/setSOSFilter DefaultListener Earmuffs`

BeRTA sends back to the sender: `/control/actionResult /binauralFilter/setSOSFilter DefaultListener true "SOS`

```
Filters (Earmuffs) has been set to listener
(DefaultListener)"
```

/BILATERALFILTER/SETFILTER

~~This command sets the filter coefficient database for simulating the binaural filter model. It works both for setting the filter coefficients of the second-order section in SOS bilateral filters and the impulse response table in FIR bilateral filters.~~
Compatible models: BilateralSOS and BilateralFIR. Available from BeRTA v3.10.0

Syntax

```
/bilateralFilter/setFilter <string bilateral_model_id>
<string filter_coefs_id>

bilateral_model_id : identifier assigned to the bilateral filter
model.

filter_coefs_id : identifier assigned to the SOS filter coefs
or FIR table resource.
```

Return

```
/control/actionResult /bilateralFilter/setFilter <string
bilateral_model_id> <bool success> <string description>
```

The return confirmation refers to the `bilateral_model_id`, indicating `success=true` if the set has been done successfully and `enabled=false` if not. In both cases a `description` is added to give more details.

In case of success, an echo is sent to all subscribers except the sender, using the same syntax as the received message.

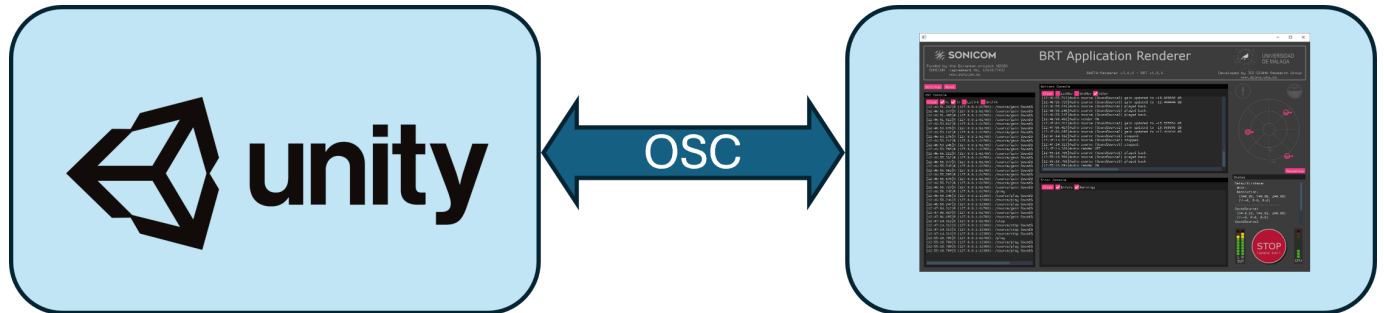
Example

BeRTA receives and echoes back to all subscribers but the sender: `/bilateralFilter/setFilter PhoneEQ FIRFilter1`

BeRTA sends back to the sender: `/control/actionResult /bilateralFilter/setFilter FIRFilter1 true "FIR Filters (FIRFilter1) has been set to model (PhoneEQ)"`

3.4 BRT OSC API for Unity - Integration Package

This package enables the integration of a Unity scene with applications from the BRT library ([BRT Application](#)) using OSC commands. It can be download from our [repository](#).



OSC communication between Unity and BRT Application

3.4.1 Package Contents

`BRTManager.cs`

A dual-purpose class designed to:

- Serve as an example of how to use the package.
- It can be used and extended by anyone, as it implements some important OSC commands.

`BRTCommands.cs`

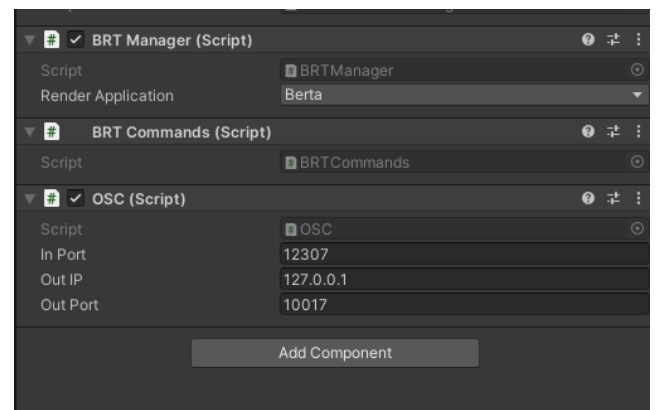
This class handles the implementation, sending, and receiving of OSC commands.

Note: This class is still under development. Bugs may be present, and not all commands from the BRT interface are currently implemented. For more details about the available OSC commands, refer to the [BRT OSC Documentation](#).

`OSC.cs`

A third-party library that implements the OSC protocol. Configure the IP and port for the remote application and the listening port for the Unity app in this script.

For the scripts described above to work, they must be added to the same object, as illustrated in the following image.



Scripts imported in Unity.

`BRTAudioListener.cs`

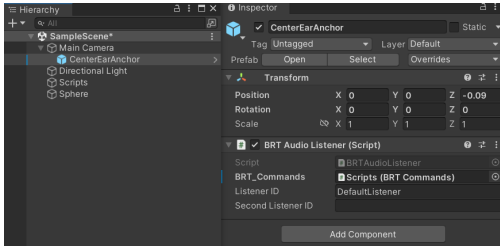
This script sends the listener's position and orientation to the BRT renderer via OSC commands.

To use this:

- Attach the script to your scene's camera object, or Instantiate our prefab (`CenterEarAnchor.prefab`) in the scene, as a child of a camera object.
- Enter the listener ID as defined in the BeRTA Renderer configuration file, see [section](#).

`CenterEarAnchor.prefab`

A prefab designed to be placed under the "center eye" object in a virtual reality scene. Its purpose is to apply an offset, positioning it virtually at the center of the listener's head. This prefab also includes the `BRTAudioListener.cs` script.



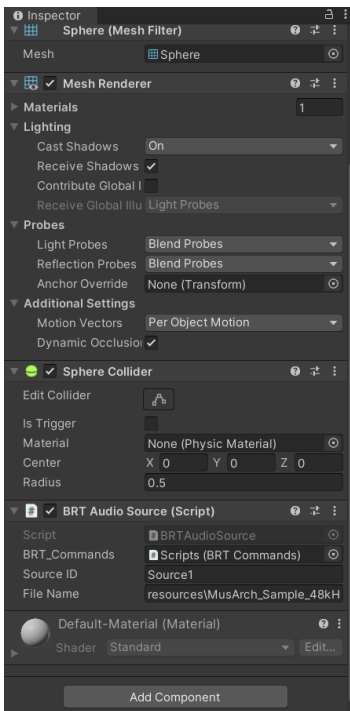
CenterEarAnchor prefab in the Unity inspector.

BRTAudioSource.cs

This script is designed to communicate the position and orientation of a virtual sound source. It must be installed on objects representing sound sources in the Unity virtual world. These values are sent every time they change. It will also load the source in BeRTA Renderer at startup.

To use this:

- Attach the script to your scene's objects
- Assign a unique ID
- Enter the full name of the audio file that the BeRTA Renderer should load.



BRTAudioSource.cs

3.4.2 Getting Started: Basic Unity Setup

This guide explains how to set up a basic Unity scene that communicates with **BeRTA** using our Unity integration

package. The setup includes **one static audio source** and **one listener**.

Step-by-Step Guide

1. IMPORT THE UNITY PACKAGE

Import the Unity package provided for BeRTA integration into your Unity project.

2. CREATE THE CORE BRT GAMEOBJECT

- In your Unity scene, create an **empty GameObject**.
- Add the following scripts as components:
 - BRTManager.cs
 - BRTCommands.cs
 - OSC.cs
- Open the `osc.cs` component and configure:
 - The **remote IP and port** to match the BeRTA application.
 - The **local listening port** for incoming OSC messages.

3. CREATE AN AUDIO SOURCE AND ASSIGN BRTCOMMANDS

- Create a new **GameObject** to act as your **audio source**.
- In the script attached to this source (e.g., `BRTAudioSource.cs`), **locate the BRTCommands field and drag the GameObject containing BRTCommands.cs into it**.
- In the audio file field, enter the absolute path to the .wav file you want BeRTA to play.

4. CONFIGURE THE LISTENER AND ASSIGN BRTCOMMANDS

- Drag the `CenterEarAnchor.prefab` into your scene as a child of the **Main Camera**.
- This prefab includes the `BRTAudioListener.cs` script and positions the listener at the center of the user's head.
- In the `BRTAudioListener.cs` script, **locate the BRTCommands field and drag the GameObject containing BRTCommands.cs into it**.

5. LAUNCH BERTA

- Open and run the **BeRTA application**.
- With Unity and BeRTA running, the **OSC connection will be established**, and audio will be rendered in real time.

Notes

- `BRTManager.cs`, `BRTCommands.cs`, and `OSC.cs` must be attached to the **same GameObject**.
- Both the **audio source** and the **listener prefab** must reference this GameObject through their `BRTCommands` field.

- Ensure your network settings and firewall allow OSC communication.

3.4.3 License

Except for the `OSC.cs` class, which was not developed by us, all other scripts in this package are distributed under the MIT license.

3.5 BRT OSC API for Matlab

Section under construction